



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Escuela Técnica Superior de Ingeniería Informática

Automatización de la Gestión del Conocimiento
Empresarial mediante Workflows y Agentes Inteligentes

Trabajo Fin de Grado

Grado en Ingeniería Informática

AUTOR/A: Farsane Farsane, Ziad

Tutor/a: Julian Inglada, Vicente Javier

CURSO ACADÉMICO: 2024/2025

A mis padres y a mi hermana, por su amor, su apoyo incondicional, su paciencia infinita y por estar siempre a mi lado, en cada paso del camino.

Agradecimientos

Quiero expresar mi más sincero agradecimiento a todas las personas que han contribuido de forma directa o indirecta al desarrollo de este Trabajo de Fin de Grado.

En primer lugar, agradezco profundamente a mis padres y a mi hermana por su apoyo constante, su paciencia y su confianza incondicional durante todo este proceso académico y personal.

Agradezco especialmente a mi tutor académico, Vicente Javier Julián Inglada, por su orientación, seguimiento y compromiso continuo a lo largo del desarrollo del trabajo.

Asimismo, deseo expresar mi gratitud a Federico López Esquibel por brindarme la oportunidad de colaborar en un entorno profesional real. Su implicación, asesoramiento técnico y disposición han sido claves para llevar a cabo este proyecto con una orientación práctica y aplicada.

Finalmente, quiero dar las gracias a mis compañeros y amigos por acompañarme durante estos años, por las risas, el apoyo mutuo y por hacer que este camino haya sido mucho más llevadero y enriquecedor, dentro y fuera del aula.

Resumen

En la actualidad, las empresas manejan grandes volúmenes de información, lo que dificulta su acceso eficiente y organizado. La búsqueda manual de documentos y datos internos genera una pérdida de tiempo considerable, afectando la productividad y ralentizando la toma de decisiones. Este Trabajo de Fin de Grado tiene como objetivo desarrollar un sistema de automatización de la gestión del conocimiento empresarial, basado en workflows inteligentes y agentes inteligentes. Para ello, se implementará un sistema que consta de dos workflows principales: un workflow de indexación, encargado de procesar e indexar documentos provenientes de diferentes fuentes (archivos locales, servicios en la nube, y bases de datos empresariales), y un workflow de recuperación, diseñado para responder consultas en lenguaje natural mediante la técnica Retrieval-Augmented Generation (RAG). El sistema integrará LlamaIndex y modelos de lenguaje avanzados (LLMs) para optimizar la segmentación de documentos, la extracción de metadatos y la generación de respuestas coherentes y fundamentadas. Además, se utilizará una base de datos estructurada para almacenar los documentos indexados, facilitando su recuperación eficiente. La solución propuesta permitirá a las empresas mejorar la gestión de su conocimiento interno, reduciendo el tiempo de búsqueda de información y optimizando la toma de decisiones. Su implementación proporcionará una base escalable y adaptable a diferentes entornos empresariales, contribuyendo al desarrollo de nuevas estrategias de automatización en la gestión de datos.

Palabras clave: automatización; gestión del conocimiento; inteligencia artificial; workflows; agentes inteligentes; indexación; Retrieval-Augmented Generation (RAG); modelos de lenguaje.

Abstract

Nowadays, companies manage large volumes of information, making efficient access and organization increasingly challenging. The manual search for internal documents and data results in significant time loss, reducing productivity and slowing down decision-making processes. This Final Degree Project aims to develop a system for automating business knowledge management, based on intelligent workflows and AI agents. To achieve this, a system consisting of two main workflows will be implemented: an indexing workflow, responsible for processing and indexing documents from various sources (local files, cloud services, enterprise databases), and a retrieval workflow, designed to respond to natural language queries using the Retrieval-Augmented Generation (RAG) technique. The system will integrate LlamaIndex and advanced language models (LLMs) to optimize document segmentation, metadata extraction, and the generation of coherent and well-founded responses. Additionally, a structured database will be used to store indexed documents, ensuring efficient retrieval. The proposed solution will enhance business knowledge management, reducing the time spent searching for information and improving decision-making processes. Its implementation will provide a scalable and adaptable foundation for different business environments, contributing to the development of new automation strategies in data management.

Keywords : automation; knowledge management; artificial intelligence; workflows; intelligent agents; indexing; Retrieval-Augmented Generation (RAG); language models.

Índice de contenidos

Índice de figuras.....	9
Índice de tablas.....	9
1. Introducción.....	11
1.1. Motivación.....	12
1.2. Objetivos.....	12
1.3. Impacto Esperado.....	13
1.4. Estructura del documento.....	14
2. Estado del Arte y Fundamentos Teóricos.....	16
2.1. Evolución tecnológica en la gestión del conocimiento empresarial.....	16
2.2. Introducción a los sistemas RAG y su impacto en la recuperación de información.....	17
2.3. El uso de workflows en el ámbito de la Inteligencia Artificial.....	19
2.4. Herramientas para la creación de workflows.....	21
2.5. Estudio de herramientas RAG.....	24
2.6. Conclusiones.....	28
3. Presentación de la propuesta y fundamentos técnicos.....	29
3.1. Problema identificado y motivación técnica.....	29
3.2. Sistema propuesto: visión general y arquitectura funcional.....	30
3.2.1. Workflow de indexación.....	31
3.2.2. Workflow de recuperación.....	32
3.3. Fundamentos técnicos sobre LlamaIndex y arquitectura basada en workflows.....	34
3.3.1. ¿Qué es LlamaIndex y por qué se utiliza? Ventajas técnicas de su arquitectura...	34
3.3.2. Arquitectura y ejecución de workflows en LlamaIndex.....	35
3.4. Configuración modular y parametrización del workflow.....	36
3.4.1. Inicialización con from_config: principio de configuración como código.....	37
3.4.2. Estructura del bloque settings: definición del flujo de ejecución.....	38
3.4.3. Estructura del bloque agents: definición de agentes inteligentes.....	40
3.4.4. Personalización del flujo según fuente o consulta.....	44
3.5. Conclusiones.....	44
4. Diseño e implementación del workflow de Indexación.....	46
4.1. Arquitectura del workflow de indexación.....	46
4.1.1. Estructura de la clase IndexingWorkflow.....	46
4.1.2. Diagrama del flujo completo de ejecución.....	47
4.1.3. Encaje dentro del sistema global.....	48
4.2. Descripción técnica y funcional de cada etapa.....	49
4.2.1. Parsing de referencias (parse_source_references).....	49
4.2.2. Extracción de nodos (nodes_extraction_from_references).....	50
4.2.3. Filtrado de nodos (nodes_filtering).....	51
4.2.4. Procesamiento avanzado de nodos (nodes_processing).....	52
4.2.5. Transformación de nodos (nodes_transformation).....	53
4.2.6. Almacenamiento en base de datos (add_nodes_to_db).....	55
4.2.7. Postprocesamiento final (post_indexing_process).....	55

4.3. Diseño extensible y desacoplado por herramientas.....	56
4.4. Funcionamiento del workflow de indexación: ejemplo y traza del proceso.....	57
4.5. Conclusiones.....	58
5. Diseño e implementación del workflow de Recuperación.....	60
5.1. Arquitectura del workflow de recuperación.....	60
5.1.1. Estructura de la clase RAGWorkflow.....	60
5.1.2. Diagrama del flujo completo de ejecución.....	62
5.1.3. Encaje dentro del sistema global.....	63
5.2. Descripción técnica y funcional de cada etapa.....	63
5.2.1. Reformulación y distribución de la consulta (rewrite_query).....	63
5.2.2. Selección de mecanismos de recuperación (query_router_step).....	64
5.2.3. Recuperación de información (recursive_retrieval, vector_retrieval, hierarchical_retriever).....	65
5.2.4. Reranking de resultados (cohere_rerank).....	66
5.2.5. Enriquecimiento mediante resúmenes y secciones completas (summary_and_full_section_nodes).....	67
5.2.6. Añadido de nodos relacionados internos (internal_related_nodes).....	68
5.2.7. Síntesis de respuestas (normal_synthesizer, response_synthesizer_agent).....	69
5.2.8. Evaluación de fidelidad de las respuestas (faithfulness_evaluation).....	70
5.2.9. Consolidación de respuestas (consolidate_synthesizing).....	70
5.2.10. Crítica y revisión final de la respuesta (response_critic).....	71
5.3. Configuraciones evaluadas del workflow de recuperación.....	72
5.4. Funcionamiento del workflow de recuperación: ejemplo y traza del proceso.....	73
5.5. Conclusiones.....	74
6. Evaluación y Resultados.....	76
6.1. Metodología de evaluación.....	76
6.1.1. Casos de uso definidos.....	76
6.1.2. Agente evaluador y criterios de evaluación.....	77
6.1.3. Metodología de evaluación.....	79
6.2. Caso de uso 1: Consulta sobre documentos legales y normativos.....	80
6.3. Caso de uso 2: Recuperación de URLs de documentos.....	81
6.4. Caso de uso 3: Análisis y recuperación de información en correos y conversaciones... ..	82
6.5. Caso de uso 4: Consulta sobre código fuente y funciones técnicas.....	83
6.6. Caso de uso 5: Indexación de fichas y catálogos de productos.....	84
6.7. Análisis comparativo y discusión.....	86
6.8. Conclusiones.....	88
7. Conclusiones y Futuras Líneas de Trabajo.....	90
7.1. Conclusiones generales.....	90
7.2. Relación del trabajo desarrollado con los estudios cursados.....	91
7.3. Trabajos futuros.....	92
Referencias:.....	94
Anexo A: Extractos representativos del código:.....	96
Extracto 1: Configuración del workflow de recuperación.....	96
Extracto 2: Inicialización dinámica del workflow mediante from_config.....	98
Extracto 3: Definición de etapas en el workflow mediante eventos y agentes.....	100

Extracto 4: Recuperación, síntesis y evaluación de respuestas.....	101
Extracto 5: Diseño y uso de prompts en el sistema.....	102
Anexo B: Objetivos de desarrollo sostenible (ODS).....	109
Glosario:.....	111

Índice de figuras

Figura 1. Funcionamiento general de un sistema RAG, combinando recuperación de información y generación de texto mediante LLMs.....	18
Figura 2. Arquitectura general del sistema propuesto basado en dos workflows interconectados.....	30
Figura 3. Arquitectura funcional general del workflow de indexación.....	32
Figura 4. Arquitectura general del workflow de recuperación.....	33
Figura 5. Estructura general de la configuración de un workflow.....	38
Figura 6. Ejemplo de configuración avanzada de un agente inteligente.....	42
Figura 7. Diagrama de ejecución del workflow de indexación.....	48
Figura 8. Arquitectura general del workflow de recuperación (RAG).....	62
Figura 9. Comparativa de puntuaciones obtenidas por configuración en los cinco casos de uso.....	87
Figura 10. Comparativa del tiempo medio de ejecución por configuración en los cinco casos de uso.....	88
Figura 11. Ejemplo simplificado de configuración de un workflow de recuperación.....	97
Figura 12. Inicialización del workflow a partir de una configuración usando el metodo from_config.....	99
Figura 13. Ejemplo de definición de etapa en un workflow.....	100
Figura 14. Ejecución paralela de tareas de recuperación de nodos.....	101
Figura 15. Ejecución paralela de tareas de generación de respuestas.....	101
Figura 16. Evaluación de fidelidad de respuestas generadas.....	102
Figura 17. Prompt del agente sintetizador de respuestas (response_synthesizer).....	103
Figura 18. Prompt del agente reformulador de consultas (query_rewriter).....	104
Figura 19. Prompt del agente evaluador de respuestas (evaluator_agent).....	108

Índice de tablas

Tabla 1. Comparativa técnica de herramientas para la creación de workflows.....	22
Tabla 2. Comparativa funcional entre el sistema RAG propuesto y las soluciones existentes....	26
Tabla 3. Comparativa de parámetros settings en los workflows de indexación y recuperación..	40
Tabla 4. Agentes inteligentes del sistema y su rol en los distintos workflows.....	43
Tabla 5. Resultados de la evaluación de configuraciones del caso de uso 1.....	80
Tabla 6. Resultados de la evaluación de configuraciones del caso de uso 2.....	81
Tabla 7. Resultados de la evaluación de configuraciones del caso de uso 3.....	82
Tabla 8. Resultados de la evaluación de configuraciones del caso de uso 4.....	84
Tabla 9. Resultados de la evaluación de configuraciones del caso de uso 5.....	85
Tabla 10: Tabla relación con ODS.....	109

1. Introducción

En la actualidad, el volumen de información que manejan las empresas crece exponencialmente, lo que dificulta su organización y acceso eficiente. En muchas organizaciones, la búsqueda de información relevante en documentos internos, correos electrónicos y bases de datos resulta un proceso lento y manual, generando ineficiencias y retrasos en la toma de decisiones. Para abordar esta problemática, se plantea el desarrollo de una solución basada en workflows (flujos de trabajo) de indexación y recuperación de información “Retrieval-Augmented Generation” (RAG), utilizando la herramienta LlamaIndex y técnicas avanzadas de Inteligencia Artificial (IA).

Este trabajo se centra en la creación de un sistema de automatización del conocimiento empresarial, compuesto por dos workflows principales. El primero es un workflow de indexación, encargado de procesar e indexar documentos provenientes de distintas fuentes, como archivos locales, servicios en la nube, y bases de datos. Para ello, se implementan diversas técnicas de preprocesamiento, incluyendo segmentación en nodos, extracción de metadatos y generación de resúmenes mediante agentes inteligentes.

El segundo workflow es el de recuperación de información, cuyo objetivo es responder a consultas de los usuarios con respuestas coherentes, fundamentadas y enriquecidas con información relevante. Para lograrlo, el sistema identifica y recupera los fragmentos de información más relevantes de los documentos previamente indexados y los sintetiza para construir una respuesta precisa. Gracias a esta arquitectura, la respuesta generada no solo es coherente con la consulta, sino que se basa en un conocimiento preexistente, evitando alucinaciones y proporcionando información verificable. Este enfoque permite que las empresas accedan rápidamente a su conocimiento estructurado, optimizando la toma de decisiones y reduciendo la necesidad de realizar búsquedas manuales.

La solución propuesta no solo permite automatizar la gestión documental de las empresas, sino que también mejora la eficiencia operativa al reducir el tiempo que los empleados invierten en la búsqueda de información. Además, la integración de agentes inteligentes especializados en distintas etapas del proceso permite optimizar tareas clave como la clasificación de documentos, la generación de consultas mejoradas y la síntesis de respuestas. A través del uso de modelos de lenguaje avanzados, ingeniería de prompts y la integración con APIs de terceros, se consigue una plataforma flexible, adaptable a distintos entornos empresariales y escalable para futuras mejoras.

1.1. Motivación

El acceso eficiente a la información sigue siendo un desafío clave para muchas organizaciones, especialmente en entornos donde el conocimiento se encuentra distribuido en múltiples fuentes y formatos. La recuperación manual de datos relevantes, ya sea desde documentos internos, bases de datos o plataformas colaborativas, representa una carga operativa que impacta directamente en la productividad y en la toma de decisiones.

Este proyecto surge con el objetivo de abordar este problema mediante el desarrollo de un sistema inteligente que automatice los procesos de indexación y recuperación de información. A través de la aplicación de técnicas avanzadas de Inteligencia Artificial, como los sistemas de tipo RAG, se busca garantizar que las respuestas generadas sean coherentes, contextualizadas y fundamentadas en información real disponible en el entorno empresarial.

Además, el uso de una arquitectura modular basada en *Workflows* permite adaptar el sistema a distintas necesidades organizativas, integrando agentes especializados, configuraciones personalizables y conectividad con servicios externos mediante APIs. Este enfoque no solo mejora la eficiencia en la gestión del conocimiento, sino que también aporta valor al estudio de soluciones escalables y trazables en el ámbito de la recuperación de información asistida por IA.

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es desarrollar un sistema automatizado basado en *Workflows* que permita a las empresas indexar y recuperar información de manera eficiente, utilizando agentes inteligentes, bases de datos y técnicas avanzadas de Inteligencia Artificial, con el fin de optimizar el acceso a su conocimiento interno y garantizar respuestas fundamentadas y coherentes en consultas empresariales. Para alcanzar este objetivo, los objetivos específicos que se abordan a lo largo del trabajo son los siguientes:

1. Analizar el estado del arte de frameworks y herramientas existentes para el diseño de *Workflows*, con el fin de identificar las tecnologías más adecuadas para la implementación del sistema.
2. Diseñar y desarrollar un *Workflow* de indexación que permita procesar e incorporar documentos empresariales desde distintas fuentes, incluyendo archivos locales o en fuentes como OneDrive y Salesforce.
3. Implementar un *Workflow* de recuperación que recupere información relevante en función de consultas en lenguaje natural, utilizando técnicas de RAG para generar respuestas precisas, contextualizadas y basadas en conocimiento estructurado.
4. Integrar y configurar agentes inteligentes en ambos *Workflows* para realizar tareas específicas como segmentación de texto, extracción de metadatos, reescritura de consultas y generación de respuestas optimizadas.
5. Utilizar bases de datos estructuradas para almacenar documentos indexados en forma de nodos y mejorar la eficiencia en la recuperación de información, asegurando que los fragmentos relevantes sean utilizados para sintetizar respuestas coherentes.

6. Evaluar la eficacia del sistema mediante pruebas con datos reales, midiendo la precisión, coherencia y tiempo necesario para la recuperación de información y generación de respuestas.
7. Explorar la aplicabilidad del sistema en diferentes sectores empresariales, considerando posibles mejoras en los *Workflows* y nuevas optimizaciones para ampliar su uso en entornos específicos.

1.3. Impacto Esperado

La implementación de este sistema tendrá un impacto significativo en la gestión del conocimiento empresarial, optimizando el acceso a la información y mejorando la eficiencia operativa. Los principales impactos previstos son los siguientes:

► **Impacto para los usuarios empresariales:**

- **Reducción del tiempo de búsqueda de información:** Los empleados podrán acceder de forma inmediata a datos relevantes sin necesidad de realizar búsquedas manuales en documentos dispersos.
- **Toma de decisiones más rápida y basada en datos:** Al contar con respuestas precisas y fundamentadas en información interna, las empresas podrán agilizar procesos críticos y mejorar su competitividad.
- **Automatización y optimización de flujos de trabajo:** La integración de agentes inteligentes reducirá la carga de trabajo manual, permitiendo a los empleados enfocarse en tareas de mayor valor estratégico.

► **Impacto para el desarrollo tecnológico y la investigación:**

- **Avance en la aplicación de RAG:** Este proyecto contribuye al estudio y desarrollo de sistemas de recuperación de información inteligentes, proporcionando un enfoque práctico para su aplicación en entornos empresariales.
- **Uso eficiente de la inteligencia artificial en procesos documentales:** La combinación de técnicas avanzadas de IA con bases de datos estructuradas permitirá mejorar la precisión y coherencia de los sistemas de recuperación de información.
- **Facilidad de adaptación a diferentes sectores:** La arquitectura flexible del sistema permitirá su implementación en distintas industrias, como marketing, asesoría legal, soporte al cliente y gestión documental.



► **Contribución a los Objetivos de Desarrollo Sostenible (ODS):**

- **ODS 8 (Trabajo decente y crecimiento económico):** Al mejorar la eficiencia y productividad dentro de las empresas, se facilita la optimización de los recursos humanos y tecnológicos.
- **ODS 9 (Industria, innovación e infraestructura):** La integración de tecnologías avanzadas en la gestión de información impulsa la innovación y el desarrollo de infraestructuras digitales más eficientes.
- **ODS 12 (Producción y consumo responsables):** La automatización y optimización de la gestión documental permiten reducir el consumo innecesario de papel y fomentar el uso eficiente de los recursos digitales.

1.4. Estructura del documento

El presente trabajo está organizado en siete capítulos principales y una sección de anexos y un glosario, estructurados para proporcionar un recorrido progresivo y coherente desde la introducción del problema hasta la evaluación de la solución y las posibles líneas de mejora futuras. La estructura es la siguiente:

- **Capítulo 1: Introducción y Motivación**

Este capítulo presentará el contexto general del problema que abordará el trabajo, la creciente necesidad de automatizar la gestión del conocimiento empresarial y las dificultades actuales en la búsqueda y recuperación de información dentro de las empresas. También se explicarán las razones que motivan la realización del proyecto, su relevancia en el ámbito empresarial y su impacto en la mejora de la productividad y eficiencia.

- **Capítulo 2: Estado del Arte y Fundamentos Teóricos**

En este capítulo se contextualizará el trabajo dentro del panorama actual de las tecnologías de gestión del conocimiento. Se presentará una evolución histórica desde los sistemas tradicionales hasta la aparición de modelos de lenguaje y soluciones RAG. Se analizarán herramientas y frameworks relevantes como LlamaIndex, LangChain, Haystack, Flowise y n8n, así como soluciones comerciales como Microsoft Copilot y ChatGPT. A partir de este análisis, se identificarán las principales limitaciones de las soluciones existentes, especialmente en lo relativo a trazabilidad, control sobre las fuentes de conocimiento y adaptabilidad a entornos empresariales reales. Finalmente, se expondrá la contribución que este TFG pretende aportar para cubrir esas carencias, detallando el espacio tecnológico que viene a ocupar y justificando su valor como propuesta original y aplicable en contextos reales.

- **Capítulo 3: Presentación de la propuesta y fundamentos técnicos**

En este capítulo se describirá el problema concreto que se desea resolver y se presentará la solución propuesta, basada en dos workflows complementarios: uno de indexación y otro de recuperación. Se incluirá una visión funcional de ambos flujos y una explicación de su interacción. Asimismo, se profundizará en los fundamentos técnicos del sistema,

describiendo la arquitectura modular de LlamaIndex, su enfoque basado en eventos y pasos, y las ventajas que aporta este modelo para construir soluciones robustas, trazables y personalizables.

- **Capítulo 4: Diseño e implementación del workflow de Indexación**

Se describirá la arquitectura del flujo de trabajo encargado del procesamiento e indexación de documentos, incluyendo las fuentes utilizadas, las técnicas de segmentación, la extracción de metadatos y la configuración de agentes inteligentes. Se detallarán los pasos técnicos seguidos y las decisiones adoptadas durante su implementación.

- **Capítulo 5: Diseño e implementación del workflow de Recuperación**

Se explicará el diseño y desarrollo del flujo de recuperación de información, centrado en responder preguntas basadas exclusivamente en el conocimiento indexado. Se presentarán las etapas del proceso, los distintos tipos de recuperación utilizados y los sintetizadores empleados para generar respuestas precisas y verificables.

- **Capítulo 6: Evaluación y Resultados**

Este capítulo detallará las pruebas realizadas para evaluar la precisión, eficiencia y tiempo de respuesta del sistema. Se presentarán métricas de rendimiento y análisis de casos de uso en distintos entornos empresariales. También se discutirá cómo la solución propuesta mejora la gestión del conocimiento y la toma de decisiones en comparación con métodos tradicionales de búsqueda de información.

- **Capítulo 7: Conclusiones y Futuras Líneas de Trabajo**

Este capítulo recogerá las conclusiones generales del proyecto, evaluando el grado de cumplimiento de los objetivos propuestos y reflexionando sobre los aprendizajes adquiridos. También se establecerá la relación del trabajo con los estudios del grado y se propondrán posibles mejoras y extensiones que podrían desarrollarse en el futuro para ampliar el alcance y aplicabilidad del sistema.

- **Anexos**

Finalmente, los anexos incluirán información complementaria como ejemplos de código, configuraciones utilizadas y documentación adicional sobre los modelos de IA empleados.

- **Glosario**

Para facilitar la comprensión del contenido, especialmente de aquellos términos técnicos específicos del área de Inteligencia Artificial y recuperación de información, el documento incluye en esta sección la definición de conceptos clave y acrónimos que pueden no ser familiares para lectores no especializados.

2. Estado del Arte y Fundamentos Teóricos

En este apartado se analiza la evolución de los sistemas de gestión del conocimiento y el papel que desempeñan las tecnologías actuales basadas en inteligencia artificial para su automatización. Se introduce el enfoque RAG como respuesta a las limitaciones de los sistemas tradicionales, y se realiza un estudio comparativo de las herramientas más relevantes que permiten construir soluciones de recuperación de información apoyadas en modelos de lenguaje.

El análisis incluye tanto frameworks de desarrollo de workflows inteligentes como LlamaIndex [1], LangChain [2], Haystack [3], Flowise [4] o N8n [5]. Este estudio permite justificar de forma técnica la elección de las tecnologías utilizadas en el proyecto, asegurando que la solución propuesta se apoya en herramientas robustas, configurables y adaptables a entornos reales.

2.1. Evolución tecnológica en la gestión del conocimiento empresarial

En las últimas décadas, el crecimiento exponencial de los datos generados por las empresas ha impulsado el desarrollo de sistemas orientados a la gestión del conocimiento. Estos sistemas, conocidos como Sistemas de Gestión del Conocimiento (Knowledge Management Systems, KMS), buscan organizar y reutilizar información relevante que reside en documentos, correos electrónicos, bases de datos, procesos internos y experiencias previas. Su objetivo principal es facilitar el acceso al conocimiento interno para mejorar la toma de decisiones estratégicas [6].

Los primeros enfoques de gestión del conocimiento se centraban en la codificación del conocimiento explícito, es decir, aquel que puede documentarse y estructurarse fácilmente. Estos sistemas estaban basados en estructuras rígidas, como bases de datos relacionales, catálogos jerárquicos y repositorios compartidos, que requerían intervención manual por parte de los usuarios, quienes debían etiquetar la información, clasificarla o ajustarla a esquemas predefinidos de metadatos. Este modelo resultaba efectivo en entornos con información estable y controlada, pero demostraba ser ineficiente ante el volumen, la variabilidad y la velocidad de crecimiento de los datos no estructurados actuales [7].

Ante estas limitaciones surgieron herramientas más avanzadas como los Enterprise Content Management (ECM) y los Document Management Systems (DMS), que incorporaron capacidades para automatizar parcialmente la organización, búsqueda y recuperación de información empresarial. Estos sistemas mejoraron la accesibilidad al conocimiento, integrando funciones como control de versiones, indexación automática y flujos de aprobación [8].

La verdadera disrupción tecnológica llegó con la adopción de tecnologías basadas en inteligencia artificial (IA), particularmente con el desarrollo de Modelos de Lenguaje de Gran Escala (Large Language Models, LLMs) como BERT, Claude, Gemini o GPT. Estos modelos marcaron un antes y un después en la forma de interactuar con el conocimiento: no solo mejoran

la comprensión semántica del lenguaje natural, sino que también son capaces de generar contenido, resumir documentos, responder preguntas y clasificar información de manera autónoma [10].

En paralelo, surgieron técnicas como la búsqueda semántica, que permite comparar la similitud de significados entre fragmentos de texto utilizando representaciones vectoriales (embeddings), sustituyendo las tradicionales búsquedas por palabra clave. Esta innovación permitió identificar y recuperar información relevante incluso cuando las consultas no coincidían textualmente con el contenido almacenado [9].

La convergencia entre estas capacidades dio lugar a una nueva generación de sistemas híbridos, conocidos como sistemas de Generación Aumentada por Recuperación (RAG). Estos sistemas combinan la fortaleza de la recuperación de documentos con la generación automática de respuestas, ofreciendo trazabilidad, personalización y control del conocimiento empleado. El impacto de este enfoque ha sido notable en sectores intensivos en información, como el legal, el médico o el financiero, donde la capacidad de generar respuestas fundamentadas en datos verificables es crítica [14].

Autores como Nonaka y Takeuchi ya anticipaban la necesidad de convertir el conocimiento tácito en activo organizacional, y los sistemas actuales permiten hacerlo de forma dinámica mediante la integración de agentes inteligentes y procesamiento contextual [16]. Este cambio ha derivado en una nueva concepción de los KMS como sistemas conversacionales, que no solo almacenan información, sino que la articulan en tiempo real según las necesidades del usuario [17].

Finalmente, esta evolución ha traído consigo nuevos desafíos: garantizar la trazabilidad de las fuentes utilizadas, mantener el control sobre los datos sensibles y diseñar arquitecturas modulares auditables. Estos requisitos han impulsado el desarrollo de frameworks como LlamaIndex, que permiten adaptar la lógica RAG a flujos empresariales concretos, asegurando transparencia, eficiencia y adaptabilidad [14].

2.2. Introducción a los sistemas RAG y su impacto en la recuperación de información

La evolución reciente de los modelos de lenguaje ha impulsado el desarrollo de sistemas híbridos que combinan capacidades de recuperación de información con generación de texto contextualizado. Este enfoque se conoce como RAG. A diferencia de los modelos generativos tradicionales, que producen respuestas exclusivamente a partir del conocimiento adquirido durante su entrenamiento, los sistemas RAG introducen un paso explícito de recuperación de documentos relevantes, lo que permite que las respuestas generadas estén fundamentadas en información verificable y actualizada [9][10].

El funcionamiento de un sistema RAG suele dividirse en dos fases:

1. **Recuperación:** Se utilizan técnicas de búsqueda por ejemplo, búsqueda semántica basada en embeddings, filtrado por metadatos, o búsqueda clásica por palabras clave para identificar los fragmentos (nodos) más relevantes relacionados con la consulta del usuario.
2. **Generación:** Un modelo de lenguaje, generalmente un LLM como GPT o Claude, sintetiza una respuesta a partir de los documentos recuperados, sin salirse del contexto proporcionado [10].

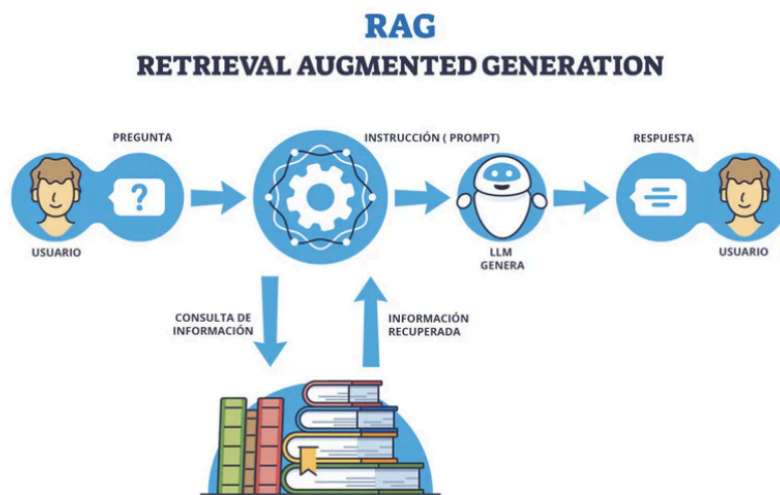


Figura 1. Funcionamiento general de un sistema RAG, combinando recuperación de información y generación de texto mediante LLMs.

En la **Figura 1** se ilustra el funcionamiento general de un sistema RAG, donde un modelo de lenguaje genera respuestas a partir de información previamente recuperada.

Este enfoque ha revolucionado el acceso al conocimiento en entornos profesionales, al ofrecer una solución intermedia entre el acceso estructurado a bases de datos y la generación libre de texto. En lugar de depender exclusivamente de lo aprendido por el modelo (lo que puede llevar a errores o alucinaciones), el sistema actúa como un asistente que busca primero en su “biblioteca” antes de hablar. Esto aumenta la fiabilidad, trazabilidad y adaptabilidad de las respuestas, haciéndolo especialmente útil en sectores donde el margen de error debe ser mínimo, como el jurídico, el médico o el financiero [10].

Una de las ventajas más destacadas del enfoque RAG es que permite operar en entornos cerrados: el conocimiento empleado puede limitarse a un conjunto de documentos seleccionados por el usuario, garantizando así que la respuesta generada proviene exclusivamente de fuentes conocidas y validadas. Esta característica es clave en casos de uso donde se requiere confidencialidad y control sobre el contenido.

Desde el punto de vista técnico, los sistemas RAG pueden combinarse con diferentes métodos de recuperación:

- **Vector stores** (como FAISS, Weaviate, Pinecone o MongoDB Atlas Search), que permiten recuperar documentos basándose en la similitud semántica entre los embeddings del texto.
- **Índices jerárquicos**, que estructuran la información por secciones o niveles de granularidad.
- **Filtros contextuales o reglas lógicas**, que permiten afinar la recuperación mediante metadatos o condiciones específicas.

Empresas líderes como OpenAI, Microsoft y Google han adoptado este enfoque en sus plataformas comerciales. Por ejemplo, ChatGPT ha introducido plugins con capacidades de recuperación, Copilot permite respuestas basadas en documentos de Office 365, y Bard puede acceder a información en Google Workspace. No obstante, estas soluciones suelen estar altamente preconfiguradas, lo que limita la personalización del flujo de trabajo, la trazabilidad de las fuentes y la integración con procesos empresariales específicos [14].

A nivel académico, se ha demostrado que los sistemas RAG no solo mejoran la precisión de las respuestas, sino también su completitud y adecuación al contexto, superando a los modelos tradicionales en tareas complejas de recuperación y síntesis de información [18]. Además, existen múltiples variantes avanzadas, como RAG con enrutamiento de consultas, RAG multi-hop (encadenamiento de respuestas), y RAG iterativo con autoevaluación y corrección [19].

El enfoque RAG no solo representa un avance técnico, sino también un cambio de paradigma en la forma de construir asistentes inteligentes, que dejan de ser modelos genéricos y pasan a ser sistemas modulares, auditables y ajustados al conocimiento interno de la organización.

2.3. El uso de workflows en el ámbito de la Inteligencia Artificial

En el ámbito de la inteligencia artificial aplicada a la gestión del conocimiento y la recuperación de información, el término *workflow* hace referencia a una secuencia estructurada de pasos que se ejecutan de forma ordenada para transformar datos de entrada en resultados útiles. Cada paso (o *step*) representa una operación específica, como la segmentación de texto, la extracción de metadatos, la generación de resúmenes o la formulación de respuestas, permitiendo así una ejecución modular, escalable y trazable [12].

El diseño basado en workflows ha cobrado especial relevancia en sistemas que integran múltiples componentes de IA, ya que facilita la orquestación de procesos complejos mediante una arquitectura clara y mantenible. A diferencia de enfoques monolíticos, los workflows permiten separar la lógica de cada etapa, favoreciendo la reutilización de componentes y la personalización de los procesos según las necesidades del dominio.

Ventajas clave de los workflows en IA:

- **Modularidad y reusabilidad:** Cada componente puede desarrollarse, probarse y mantenerse de forma independiente. Esto permite construir bibliotecas de herramientas

(tools) reutilizables en distintos proyectos o dominios.

- **Configurabilidad:** Los workflows pueden adaptarse a distintos escenarios ajustando parámetros, sustituyendo módulos o integrando agentes específicos para tareas como resumen, clasificación o análisis semántico.
- **Trazabilidad y control:** Al estar organizados paso a paso, se puede auditar el comportamiento del sistema, depurar errores fácilmente, y asegurar que las decisiones tomadas por los modelos son comprensibles y justificables.
- **Escalabilidad:** Permiten procesar grandes volúmenes de datos mediante ejecución paralela o asíncrona, lo que los hace adecuados para entornos empresariales donde se requiere eficiencia operativa.

En el contexto del presente trabajo, los workflows se utilizan para estructurar el proceso de indexación y recuperación del conocimiento empresarial. Este enfoque permite separar dos tareas principales:

- **El flujo de indexación,** encargado de transformar documentos o cualquier tipo de contenido en nodos indexables mediante técnicas como extracción de texto, segmentación semántica, generación de embeddings y enriquecimiento de metadatos.
- **El flujo de recuperación,** que utiliza esos nodos para generar respuestas fundamentadas ante consultas en lenguaje natural, aplicando pasos como enrutamiento, recuperación semántica, reranking, y síntesis de respuestas.

Frameworks como LlamaIndex, LangChain, Haystack, Flowise o n8n han incorporado el concepto de workflows para facilitar el desarrollo de sistemas RAG y asistentes inteligentes. Por ejemplo, LlamaIndex permite construir workflows componibles donde cada paso puede integrarse con agentes especializados, fuentes de datos externas, o servicios cloud, manteniendo al mismo tiempo un control detallado sobre el flujo de datos y las decisiones del sistema [13].

Además, este enfoque es compatible con estrategias de low-code/no-code, permitiendo que usuarios técnicos o incluso no técnicos configuren y modifiquen flujos de procesamiento sin necesidad de intervenir en el código fuente. Herramientas como Flowise o n8n permiten construir visualmente workflows basados en modelos de lenguaje, lo que democratiza el acceso a soluciones de IA y acelera el desarrollo de aplicaciones inteligentes.

Desde el punto de vista de la investigación, el uso de workflows en IA se ha asociado con mayor reproducibilidad de los experimentos, transparencia en la toma de decisiones algorítmicas y facilidad para transferir conocimiento entre dominios, lo que convierte esta aproximación en un estándar emergente en sistemas de IA aplicados [21].

La adopción de workflows en sistemas de inteligencia artificial no solo aporta beneficios técnicos, sino que también mejora la gobernanza del sistema, su auditabilidad y su adaptabilidad a escenarios empresariales reales. Por ello, constituyen un componente central en el diseño de soluciones robustas, personalizadas y alineadas con los objetivos estratégicos de las organizaciones.

2.4. Herramientas para la creación de workflows

En el contexto de los sistemas de recuperación de información asistidos por modelos de lenguaje, ha surgido la necesidad de herramientas que permitan definir y gestionar flujos de trabajo complejos, compuestos por múltiples etapas interdependientes. Estos workflows permiten orquestar, de forma modular y estructurada, tareas como la ingestión de documentos, el preprocesamiento, la indexación, la recuperación y la generación de respuestas.

Actualmente, existen diversas herramientas que permiten implementar este tipo de soluciones, destacando tanto por su flexibilidad como por su capacidad de integración con modelos de lenguaje y bases de datos. A continuación, se presentan las más relevantes para el desarrollo de sistemas RAG.

► LlamaIndex

LlamaIndex es un framework de código abierto diseñado específicamente para la construcción de sistemas RAG. Su principal fortaleza radica en su arquitectura basada en workflows componibles, lo que permite definir de forma clara y extensible las distintas fases del procesamiento de la información [11]. Ofrece integración directa con bases de datos vectoriales, extractores de metadatos, agentes inteligentes, y modelos de lenguaje como los de OpenAI. Además, permite personalizar cada paso del flujo de trabajo mediante herramientas específicas, lo que lo convierte en una solución flexible, trazable y altamente adaptativa [12].

► LangChain

LangChain es una herramienta ampliamente utilizada para desarrollar aplicaciones que combinan modelos de lenguaje con herramientas externas, bases de datos, interfaces de usuario y entornos de ejecución. Aunque su arquitectura se basa en cadenas de componentes (chains) más que en workflows formales, permite encadenar operaciones de forma modular y flexible. Esta aproximación es útil para desarrollar agentes conversacionales, asistentes personalizados o sistemas de razonamiento encadenado. Sin embargo, su orientación más generalista puede ser una limitación en casos donde se requiere una estructura clara de recuperación documental, trazabilidad estricta o control granular sobre los datos procesados [13].

► Haystack

Haystack, desarrollado por Deepset, es un framework centrado en la construcción de sistemas de recuperación de información semántica. Está especialmente orientado a tareas de búsqueda en documentos extensos y sistemas de preguntas y respuestas, mediante el uso de embeddings, motores de búsqueda vectorial y pipelines de procesamiento. Su arquitectura se basa en nodos conectados por pipelines, permitiendo configurar soluciones complejas a través de componentes reutilizables. Aunque no implementa una estructura de workflow tan explícita como LlamaIndex, su potencia, modularidad y escalabilidad lo convierten en una opción sólida para proyectos de alta carga de información [15].

► FlowiseAI

Flowise es una plataforma de tipo low-code que permite construir flujos de trabajo visuales combinando modelos de lenguaje, bases de datos, herramientas externas y lógica de negocio. Está pensada para facilitar la creación de agentes conversacionales y asistentes inteligentes sin necesidad de escribir código, lo que democratiza el acceso a tecnologías avanzadas de IA. Aunque su enfoque no está centrado exclusivamente en sistemas RAG, ofrece integración con LLMs (Large Language Models), vectores y servicios web, y puede utilizarse como interfaz para orquestar workflows definidos en otras herramientas.

► n8n

n8n es una herramienta de automatización de workflows orientada a eventos, que permite conectar servicios, APIs y procesos complejos de forma visual. Aunque no está pensada específicamente para tareas de procesamiento de lenguaje natural, es ampliamente utilizada como infraestructura de orquestación para integrar sistemas de IA con otras plataformas empresariales (bases de datos, correos, CRM, etc.). Gracias a su flexibilidad y soporte para ejecución condicional y asíncrona, puede ser una excelente opción como capa de integración para coordinar workflows RAG desarrollados con otras herramientas.

Herramienta	Enfoque principal	Modularidad	Estructura orientada a workflows	Configuración flexible (Personalizable)	Trazabilidad	Facilidad de uso
LlamaIndex	RAG, automatización documental	Alta	Sí	Sí	Sí	Media
LangChain	Agentes, razonamiento encadenado	Media	Parcial	Sí	Parcial	Alta
Haystack	Recuperación semántica, QA pipelines	Alta	Intermedia	Parcial	Parcial	Alta
Flowise	Agentes visuales, low-code	Media	Sí	Sí	Baja	Muy Alta
n8n	Automatización de procesos y eventos	Alta	Sí	Sí	Media	Alta

Tabla 1. Comparativa técnica de herramientas para la creación de workflows

La **Tabla 1** resume y compara las principales herramientas utilizadas para la creación de workflows en el ámbito de la inteligencia artificial aplicada a la gestión del conocimiento. Para ello, se han considerado seis características clave:

- **Enfoque principal:** Describe el tipo de aplicaciones para las que ha sido diseñada cada herramienta, ya sea recuperación documental, agentes conversacionales, automatización visual, etc.
Como se observa, cada herramienta se especializa en un ámbito diferente. LlamaIndex se centra en RAG y automatización documental, mientras que Flowise y n8n destacan por su orientación low-code o de orquestación de eventos.
- **Modularidad:** Indica el grado en que cada herramienta permite separar y reutilizar componentes, lo cual es fundamental para la construcción de sistemas flexibles y mantenibles.
La tabla muestra que LlamaIndex y n8n presentan una modularidad alta, ideal para definir flujos complejos a partir de bloques independientes, mientras que LangChain y Flowise tienen un enfoque más acoplado.
- **Estructura orientada a workflows:** Señala si la herramienta proporciona una arquitectura formal de workflows que permita definir etapas independientes con control sobre la secuencia y lógica del flujo.
En este aspecto, tanto LlamaIndex como Flowise y n8n ofrecen una estructura clara para definir workflows, siendo LlamaIndex la que mejor equilibra estructura formal con capacidades avanzadas.
- **Configuración flexible (Personalizable):** Evalúa hasta qué punto la herramienta permite adaptar los workflows a diferentes contextos mediante parámetros, herramientas externas o modelos personalizados.
Todas las herramientas permiten cierto grado de personalización, pero LlamaIndex destaca por su integración con configuraciones declarativas, lo que facilita construir flujos especializados sin modificar el código base.
- **Trazabilidad:** Representa la capacidad de seguir el origen de las decisiones o resultados generados en cada paso del flujo, lo que resulta especialmente relevante en entornos regulados o críticos.
Este aspecto diferencia claramente a LlamaIndex, que incorpora trazabilidad nativa en cada etapa del flujo, frente a la limitada visibilidad ofrecida por Flowise, Haystack o LangChain.
- **Facilidad de uso:** Refleja la curva de aprendizaje y accesibilidad para el usuario, diferenciando entre herramientas más técnicas y aquellas orientadas a perfiles no desarrolladores.
Flowise y n8n lideran este aspecto gracias a sus interfaces visuales. LlamaIndex, aunque algo más técnica, logra un equilibrio razonable entre capacidad avanzada y curva de aprendizaje moderada.

A partir de esta comparativa, se observa que LlamaIndex destaca como la opción más adecuada para el desarrollo de workflows. Su orientación nativa hacia flujos de trabajo componibles, su integración con bases vectoriales, agentes inteligentes y modelos de lenguaje avanzados, junto con su alto nivel de trazabilidad, lo posicionan como la herramienta más robusta para construir sistemas de recuperación aumentada por generación personalizados, auditables y escalables.

2.5. Estudio de herramientas RAG

Las tecnologías de generación aumentada por recuperación, junto con los modelos de lenguaje de última generación, han supuesto un avance notable en las tareas de recuperación y síntesis de información. Sin embargo, cuando estas soluciones se trasladan a entornos reales, surgen importantes limitaciones que condicionan su utilidad práctica.

Una de las principales carencias es la falta de control sobre los documentos que se indexan. Herramientas como ChatGPT o Microsoft Copilot permiten una integración básica con archivos del usuario, pero no ofrecen mecanismos claros para definir con precisión qué información se indexa ni cómo se transforma antes de utilizarse. Esto puede dar lugar a la inclusión de datos irrelevantes o confidenciales sin ofrecer garantías claras sobre la trazabilidad y validación de la información.

Además, en estos sistemas las respuestas generadas suelen basarse en el conocimiento preentrenado del modelo, incluso cuando están conectados a contenidos específicos del usuario. Esta dependencia conlleva múltiples riesgos, como la generación de respuestas potencialmente correctas pero no verificables, la mezcla entre datos internos y conocimiento externo difícilmente validable, y la ausencia de referencias explícitas que justifiquen la información proporcionada.

A nivel arquitectónico, muchas herramientas tampoco ofrecen la modularidad necesaria para personalizar los flujos de trabajo. La imposibilidad de definir etapas como la extracción automática de metadatos, la división semántica del contenido o la aplicación de filtros contextuales, limita gravemente su utilidad en procesos complejos.

Asimismo, la falta de transparencia en la generación de respuestas es una deficiencia crítica. En numerosos casos, los usuarios no pueden rastrear con claridad el origen de la información mostrada, lo que representa un obstáculo en sectores donde la fiabilidad del contenido es prioritaria, como el ámbito jurídico, sanitario o financiero.

A todas estas limitaciones se suma un aspecto aún más fundamental: los asistentes generalistas como ChatGPT, Bard o Copilot no están diseñados para responder a preguntas personalizadas basadas en conocimiento interno del usuario. Por ejemplo, no pueden procesar consultas como *“¿qué me respondió la persona X por correo?”* o *“¿cuáles son los documentos que me envió la persona X la semana pasada?”*, ya que carecen de acceso estructurado y trazable a fuentes documentales propias del usuario. En cambio, una solución RAG personalizada, que indexe selectivamente correos, documentos y contenidos relevantes para un caso de uso concreto, sí es capaz de dar respuesta a estas preguntas con precisión, trazabilidad y total alineación con la información real del entorno de trabajo.

Estas carencias ponen de manifiesto una brecha clara entre las capacidades actuales de las herramientas disponibles y las necesidades concretas de las organizaciones. A pesar de la potencia de los modelos existentes, su uso directo en entornos reales todavía presenta obstáculos importantes en términos de precisión, control, personalización y confianza. Este vacío justifica la necesidad de desarrollar enfoques alternativos más adaptados al contexto empresarial, con énfasis en la selección de fuentes, la trazabilidad de las respuestas y la integración fluida con el ecosistema digital de cada organización.

Ante estas limitaciones, este Trabajo Fin de Grado propone una solución personalizada basada en una arquitectura RAG modular y configurable, capaz de adaptarse a distintos entornos empresariales con control total sobre los datos indexados, trazabilidad en las respuestas generadas y una alta capacidad de integración.

Por un lado, frente a la falta de control sobre las fuentes documentales, el sistema propuesto permite que el propio usuario decida explícitamente qué documentos incorporar, garantizando que únicamente se indexe información relevante, verificada y útil para sus necesidades concretas. Esto se traduce en una mejora sustancial en términos de precisión y fiabilidad.

Además, todas las respuestas generadas por el sistema se fundamentan exclusivamente en fragmentos previamente indexados, evitando cualquier mezcla con el conocimiento preentrenado del modelo de lenguaje. Esta separación garantiza que las respuestas sean verificables, auditables y directamente trazables a su fuente de origen. Además, el sistema incorpora mecanismos que permiten que cada respuesta esté acompañada de referencias documentales claras, facilitando la validación de la información ofrecida. Esta característica resulta esencial en contextos críticos como el legal, el médico o el financiero, donde la confianza en la fuente es indispensable.

Por otro lado, la solución está diseñada con un enfoque modular que permite su integración con diversas fuentes externas (como APIs, gestores documentales o plataformas cloud) y su adaptación a diferentes flujos de trabajo internos. Esta flexibilidad facilita la incorporación de nuevas funcionalidades o el ajuste del sistema a nuevos dominios sin necesidad de rediseñar la arquitectura base. Estas aportaciones no pretenden reemplazar las herramientas existentes, sino ofrecer una alternativa más controlada, trazable y adaptable que responda a necesidades específicas no cubiertas por soluciones generalistas. El TFG se plantea, por tanto, como una contribución práctica y orientada a la aplicabilidad real de las tecnologías RAG en entornos empresariales complejos.

A continuación se presenta una tabla comparativa que sintetiza las principales diferencias entre la solución personalizada desarrollada en este trabajo y algunas de las herramientas más utilizadas en el ámbito de la generación aumentada por recuperación:

Característica	RAG Personalizado (Sistema propuesto)	ChatGPT / Bard	Microsoft Copilot	RAG Genérico (RAGChain)
Acceso a conocimiento interno	Sí	Limitado o inexistente	Parcial	Sí
Trazabilidad de respuestas	Alta (referencias directas)	Nula	Nula	Baja
Personalización	Alta	Nula	Nula	Media
Control del contenido indexado	Total	Ninguno	Ninguno	Parcial
Integración con APIs	Directa y flexible	No disponible	Solo MS 365	Requiere desarrollo
Soporte para agentes inteligentes	Sí	No	No	Limitada
Adaptabilidad a distintos casos de uso	Alta	Baja	Media	Media

Tabla 2. Comparativa funcional entre el sistema RAG propuesto y las soluciones existentes

En la **Tabla 2** se presenta una comparativa funcional entre el sistema RAG personalizado desarrollado en este proyecto y otras soluciones existentes ampliamente utilizadas: ChatGPT/Bard, Microsoft Copilot y RAGChain de LangChain (como ejemplo de RAG genérico).

RAGChain es una pipeline predefinida incluida en el framework LangChain que combina recuperación vectorial y generación sin permitir una personalización avanzada. Su estructura fija y limitada configurabilidad lo convierten en un ejemplo representativo de enfoque RAG genérico, útil para casos simples pero poco adecuado para entornos empresariales que requieren trazabilidad, control de fuentes y adaptación a distintos flujos de trabajo.

Se han considerado criterios clave para evaluar la adecuación de cada enfoque a entornos empresariales, especialmente aquellos donde el control del conocimiento, la trazabilidad y la personalización son esenciales.

Los factores analizados incluyen:

- **Acceso a conocimiento interno:** Evalúa si el sistema puede consultar información propia de la organización (documentos, correos, bases de datos internas). Esto es fundamental para generar respuestas basadas en el conocimiento específico del entorno. *En la tabla se observa que únicamente el sistema propuesto y las soluciones tipo RAGChain permiten esta capacidad, aunque en el caso de RAGChain, su acceso está limitado por la arquitectura predefinida y la falta de trazabilidad estructurada. Mientras que las soluciones comerciales (ChatGPT/Bard y Copilot) presentan serias limitaciones o dependen de integraciones específicas.*
- **Trazabilidad de respuestas:** Indica si las respuestas generadas están respaldadas por fuentes específicas, algo crucial para verificar y validar la información proporcionada. *Aquí el sistema propuesto destaca con trazabilidad directa mediante nodos referenciados, frente a la ausencia de esta funcionalidad en ChatGPT y Copilot, y la trazabilidad limitada en RAGChain, que no ofrece mecanismos claros de justificación o referencias documentales.*
- **Personalización:** Se refiere a la capacidad del sistema para adaptarse a dominios concretos, estilos de respuesta, reglas internas o lógicas empresariales. *La tabla refleja que el sistema propuesto permite una personalización completa mediante configuración, a diferencia de las alternativas, que presentan capacidades muy limitadas o nulas.*
- **Control del contenido indexado:** Evalúa si el usuario puede decidir qué documentos se indexan y cómo se pre-procesan, aspecto clave para asegurar calidad y privacidad. *Solo el sistema desarrollado proporciona control total sobre qué se indexa y cómo, mientras que los sistemas externos operan con un enfoque cerrado y poco flexible.*
- **Integración con APIs:** Considera la posibilidad de conectar el sistema con otros servicios, bases de datos o flujos de trabajo automatizados. *La tabla deja claro que solo el sistema propuesto permite una integración directa y personalizable, frente a la rigidez de Copilot o la falta total de acceso externo en ChatGPT. RAGChain puede integrarse con otras herramientas de LangChain, pero requiere desarrollo adicional y carece de interfaces universales o configurables.*
- **Soporte para agentes inteligentes:** Analiza si se pueden incorporar agentes específicos para tareas como resumen, evaluación de fidelidad, o reformulación de consultas. *El sistema propuesto admite de forma nativa múltiples agentes especializados, en contraposición al resto de soluciones, que no permiten esta extensión o lo hacen de forma muy limitada.*
- **Adaptabilidad a distintos casos de uso:** Mide el grado de reutilización y ajuste de la solución ante nuevos escenarios o dominios empresariales. *La adaptabilidad del sistema propuesto es alta gracias a su arquitectura modular y configurable, algo que queda patente en la tabla al contrastarlo con la rigidez de las soluciones generalistas.*

Como puede observarse, el sistema RAG propuesto destaca en todos los aspectos clave, superando ampliamente a las soluciones generalistas como ChatGPT o Copilot. Mientras estas últimas pueden ser útiles en contextos simples o de productividad individual, no ofrecen control sobre las fuentes de información, ni permiten garantizar trazabilidad ni personalización profunda. Incluso los sistemas RAG genéricos en este caso RAGChain requieren desarrollos adicionales para alcanzar el nivel de adaptación y gobernanza que exige un entorno corporativo real.

2.6. Conclusiones

A modo de cierre del análisis del estado del arte, este apartado resume y justifica las decisiones tecnológicas tomadas para el desarrollo del sistema propuesto, basadas en los criterios de adecuación técnica, modularidad, facilidad de integración y viabilidad en un entorno de desarrollo realista.

En primer lugar, se ha optado por LlamaIndex como framework principal para la construcción de los workflows de indexación y recuperación. Esta elección se fundamenta en su arquitectura orientada a workflows componibles, que permite diseñar pipelines modulares y personalizables para tareas de recuperación aumentada por generación. Frente a alternativas como Flowiseai, n8n, LangChain o Haystack, LlamaIndex ofrece una estructura más clara y flexible para definir pasos independientes, integrando fácilmente modelos de embeddings, bases de datos vectoriales y agentes inteligentes, lo que facilita tanto el desarrollo como el mantenimiento y la evolución futura del sistema.

En cuanto a los agentes inteligentes que forman parte de los distintos workflows, se ha seleccionado el uso de modelos de lenguaje de OpenAI, en este caso el modelo *gpt-4o-mini*. Esta elección responde tanto al rendimiento avanzado y la capacidad demostrada del modelo en tareas complejas de generación y evaluación de texto, como a su facilidad de uso gracias a APIs bien documentadas y ampliamente adoptadas por la comunidad. Además, su relación coste-beneficio es especialmente competitiva, permitiendo realizar inferencias de forma eficiente que mantienen un rendimiento óptimo con un consumo económico reducido.

El uso de modelos OpenAI también permite una integración directa con agentes programables mediante esquemas JSON y técnicas de ingeniería de prompts, lo que simplifica el diseño de agentes especializados para tareas como la evaluación de fidelidad, la síntesis de respuestas o la comparación semántica entre nodos.

En resumen, las herramientas seleccionadas —LlamaIndex para la construcción de workflows y OpenAI para la lógica de los agentes— permiten abordar los retos detectados en el estado del arte, ofreciendo una solución robusta, escalable y accesible tanto desde el punto de vista técnico como económico.

3. Presentación de la propuesta y fundamentos técnicos

3.1. Problema identificado y motivación técnica

Acceder al conocimiento interno de una organización debería ser tan fácil como hacer una pregunta clara y obtener una respuesta precisa. Sin embargo, la realidad dista mucho de este ideal, la información relevante suele estar dispersa, desestructurada o enterrada en sistemas que no se comunican entre sí.

En muchas empresas, un simple proceso como recuperar las condiciones legales de un contrato, encontrar el procedimiento técnico más actualizado o responder a una duda recurrente de un cliente puede requerir consultar múltiples fuentes: bases de datos, carpetas compartidas en la nube, correos electrónicos antiguos o incluso preguntar directamente a compañeros con experiencia previa. Este tipo de fricciones no solo ralentiza la productividad, sino que puede provocar errores, duplicidad de esfuerzos y pérdida de conocimiento a largo plazo.

Por ejemplo, en un departamento legal, un trabajador puede necesitar conocer si una determinada cláusula ha sido aplicada previamente en otros contratos. Sin un sistema de búsqueda inteligente, deberá revisar manualmente múltiples documentos legales. En el área de soporte técnico, responder a una pregunta del cliente puede implicar revisar documentación obsoleta o consultar a varios responsables para confirmar la versión vigente de un procedimiento. Incluso en entornos administrativos, tareas como encontrar el formato correcto de una plantilla o verificar instrucciones de uso interno pueden convertirse en cuellos de botella si la información no está correctamente indexada.

Aunque los modelos de lenguaje actuales permiten generar respuestas rápidas a partir de texto, carecen del control, la trazabilidad y la personalización necesarios para trabajar con información empresarial sensible. No es posible garantizar qué fuentes se han consultado ni ajustar las respuestas al estilo, políticas o terminología interna de la organización. Esta brecha entre lo que las organizaciones necesitan y lo que las soluciones generalistas pueden ofrecer motiva el diseño de un sistema distinto.

Este proyecto propone una solución estructurada, basada en dos workflows principales, que permite a las empresas indexar su conocimiento interno y consultarlo de forma inteligente mediante técnicas de RAG. La propuesta no busca competir con los modelos generalistas, sino construir sobre ellos una arquitectura adaptada al contexto empresarial, con control total sobre qué se indexa, cómo se responde y de dónde proviene cada respuesta.

El resto del capítulo presenta esta propuesta: primero desde una visión funcional y luego desde los fundamentos técnicos que la hacen posible.

3.2. Sistema propuesto: visión general y arquitectura funcional

El sistema propuesto se basa en la integración de dos workflows independientes pero interconectados que, en conjunto, permiten automatizar el ciclo completo de gestión del conocimiento, desde la incorporación de nuevas fuentes de información hasta la generación de respuestas inteligentes a partir de ellas.

- Por un lado, el workflow de indexación actúa como punto de entrada del sistema. Su función es procesar e incorporar varios tipos de contenidos (como documentos, correos electrónicos, informes, etc.), segmentándolo en fragmentos significativos (nodos) y almacenándolo de forma estructurada para facilitar su posterior recuperación.
- Por otro lado, el workflow de recuperación permite a los usuarios formular consultas en lenguaje natural y obtener respuestas fundamentadas únicamente en el conocimiento previamente indexado. Este flujo se apoya en modelos de lenguaje avanzados, pero restringe sus respuestas al contenido verificado que ha sido previamente procesado por la etapa de indexación.

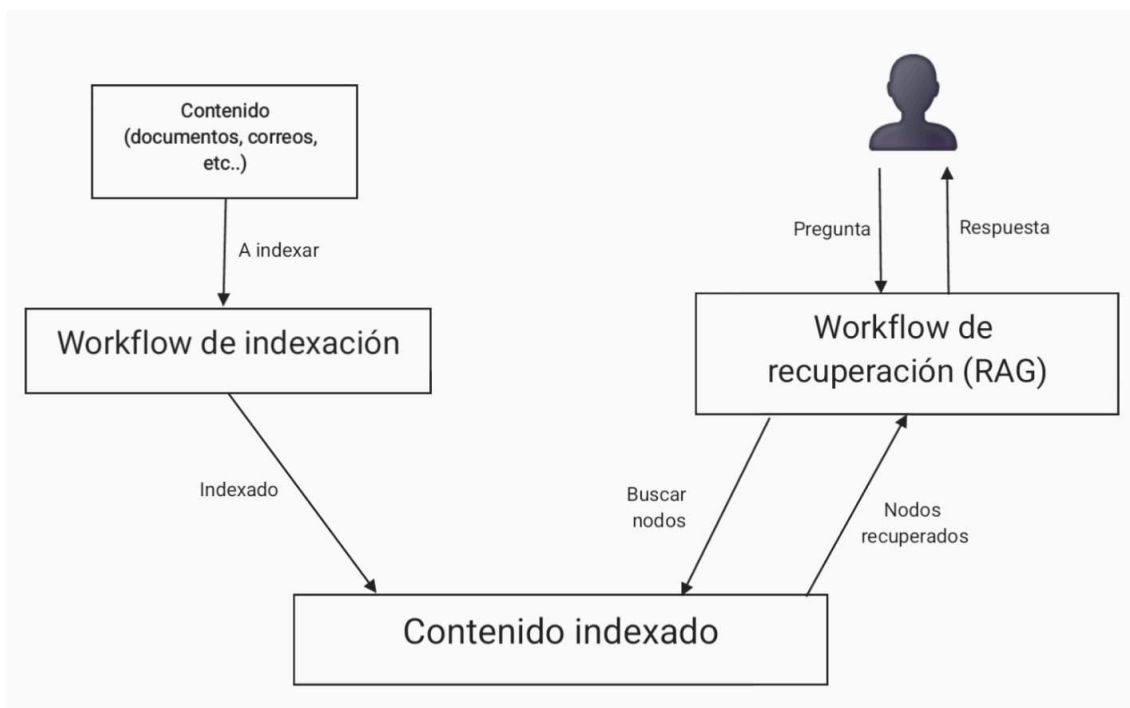


Figura 2. Arquitectura general del sistema propuesto basado en dos workflows interconectados

Ambos workflows están diseñados de forma modular, permitiendo su ejecución independiente. Sin embargo, su utilidad práctica está profundamente entrelazada: el segundo depende del primero para tener contenido fiable sobre el que razonar, y el primero adquiere sentido estratégico al servir como base para un sistema consultivo inteligente. Sin el workflow de indexación, no hay información estructurada sobre la que realizar recuperación; y sin el workflow de recuperación, el conocimiento indexado queda inerte y sin uso práctico. Esta dinámica puede observarse con claridad en la **Figura 2**, donde se representa la arquitectura general del sistema propuesto y la relación directa entre las dos fases del proceso: indexación y

recuperación. Esta dependencia mutua es clave para garantizar que el sistema funcione como una solución completa, cerrando el ciclo entre entrada de conocimiento y acceso contextualizado al mismo.

3.2.1. Workflow de indexación

El workflow de indexación constituye el punto de entrada del sistema. Su objetivo es transformar contenido empresarial multiformato (como documentos, correos electrónicos, informes o mensajes, etc..) en una representación estructurada llamada nodos, que servirá de base para la posterior recuperación de información.

El flujo sigue una secuencia modular de etapas, cada una encargada de realizar una transformación específica sobre los datos:

1. Interpretar las referencias de entrada, como rutas de archivos, enlaces o identificadores de correos.
2. Extraer el contenido textual relevante.
3. Aplicar filtros para eliminar fragmentos vacíos, irrelevantes o redundantes.
4. Enriquecer los nodos con metadatos adicionales o contenido generado automáticamente (como resúmenes o etiquetas).
5. Transformar y almacenar los nodos en una base de datos estructurada o vector store, lista para ser consultada.

Cada etapa está diseñada para ser desacoplada, configurable y trazable, lo que permite adaptar el workflow a diferentes fuentes, tipos de contenido o requisitos de negocio. Esta flexibilidad resulta clave para integrar el sistema en entornos empresariales reales, donde la heterogeneidad de la información es la norma.

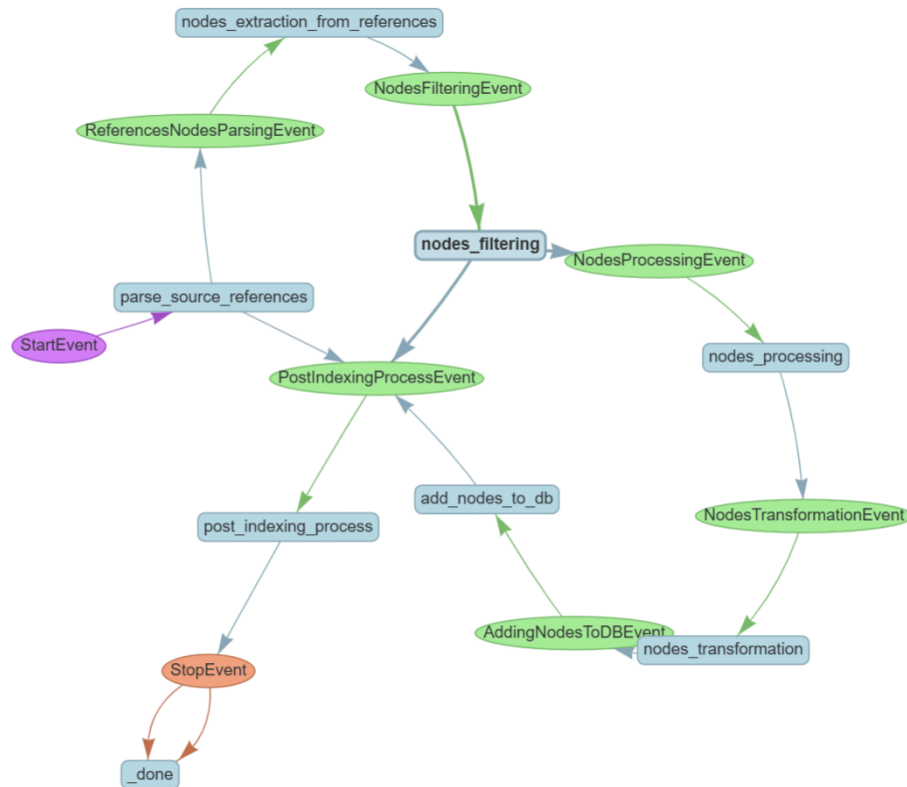


Figura 3. Arquitectura funcional general del workflow de indexación.

En la **Figura 3** se representa el flujo general del workflow de indexación. Las etapas del proceso están señaladas en azul, los eventos que conectan dichas etapas en verde, el punto de inicio del flujo (*StartEvent*) en morado y el final del proceso (*StopEvent*) en naranja. Esta estructura refleja cómo la información fluye de forma modular y controlada a lo largo de todo el proceso de indexación.

3.2.2. Workflow de recuperación

El workflow de recuperación representa la fase consultiva del sistema, en la que los usuarios pueden realizar preguntas en lenguaje natural y obtener respuestas fundamentadas exclusivamente en el conocimiento previamente indexado.

Este flujo opera como un sistema RAG, en el que se combinan mecanismos de búsqueda semántica y técnicas de generación de texto controlada. A nivel funcional, el proceso sigue las siguientes etapas generales:

1. Recepción de la consulta del usuario, que puede ser transformada o expandida para mejorar su calidad interpretativa.
2. Recuperación de los nodos más relevantes en base al contenido indexado, utilizando distintos tipos de recuperadores y técnicas de reranking.

3. Enriquecimiento contextual, añadiendo nodos relacionados, resúmenes de secciones, fragmentos ampliados o secciones clave.
4. Generación de la respuesta final, sintetizada únicamente a partir del contenido recuperado, con posibilidad de evaluación de fidelidad y consolidación mediante agentes especializados.

A diferencia de los modelos generativos tradicionales, esta workflow no inventa información, solo produce respuestas basadas en el material previamente procesado por el workflow de indexación. Esto garantiza una alta trazabilidad, evita alucinaciones y permite auditar el origen de cada fragmento utilizado.

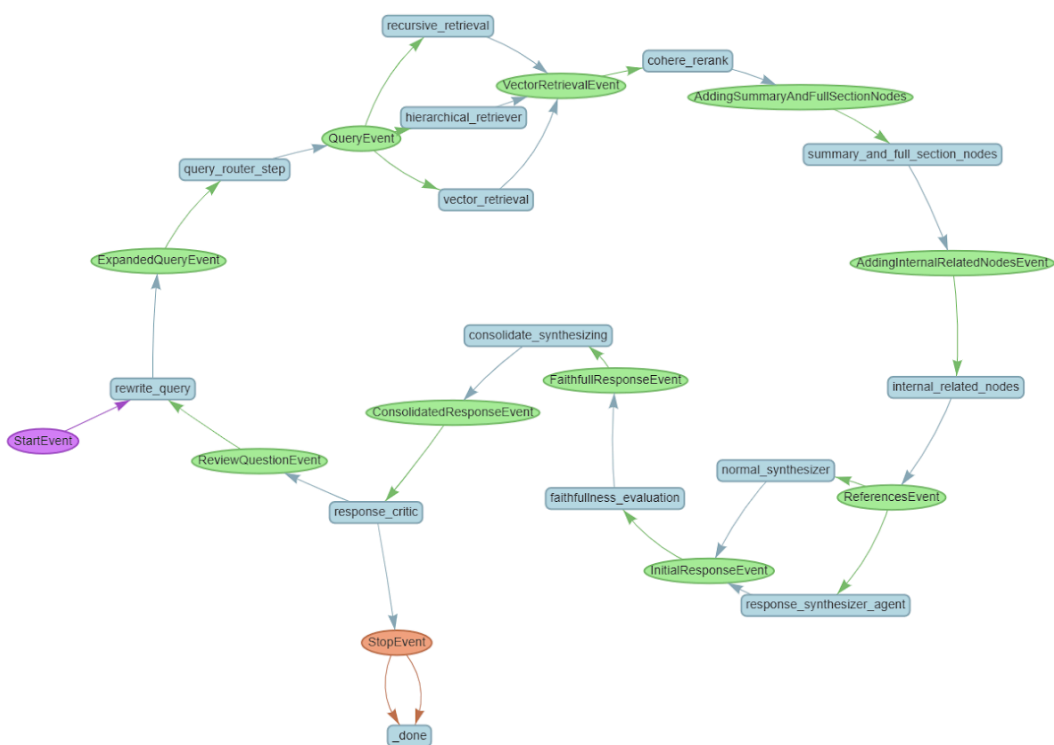


Figura 4. Arquitectura general del workflow de recuperación.

En la **Figura 4** se muestra el flujo completo del workflow de recuperación. Las etapas principales del proceso están representadas en azul, los eventos que conectan dichas etapas en verde, el punto de entrada al flujo (*StartEvent*) en morado y el cierre del proceso (*StopEvent*) en naranja. Esta representación refleja la estructura modular y controlada con la que se ejecuta cada fase de la recuperación.

3.3. Fundamentos técnicos sobre LlamaIndex y arquitectura basada en workflows

3.3.1. ¿Qué es LlamaIndex y por qué se utiliza? Ventajas técnicas de su arquitectura

LlamaIndex es un framework de código abierto diseñado específicamente para la creación de sistemas basados en workflows. Su propuesta principal es facilitar la conexión entre grandes modelos de lenguaje (LLMs) y datos externos, estructurados o no estructurados, a través de una arquitectura modular, extensible y trazable.

A diferencia de otros entornos más generalistas, como LangChain o Haystack, LlamaIndex está concebido desde su base para trabajar con flujos de procesamiento complejos, especialmente en entornos donde se necesita controlar con precisión el contenido indexado, la forma en que se consulta y cómo se generan las respuestas.

Uno de los pilares fundamentales de LlamaIndex es su modelo de workflows componibles, que permite estructurar un sistema como una secuencia ordenada de pasos (*steps*), donde cada etapa cumple una función clara y se comunica con las demás a través de eventos. Gracias a esta estructura, se pueden definir procesos altamente personalizables [12].

El uso de workflows en LlamaIndex no solo organiza el código de forma limpia y reutilizable, sino que además permite construir sistemas complejos con una lógica fácilmente trazable y auditable. Esto resulta especialmente útil en contextos empresariales, donde es fundamental entender cómo se ha procesado la información y qué decisiones ha tomado el sistema en cada momento.

Ventajas técnicas clave del enfoque de LlamaIndex:

1. **Modularidad y reutilización:** cada paso del flujo se implementa como una unidad independiente, lo que facilita la reutilización, la prueba individual y el mantenimiento de cada componente del sistema.
2. **Trazabilidad total:** la estructura basada en eventos permite registrar y seguir el recorrido de cada entrada, saber qué transformaciones se han aplicado y en qué punto del flujo se encuentra la información.
3. **Configurabilidad dinámica:** los workflows pueden inicializarse mediante configuraciones externas (como archivos JSON o modelos Pydantic), lo que permite cambiar el comportamiento del sistema sin necesidad de modificar el código.
4. **Desacoplamiento y extensibilidad:** cada etapa puede delegar su lógica a herramientas externas (tools), permitiendo una separación clara entre el diseño del flujo y la lógica específica de negocio.

5. **Integración fluida con componentes externos:** como bases de datos vectoriales, extractores de metadatos, sintetizadores de respuesta, evaluadores de calidad, y agentes inteligentes.
6. **Asincronía nativa y ejecución escalable:** gracias a su arquitectura orientada a eventos, LlamaIndex permite ejecutar workflows de forma asíncrona, escalar su ejecución por etapas y supervisar el estado de cada proceso [11].

3.3.2. Arquitectura y ejecución de workflows en LlamaIndex

La arquitectura de workflows de LlamaIndex constituye uno de los pilares fundamentales del framework y una de las razones clave por las que ha sido seleccionado como base tecnológica en este trabajo. Lejos de proponer sistemas monolíticos o procesos ocultos en bloques de código indivisibles, LlamaIndex permite estructurar el comportamiento del sistema como un flujo de pasos independientes, conectados entre sí mediante eventos. Esta estructura modular no solo aporta claridad, sino también control, trazabilidad y adaptabilidad, cualidades esenciales cuando se trabaja con información empresarial.

Un workflow en LlamaIndex se define como una secuencia de pasos, o *steps*, que transforman datos progresivamente. Cada uno de estos pasos es una función decorada con *@step*, y representa una unidad de lógica especializada, como puede ser analizar un texto, extraer entidades, filtrar resultados o sintetizar respuestas. Sin embargo, lo verdaderamente distintivo de esta arquitectura es que los pasos no se invocan directamente entre ellos. En su lugar, se comunican exclusivamente a través de objetos estructurados llamados eventos (*Events*). Estos eventos encapsulan la información relevante que un paso necesita para ejecutarse, y constituyen la base sobre la que se orquesta toda la lógica del workflow.

El flujo de ejecución comienza siempre con un *StartEvent*, que activa uno o varios pasos iniciales según lo definido en la arquitectura. Estos pasos reciben el evento, ejecutan su función, y emiten un nuevo evento como salida. Dicho evento será recibido por otro o otros pasos que esperan ese tipo específico de entrada, y así continúa la ejecución paso a paso. Esta dinámica no es secuencial por defecto, los pasos son activados explícitamente mediante el tipo de evento que reciben, lo que permite controlar con precisión el flujo, incluir bifurcaciones o ramificaciones, y adaptarse dinámicamente según las necesidades. La ejecución finaliza con un *StopEvent*, que marca el cierre del proceso y puede usarse para llevar a cabo operaciones finales, como validaciones, logs, auditorías o notificaciones de estado.

Este modelo orientado a eventos proporciona una serie de ventajas técnicas que hacen que los workflows de LlamaIndex sean especialmente adecuadas para sistemas complejos y sensibles al control del flujo. En primer lugar, garantiza un desacoplamiento total entre las etapas, cada paso se implementa como un módulo autónomo, que no depende del orden del flujo ni del paso anterior, sino únicamente del evento que recibe. Esto permite construir componentes reutilizables, que se pueden probar de forma independiente y fácilmente intercambiables. En segundo lugar, aporta trazabilidad completa al estar cada dato encapsulado en un evento bien definido, es posible auditar el recorrido de la información desde la entrada hasta la salida, facilitando la depuración y el análisis de decisiones tomadas por el sistema.

Además, el sistema permite definir workflows de distinta complejidad. Se pueden construir flujos lineales, donde los pasos se ejecutan en orden; paralelos, donde varios pasos se activan simultáneamente a partir de un mismo evento; condicionales, donde la ejecución depende del contenido de los eventos; o incluso flujos jerárquicos, donde un workflow contiene sub-workflows que actúan como bloques reutilizables. Esta versatilidad convierte a LlamaIndex en una herramienta muy potente, capaz de adaptarse tanto a tareas simples como a arquitecturas altamente sofisticadas.

Otro aspecto clave es la capacidad de configuración dinámica. Los workflows pueden definirse y ejecutarse a partir de configuraciones externas mediante estructuras como *from_config*, lo que permite alterar su comportamiento sin modificar el código fuente. De este modo, se facilita la adaptación a distintos entornos de despliegue o a diferentes necesidades empresariales, promoviendo la reutilización y el mantenimiento sostenible del sistema.

Por último, cabe destacar que cada paso dentro de un workflow puede delegar su lógica en herramientas externas específicas, denominadas *tools*. Esto permite separar completamente el diseño del flujo (es decir, la estructura del sistema) de la lógica de negocio concreta. Por ejemplo, un paso de análisis puede delegar en una herramienta de resumen, una herramienta de clasificación o una herramienta de extracción de metadatos, en función de los objetivos definidos. Este patrón de diseño basado en delegación favorece la extensibilidad, ya que nuevas herramientas pueden incorporarse sin alterar la arquitectura existente.

En conjunto, la arquitectura de workflows de LlamaIndex proporciona una base robusta y flexible para la construcción de sistemas de procesamiento inteligente de información. Su enfoque composable, la ejecución basada en eventos y su capacidad de trazabilidad y configuración hacen de este modelo una solución altamente alineada con los principios de diseño moderno de software, como la responsabilidad única, la inyección de dependencias y la orientación a eventos. Estas características la convierten en una pieza central para la propuesta técnica de este trabajo, ya que permite construir soluciones controladas, auditables y fácilmente adaptables al contexto empresarial real.

3.4. Configuración modular y parametrización del workflow

Una de las características más relevantes del sistema propuesto es su capacidad para adaptarse dinámicamente a distintos contextos empresariales sin necesidad de modificar su código fuente. Esta flexibilidad se logra gracias a una arquitectura de configuración modular, que permite parametrizar el comportamiento tanto del workflow de indexación como del workflow de recuperación mediante un único esquema declarativo.

La configuración se externaliza en forma de un diccionario estructurado que define parámetros clave como las herramientas utilizadas en cada etapa, los modelos de embeddings empleados, los agentes inteligentes activados, así como las estrategias específicas de segmentación y extracción de metadatos. Esta abstracción facilita la reutilización del sistema en diferentes dominios (como documentación legal, correos electrónicos o informes técnicos), manteniendo inalterado el núcleo de ejecución.

En este apartado se describe cómo se organiza y aplica dicha configuración dentro de los workflows, utilizando el método *from_config* y los bloques principales *settings* y *agents*. Este

enfoque no solo permite construir flujos especializados sin modificar su estructura interna, sino que también garantiza la trazabilidad, reproducibilidad y escalabilidad del sistema.

A continuación, se presentan los distintos componentes que conforman este modelo de configuración, comenzando por el mecanismo de inicialización y continuando con la estructura de parámetros que permiten adaptar el comportamiento de cada workflow a los requisitos específicos de su contexto de ejecución.

3.4.1. Inicialización con `from_config`: principio de configuración como código

El mecanismo de inicialización mediante *from_config* constituye uno de los pilares clave del diseño modular del sistema propuesto. Este método permite construir automáticamente una instancia funcional de cualquier workflow ya sea de indexación o de recuperación, a partir de un diccionario de configuración externo. Esta estrategia proporciona un alto grado de flexibilidad, favorece la adaptabilidad a distintos contextos empresariales y garantiza un desacoplamiento efectivo entre la lógica de ejecución y su implementación concreta.

Frente a enfoques más rígidos basados en la codificación manual de parámetros, el uso de *from_config* promueve una arquitectura declarativa, en la que el comportamiento del sistema puede modificarse sin alterar el código fuente de la clase que define el flujo. Esto facilita tanto la evolución del sistema como su personalización para distintos tipos de datos, tareas o entornos.

El método recibe como entrada un diccionario estructurado en 2 bloques funcionales (*settings* y *agents*) que contienen los parámetros necesarios para instanciar herramientas, modelos y componentes auxiliares. A partir de esta información, se invocan funciones especializadas para inicializar elementos como los parsers (analizadores de contenido), los modelos de embeddings, los extractores de metadatos o los agentes inteligentes. Todos estos elementos quedan integrados en la instancia final del workflow, completamente operativa bajo la configuración indicada.

Este enfoque permite reutilizar la misma definición de workflow en múltiples contextos, adaptándola dinámicamente a dominios tan diversos como la gestión de correos electrónicos, la recuperación de normativas legales o el análisis de documentación técnica. Además, favorece una separación clara entre el diseño estructural del sistema (estable y genérico) y su parametrización concreta (específica y fácilmente modificable).

Más allá de su utilidad práctica, *from_config* también refuerza la trazabilidad y la documentación del sistema. Al centralizar todos los parámetros relevantes en una única fuente de configuración, se facilita la comprensión global del comportamiento del sistema, su replicación en distintos entornos y la colaboración entre equipos. Esta filosofía de configuración como código no solo mejora la transparencia y mantenibilidad de la solución, sino que además refuerza su capacidad de adaptación a futuros requisitos sin comprometer la robustez del diseño inicial.

```
config = {  
  "agents": {  
    "query_rewriter": {...},  
    "response_synthesizer": {...},  
    ...  
  },  
  "settings": {  
    "vector_store": { ... },  
    "embedding_model": { ... },  
    ...  
  }  
}
```

Figura 5. Estructura general de la configuración de un workflow.

La **Figura 5** muestra un ejemplo simplificado del diccionario *config* utilizado para inicializar un workflow a través del método *from_config*. En esta estructura se distinguen dos bloques principales: *agents*, donde se definen los agentes inteligentes involucrados en las distintas etapas del flujo, y *settings*, que agrupa los parámetros técnicos esenciales como el modelo de embeddings, el vector store o las herramientas personalizadas asociadas a cada paso del workflow. Esta organización permite una inicialización dinámica y desacoplada del flujo, facilitando la adaptación a diferentes casos de uso sin modificar el código fuente.

3.4.2. Estructura del bloque settings: definición del flujo de ejecución

El bloque *settings* constituye uno de los componentes clave del diccionario de configuración que permite parametrizar el comportamiento de cada workflow. Este bloque agrupa los parámetros técnicos y operativos que definen cómo se ejecuta el flujo, qué herramientas se emplean en cada etapa, qué modelos se utilizan, y qué transformaciones o tratamientos adicionales deben aplicarse sobre los datos. Su función es proporcionar una forma estructurada, declarativa y externa de controlar todos los aspectos críticos del flujo, sin necesidad de alterar el código fuente.

Entre los parámetros más relevantes del bloque *settings* se encuentran:

- **vector_store**: define el sistema de almacenamiento vectorial donde se guardarán los nodos una vez transformados. Puede configurarse con distintas bases de datos, como MongoDB, ChromaDB, en función del despliegue deseado.
- **embedding_model**: especifica el modelo de embeddings que se utilizará para vectorizar los fragmentos textuales (nodos). Este modelo es imprescindible para que los contenidos puedan ser recuperados por similitud semántica en la fase de recuperación. Si no se indica explícitamente, el sistema utiliza por defecto el modelo *text-embedding-3-large* de OpenAI, por su buen rendimiento y compatibilidad general.

- **tools**: es un diccionario que asocia cada etapa del workflow con una herramienta externa (tool) concreta. Cada tool encapsula la lógica específica de un paso del flujo, como puede ser un parser, un filtro semántico o un extractor de metadatos.
- **parser_params**: en el caso del workflow de indexación, permite pasar parámetros personalizados al parser (analizador sintáctico) utilizado para leer y extraer nodos desde los documentos de entrada.
- **metadata_extractors**: define una lista de extractores adicionales que pueden aplicarse durante la transformación de nodos. Estos extractores como *SummaryExtractor*, *TitleExtractor*, o *QuestionsAnsweredExtractor* enriquecen el contenido con información estructurada adicional relevante.
- **retrievers**, **synthesizers**, **cohere_reranker**, **add_summary_nodes**, **add_related_nodes**, entre otros: son parámetros propios del flujo de recuperación, que definen cómo recuperar información, cómo generar respuestas, y qué estrategias de enriquecimiento y evaluación aplicar durante el proceso de generación.
- **max_attempts**: define el número de veces que podemos volver a ejecutar el proceso del workflow de recuperación si la respuesta final generada no responde correctamente a la pregunta del usuario

No todos los parámetros son comunes a ambas workflows. Algunos, como *tools* o *parser_params*, son exclusivos del workflow de indexación, mientras que otros como *retrievers*, *synthesizers* o *cohere_reranker* solo tienen sentido en la recuperación. Esta separación permite una especialización clara de cada flujo, manteniendo la coherencia y simplicidad del modelo de configuración.

A continuación, se presenta una tabla que resume los principales parámetros configurables en el bloque *settings*, indicando su aplicabilidad concreta en cada uno de los dos workflows:

Parámetros	Workflow de Indexación	Workflow de Recuperación
vector_store		
embedding_model		
tools		

parser_params	✓	✗
metadata_extractors	✓	✗
retrievers	✗	✓
synthesizers	✗	✓
max_attempts	✗	✓
cohere_reranker	✗	✓
add_summary_nodes	✗	✓
add_related_nodes	✗	✓

Tabla 3. Comparativa de parámetros settings en los workflows de indexación y recuperación.

En la **Tabla 3** se puede identificar de forma rápida qué opciones de configuración están disponibles para cada tipo de flujo, y facilita el diseño de configuraciones especializadas según los requisitos del caso de uso. Además, pone de manifiesto la complementariedad entre ambas workflows y la manera en que cada una explota distintos aspectos del modelo composable definido por LlamaIndex. Cada fila representa un parámetro configurable dentro del bloque settings, y las columnas indican si dicho parámetro está habilitado o no en el workflow correspondiente. El símbolo ✓ indica que el parámetro puede ser utilizado y forma parte activa del diseño de ese flujo; por el contrario, el símbolo ✗ señala que dicho parámetro no es relevante o no se emplea en ese workflow concreto.

3.4.3. Estructura del bloque agents: definición de agentes inteligentes

El bloque *agents* define los componentes inteligentes del sistema, basados en modelos de lenguaje, encargados de realizar tareas especializadas dentro de cada workflow. A diferencia de las herramientas funcionales que ejecutan transformaciones deterministas, los agentes trabajan

sobre el contenido de forma contextual e inferencial, generando salidas complejas como resúmenes, respuestas, reformulaciones o evaluaciones.

Este enfoque dota al sistema de una capa de razonamiento flexible y configurable, en la que cada agente puede adaptarse a una función concreta según el dominio y la etapa del flujo. Además, cada agente puede activarse o desactivarse de forma independiente, permitiendo una configuración granular del comportamiento general del sistema.

Cada agente se define mediante una estructura JSON o un diccionario que especifica:

- **name:** nombre identificador del agente.
- **cls:** clase responsable del comportamiento (por ejemplo, *SimpleChatEngine*).
- **params:** parámetros de configuración, que pueden incluir:
 - **system_prompt:** prompt base que guía el razonamiento del agente.
 - **llm:** definición del modelo de lenguaje utilizado (proveedor, nombre, parámetros, formato de respuesta...).

Esta configuración modular permite definir agentes muy específicos, incluyendo el control del formato de salida mediante esquemas JSON estructurados. A continuación, se muestra un ejemplo típico de configuración de agente con validación estructurada:

```

"agent_name": {
  name: "identificador_unico_del_agente",
  cls: "ClaseDelAgente", # p.ej., SimpleChatEngine
  params: {
    system_prompt: "Define el comportamiento del agente",
    llm: {
      cls: "ProveedorLLM", # p.ej., OpenAI
      params: {
        model: "modelo_llm_usado", # p.ej., gpt-4o-mini
        additional_kwargs: {
          response_format: {
            type: "json_schema",
            json_schema: {
              name: "nombre_del_esquema",
              schema: {
                type: "object",
                properties: {
                  propiedad_1: {
                    type: "tipo_de_dato",
                    description: "¿Qué debe incluir esta propiedad?"
                  },
                  propiedad_2: { ... }
                },
                required: ["propiedad_1", "propiedad_2"],
                additionalProperties: false
              }
            }
          }
        }
      }
    }
  }
}

```

Figura 6. Ejemplo de configuración avanzada de un agente inteligente.

En la **Figura 6** se muestra un ejemplo de definición estructurada de un agente dentro del bloque agents del diccionario config. Esta configuración incluye el nombre único del agente, la clase utilizada para su ejecución, el prompt de sistema que define su comportamiento, así como los parámetros del modelo de lenguaje subyacente. Este enfoque declarativo permite ajustar con precisión la lógica de cada agente según la tarea que se le asigne, facilitando su reutilización y adaptación en distintos contextos sin modificar el código del workflow.

En el sistema propuesto, los agentes inteligentes juegan un papel clave tanto en el flujo de indexación como en el de recuperación. El workflow de indexación, su principal función es enriquecer los nodos antes de ser almacenados, añadiendo información generada automáticamente a partir del contenido original. En el caso del workflow de recuperación, los agentes intervienen en etapas críticas como la interpretación de la consulta del usuario, la síntesis de la respuesta o la validación de su coherencia y fidelidad. Esta separación funcional permite que cada workflow utilice únicamente los agentes que son relevantes para su propósito, garantizando eficiencia y especialización. Además, el sistema permite configurar con precisión qué agentes se activan en cada caso, lo que posibilita la creación de flujos personalizados para distintos dominios y casos de uso. A continuación, se presenta una tabla que recoge todos los agentes disponibles, especificando su función principal y el flujo en el que se integran.

A continuación, se presenta la **Tabla 4**, que recoge todos los agentes inteligentes integrados en el sistema, junto con su función principal, el workflow en el que participan y su carácter opcional dentro del diseño general.

Agente Inteligente	Función principal	Workflow asociado	Opcionalidad
document_name_extractor_agent	Sacar el nombre real del documento	Indexación	Opcional
document_summarizer	Generar un resumen del texto de un nodo	Indexación	Opcional
document_relation_agent	Detectar relaciones entre secciones o nodos	Indexación	Opcional
question_extractor	Formular preguntas relevantes a partir del contenido de los nodos	Indexación	Opcional
response_synthesizer	Generar una respuesta a la pregunta del usuario a partir de los nodos recuperados	Recuperación (RAG)	Opcional
query_rewriter	Genera nuevas preguntas a partir de la pregunta original del usuario	Recuperación (RAG)	Opcional
query_fusioner	Fusiona todas las preguntas generadas en una completa	Recuperación (RAG)	Opcional
faithfulness_evaluator	Verificar la coherencia interna de la respuesta con la pregunta del usuario	Recuperación (RAG)	Opcional
consolidate_synthesizer	Fusiona un conjunto de respuestas generadas en una respuesta final completa	Recuperación (RAG)	Opcional
judge_agent	Evalúa si la respuesta final responde correctamente a la pregunta del usuario	Recuperación (RAG)	Opcional

Tabla 4. Agentes inteligentes del sistema y su rol en los distintos workflows.

La **Tabla 4** recoge todos los agentes inteligentes integrados en el sistema y resume su propósito, el flujo en el que intervienen y su carácter opcional dentro del diseño. Cada uno de estos agentes puede configurarse con su propio modelo de lenguaje, sistema de prompts y esquema de salida, gracias a la estructura declarativa de la configuración. Esta flexibilidad permite diseñar flujos

adaptativos y especializados, donde los agentes actúan como componentes intercambiables según el objetivo del sistema: enriquecimiento del contenido, mejora en la formulación de consultas, o síntesis de respuestas precisas y auditables.

3.4.4. Personalización del flujo según fuente o consulta

Uno de los puntos fuertes del sistema propuesto es su capacidad para adaptarse de forma dinámica al tipo de contenido que se desea procesar o consultar. Esta flexibilidad se consigue gracias a una arquitectura parametrizable, en la que tanto el workflow de indexación como el workflow de recuperación pueden configurarse mediante estructuras externas, sin necesidad de modificar el código fuente. Esta capacidad resulta esencial en contextos empresariales donde coexisten múltiples tipos de información: correos electrónicos, informes legales, fichas técnicas, actas de reunión, entre otros.

En el caso del workflow de indexación, esta adaptabilidad permite definir diferentes pipelines en función del tipo de documento. Por ejemplo, es posible activar un parser especializado para correos electrónicos, un extractor de entidades jurídicas para informes normativos, o un clasificador temático para documentos técnicos. También se pueden configurar qué agentes inteligentes deben ejecutarse en cada caso, por ejemplo, decidir si generar resúmenes, extraer preguntas o establecer relaciones internas, dependiendo de la naturaleza del contenido. Esto se define a través del bloque *settings* y del bloque *agents* dentro del diccionario de configuración, lo que permite personalizar la segmentación, la transformación y el enriquecimiento semántico de los nodos generados.

Por su parte, el workflow de recuperación también admite una alta personalización. Según el tipo de pregunta o el dominio de aplicación, pueden configurarse distintas estrategias de recuperación (recursiva, vectorial o jerárquica), distintos sintetizadores de respuesta y distintos módulos de evaluación de fidelidad o consolidación. Por ejemplo, una consulta legal puede activar recuperadores con filtros por tipo de documento y sintetizadores formales, mientras que una consulta sobre un producto puede usar un sintetizador directo con explicaciones más concisas. Además, la lógica de enrutamiento de la consulta también puede personalizarse a través de agentes como *response_synthesizer* o *query_rewriter*, adaptando el tratamiento de cada pregunta al contexto en el que se formula.

Esta personalización basada en configuración no solo mejora la calidad de los resultados y la eficiencia del sistema, sino que también favorece la reutilización de código y facilita el mantenimiento. Basta con modificar un archivo de configuración para adaptar el comportamiento de ambos workflows a un nuevo tipo de fuente o necesidad de negocio, sin tocar la lógica principal de ejecución. Este enfoque composable y adaptativo refuerza el carácter versátil del sistema, permitiendo su despliegue en múltiples dominios sin perder control ni trazabilidad.

3.5. Conclusiones

En este capítulo se ha presentado la visión general del sistema propuesto, detallando tanto el problema que se pretende resolver como la solución técnica planteada a través de dos workflows complementarios, uno para la indexación de contenidos y otro para la recuperación de

información basada en conocimiento previamente estructurado. También se ha introducido el marco técnico sobre el que se apoya toda la arquitectura.

A lo largo del capítulo se han analizado los fundamentos que hacen de esta tecnología una elección adecuada para entornos empresariales, destacando su capacidad de adaptación, su trazabilidad, y su enfoque composable basado en pasos y eventos. Este modelo no solo aporta claridad estructural, sino que permite desarrollar soluciones inteligentes con un alto grado de control y personalización, facilitando tanto la integración con herramientas externas como la evolución futura del sistema.

Con esta base establecida, los próximos capítulos se centrarán en la descripción detallada de cada uno de los dos flujos principales del sistema. En el capítulo 4 se analizará en profundidad el workflow de indexación, explicando cada una de sus etapas, eventos y decisiones de diseño. Posteriormente, en el capítulo 5, se abordará el workflow de recuperación, centrado en cómo se generan respuestas coherentes y fundamentadas a partir del contenido indexado. Ambos capítulos mostrarán no solo la lógica interna de cada flujo, sino también cómo se integran dentro del sistema global para cerrar el ciclo completo de gestión del conocimiento empresarial.

4. Diseño e implementación del workflow de Indexación

La automatización del proceso de indexación de contenidos es el primer paso crítico para garantizar la trazabilidad, estructuración y reutilización del conocimiento empresarial. En este capítulo se describe de forma detallada el diseño y la implementación del workflow de indexación, uno de los dos flujos principales que conforman el sistema propuesto.

A diferencia del Capítulo 3, donde se presentó una visión general del sistema y se introdujo el funcionamiento técnico de los workflows en LlamaIndex, aquí se aborda específicamente la construcción del flujo encargado de transformar entradas heterogéneas (como documentos, correos electrónicos o informes) en una representación estructurada denominada nodos. Estos nodos, una vez almacenados, conforman la base del conocimiento sobre la que operará posteriormente el sistema de recuperación.

El capítulo se organiza en varias secciones. En primer lugar, se describe la arquitectura del workflow de indexación, explicando su estructura interna y cómo se integra dentro del sistema general. Posteriormente, se analiza cada una de las etapas funcionales que lo componen, detallando los pasos técnicos, eventos implicados y herramientas utilizadas. A continuación, se presenta el modelo de configuración modular, que permite adaptar el flujo a distintos contextos y tipos de contenido. Finalmente, se discuten aspectos relacionados con el diseño desacoplado, los retos técnicos enfrentados durante su desarrollo y las decisiones adoptadas para lograr una implementación robusta y escalable.

4.1. Arquitectura del workflow de indexación

4.1.1. Estructura de la clase `IndexingWorkflow`

La clase *IndexingWorkflow* representa la implementación específica del flujo de trabajo encargado de indexar contenidos multiformatos dentro del sistema propuesto. Su diseño sigue el modelo de ejecución por pasos asincrónicos definido por el framework LlamaIndex, en el que cada etapa del flujo se implementa como un método decorado con `@step` y está conectado con el resto del sistema mediante eventos tipados. Esta estructura permite una ejecución modular, flexible y trazable, adaptada a entornos empresariales donde los datos pueden ser heterogéneos y altamente dinámicos.

La clase hereda de la clase base *Workflow*, y en su constructor define tres componentes clave: la configuración general del flujo (*config*), el conjunto de herramientas funcionales (*tools*) y los parámetros de control (*workflow_params*) como el timeout. Las herramientas se agrupan en un diccionario llamado *toolkit*, que contiene las implementaciones necesarias para que cada paso del flujo pueda delegar su lógica en un componente externo especializado. Esta arquitectura basada en herramientas desacopladas permite sustituir fácilmente cualquier parte del flujo sin afectar a la lógica global del sistema.

El flujo está compuesto por siete etapas, cada una implementada como un método asíncrono decorado con *@step*:

1. **parse_source_references**: recibe el evento inicial (*StartEvent*) y se encarga de procesar la entrada del usuario para obtener una lista de referencias a indexar.
2. **nodes_extraction_from_references**: toma las referencias anteriores y extrae de ellas nodos de contenido utilizando el parser configurado.
3. **nodes_filtering**: aplica reglas de filtrado sobre los nodos extraídos, eliminando duplicados, vacíos o irrelevantes.
4. **nodes_processing**: aplica agentes inteligentes configurables para enriquecer los nodos mediante generación de resúmenes y detección de relaciones internas o externas. Puede generar nuevos nodos derivados y añadir metadatos contextuales, en función de la configuración establecida.
5. **nodes_transformation**: transforma los nodos mediante la aplicación de embeddings, segmentación y extracción de metadatos. Prepara los nodos para su almacenamiento estructurado, generando los chunks finales listos para indexar.
6. **add_nodes_to_db**: guarda los nodos transformados en una base de datos estructurada (por ejemplo, MongoDB o un vector store).
7. **post_indexing_process**: ejecuta cualquier lógica final posterior al almacenamiento, como validaciones, logs o generación de respuestas de estado.

Cada una de estas etapas está conectada mediante eventos específicos, definidos también en la clase: *ReferencesNodesParsingEvent*, *NodesFilteringEvent*, *NodesProcessingEvent*, *NodesTransformationEvent*, *AddingNodesToDBEvent*, *PostIndexingProcessEvent*, y finalmente *StopEvent*. Esto garantiza que el flujo pueda trazarse con precisión en cada momento, y que el sistema solo avance cuando se emita el evento correspondiente.

La clase también incluye el método de clase *from_config*, que permite inicializar todo el flujo a partir de un diccionario externo de configuración. Este método carga dinámicamente las herramientas necesarias, a partir de sus nombres, utilizando funciones auxiliares, y permite definir todos los parámetros sin necesidad de modificar el código fuente. Esta capacidad de configuración como código permite adaptar la misma clase de workflow a distintos entornos o tipos de documentos (por ejemplo, correos electrónicos, informes técnicos o fichas legales), manteniendo el núcleo de ejecución estable.

4.1.2. Diagrama del flujo completo de ejecución

El workflow de indexación está compuesto por una secuencia estructurada de pasos que se ejecutan de forma asincrónica, en respuesta a eventos específicos. Esta organización modular permite transformar cualquier contenido de entrada, ya sean documentos, correos o fragmentos textuales, en una representación indexada, lista para ser consultada. Para visualizar el

comportamiento global del sistema, se presenta a continuación un diagrama de flujo que ilustra todas las etapas del proceso y los eventos que las conectan.

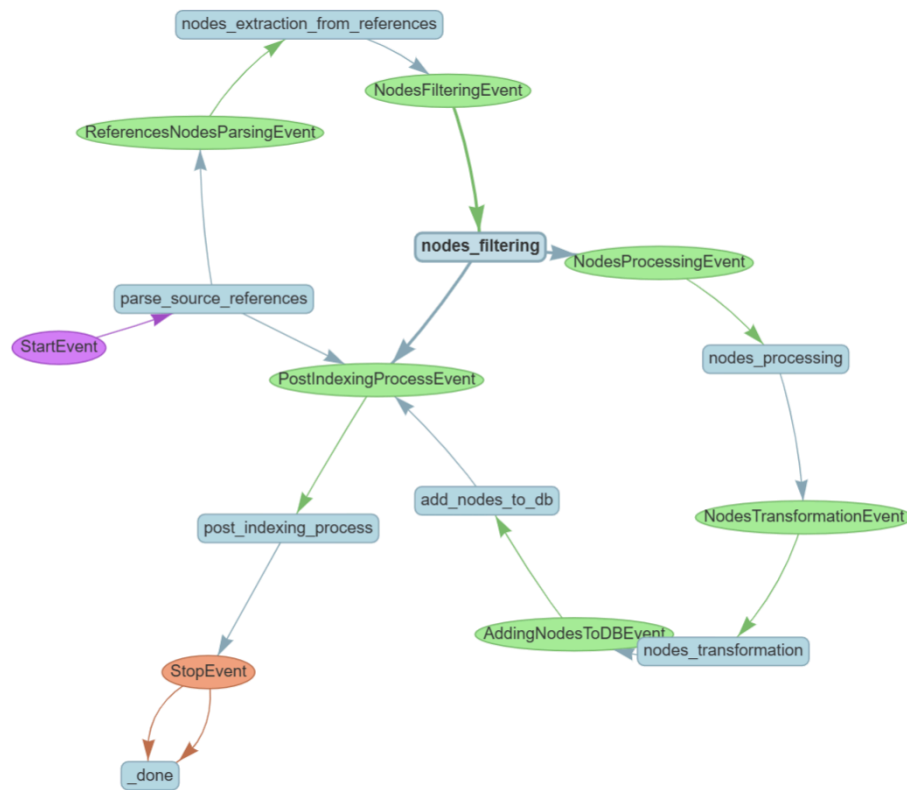


Figura 7. Diagrama de ejecución del workflow de indexación.

Para comprender el comportamiento global del sistema, la **Figura 7** presenta un diagrama visual que ilustra todas las etapas involucradas en el proceso de indexación, así como los eventos que gobiernan el avance entre ellas. Cada bloque azul representa una etapa (@step) del flujo de trabajo, mientras que cada óvalo verde indica un tipo de evento que transporta los datos desde una etapa hasta la siguiente. Como puede observarse, el flujo comienza con un *StartEvent*, que activa el paso de parsing de referencias. A partir de ahí, cada paso transforma la información y emite un nuevo evento que será consumido por el siguiente step. El recorrido finaliza con un *StopEvent*, tras ejecutar el proceso final opcional de post-indexación. Esta representación resulta especialmente útil para entender la lógica completa antes de adentrarse en el detalle de cada paso. En las secciones siguientes del capítulo, se analizará individualmente cada etapa, explicando su función, herramientas asociadas y configuración.

4.1.3. Encaje dentro del sistema global

El workflow de indexación no opera de forma aislada, sino que constituye la puerta de entrada fundamental del sistema de gestión del conocimiento. Su función es preparar de forma estructurada todo el contenido sobre el que más adelante se podrá razonar, buscar o responder preguntas. En ese sentido, actúa como la fase previa y necesaria para que el workflow de recuperación pueda funcionar con garantías de precisión, trazabilidad y relevancia.

Una vez completado el flujo de indexación, los nodos generados son almacenados en una base de datos estructurada, junto con sus metadatos, embeddings, identificadores de fragmento y atributos semánticos. Este conjunto de nodos constituye el conocimiento indexado del sistema, es decir, la representación unificada del contenido que los agentes inteligentes pueden consultar posteriormente.

El flujo de recuperación accede directamente a este contenido estructurado. Cuando el usuario plantea una consulta, el sistema activa el workflow de recuperación, que lanza procesos de recuperación sobre los nodos generados por el workflow de indexación. A partir de esos nodos recuperados, el sistema genera una respuesta fundamentada y contextualizada, garantizando que el contenido devuelto esté alineado exclusivamente con la información validada y previamente indexada.

Esta separación entre fase de incorporación del conocimiento (*indexación*) y fase de acceso al conocimiento (*recuperación*) responde a un principio de diseño que favorece la robustez y el mantenimiento del sistema. Además, permite actualizar el corpus indexado de forma independiente a la lógica de recuperación, y viceversa. La modularidad también facilita tareas como re-indexar contenido modificado, aplicar nuevos métodos de chunking, o realizar auditorías sobre el conocimiento utilizado en una respuesta determinada.

4.2. Descripción técnica y funcional de cada etapa

4.2.1. Parsing de referencias (*parse_source_references*)

El primer paso del workflow de indexación se encarga de interpretar la consulta inicial del usuario para determinar qué contenido debe ser procesado e indexado. Esta etapa, denominada *parsing de referencias*, permite traducir una entrada genérica, como una solicitud de recuperar los archivos o correos electrónicos, en un conjunto de referencias estructuradas que servirán como base para las etapas posteriores del flujo.

El funcionamiento de esta etapa parte de la recepción de un evento inicial (*StartEvent*) que contiene, en su interior, una consulta (*query*) que describe qué elementos deben ser incorporados (indexados) al sistema. Esta consulta puede adoptar múltiples formas, desde rutas locales a archivos, hasta identificadores de carpetas en la nube, búsquedas en hilos de correo o entradas en plataformas como Salesforce. En lugar de procesar directamente esta entrada dentro del flujo, el sistema delega su interpretación a una herramienta especializada que ha sido previamente definida en la configuración del workflow. Esta herramienta se selecciona dinámicamente desde el diccionario *tools*, bajo la clave *parse_source_references_tool*, y su función es transformar la consulta original en una lista concreta de referencias normalizadas.

El diseño modular del sistema permite que esta herramienta se adapte a distintas fuentes de contenido. Por ejemplo, el sistema puede utilizar una herramienta para acceder a OneDrive si se desea indexar documentos almacenados en la nube, o una diferente si el contenido proviene de hilos de correo electrónico o entradas de bases de datos. Esta flexibilidad es esencial para garantizar que el mismo flujo de indexación pueda utilizarse en entornos muy distintos sin necesidad de reescribir su lógica interna, simplemente cambiando la configuración de la herramienta de parsing.

En caso de que no se haya definido una herramienta específica para esta etapa, el sistema asume que la entrada proporcionada ya es una lista de referencias válidas y continúa el flujo sin transformación adicional. Por el contrario, si la herramienta devuelve una lista vacía, el flujo se interrumpe mediante un evento de tipo *PostIndexingProcessEvent*, lo que evita realizar trabajo innecesario cuando no hay información que procesar.

Cuando el parsing es exitoso, se emite un evento *ReferencesNodesParsingEvent*, que contiene la lista de referencias generadas. Este evento será consumido por la siguiente etapa del workflow, encargada de extraer nodos de información a partir de estas referencias. De este modo, la etapa de parsing de referencias actúa como una interfaz entre la intención del usuario y la lógica operativa del sistema, permitiendo adaptar el flujo de indexación a múltiples dominios sin comprometer su estructura general.

Gracias a este enfoque desacoplado, basado en herramientas externas y configuración dinámica, el sistema adquiere una gran versatilidad, convirtiendo esta etapa en una pieza clave para lograr una automatización flexible y escalable del proceso de incorporación de conocimiento.

4.2.2. Extracción de nodos (*nodes_extraction_from_references*)

Una vez obtenidas las referencias estructuradas a partir de la consulta inicial del usuario, el flujo de trabajo de indexación procede a extraer el contenido útil que será transformado en conocimiento indexable. Esta etapa, conocida como *extracción de nodos*, tiene como finalidad procesar las referencias recibidas y convertirlas en unidades de información llamadas *nodos* que encapsulan fragmentos significativos del contenido original.

El sistema activa esta etapa en cuanto se recibe un evento de tipo *ReferencesNodesParsingEvent*, emitido por la fase anterior. Este evento contiene una lista de referencias estandarizadas, que pueden representar documentos, correos, informes o cualquier otra fuente estructurada. Para realizar la extracción, el flujo delega la operación a una herramienta definida en la configuración como *nodes_extraction_from_references_tool*. Esta herramienta actúa como un *parser* (analizador del contenido), y su función principal es procesar cada referencia individual, acceder al contenido correspondiente y segmentarlo en bloques interpretables que puedan ser utilizados por los pasos posteriores.

Dado que el sistema está diseñado para operar en múltiples dominios, esta herramienta puede ser distinta según el tipo de contenido que se desee indexar. En el caso de documentos en formato PDF o DOCX, por ejemplo, se puede utilizar un *parser* como *LlamaParse* (un *parser* que ya viene creado por Llamaindex), capaz de preservar la estructura semántica del texto. Para correos electrónicos, se recurre a herramientas especializadas que extraen los cuerpos de los mensajes, sus metadatos, asuntos y remitentes, aplicando una lógica de desambiguación. En contextos más complejos, como registros legales o tickets de soporte, pueden emplearse *parsers* personalizados que identifican secciones específicas o extraen entidades clave. Todas estas variantes se implementan de forma intercambiable, y su uso depende únicamente de la herramienta configurada en el diccionario de *tools*.

Durante el proceso de extracción, la herramienta seleccionada puede aplicar una estrategia de segmentación del contenido, dividiendo el texto en fragmentos que respetan límites semánticos, de longitud o de estructura. El objetivo es que cada nodo represente una unidad informativa

compacta, coherente y trazable, lo que facilitará su recuperación posterior y la generación de respuestas más precisas. Estos nodos pueden enriquecerse desde el inicio con metadatos básicos como la fuente original, el nombre del archivo, la fecha o el tipo de contenido.

Una vez procesadas todas las referencias, la herramienta devuelve una lista de nodos que se encapsula en un evento de tipo *NodesFilteringEvent*, el cual será procesado por la siguiente etapa.

La extracción de nodos constituye uno de los pilares fundamentales del flujo de indexación. Es en esta etapa donde el sistema toma contacto con el contenido real y lo transforma en una representación operativa que podrá ser almacenada, embebida y consultada en fases posteriores. Por ello, su correcta configuración y su adaptación al tipo de fuente son determinantes para asegurar la calidad y utilidad del conocimiento que se generará.

4.2.3. Filtrado de nodos (*nodes_filtering*)

Una vez extraídos los nodos a partir del contenido original, el flujo de indexación continúa con una etapa de depuración cuyo objetivo es asegurar que únicamente se mantengan los fragmentos relevantes, válidos y útiles para su posterior procesamiento. Esta fase, conocida como *filtrado de nodos*, actúa como un mecanismo de control de calidad que permite eliminar información redundante, vacía o irrelevante antes de que esta sea procesada por los agentes inteligentes del sistema.

El paso se activa mediante la recepción de un evento *NodesFilteringEvent*, que transporta la lista de nodos generados por el *parser* en la etapa anterior. Cada nodo representa una unidad de información que puede haber sido segmentada en base a criterios de estructura, semántica o longitud, y puede contener texto, metadatos, y atributos adicionales. Sin embargo, no todos los nodos extraídos son necesariamente útiles. Algunos pueden estar vacíos, duplicados, demasiado breves o fuera del dominio de interés del sistema. Es por ello que esta etapa aplica una o varias funciones de filtrado para limpiar y refinar el conjunto de datos.

La lógica de filtrado es delegada a una herramienta externa definida como *nodes_filtering_tool* dentro de la configuración del workflow. Esta herramienta recibe la lista de nodos y devuelve una versión filtrada, ajustada a los criterios definidos. La herramienta puede aplicar reglas genéricas como eliminar nodos vacíos o sin contenido textual o reglas específicas adaptadas al dominio, como detectar fragmentos irrelevantes según palabras clave o patrones semánticos. Gracias a la arquitectura modular del sistema, estas herramientas pueden diseñarse de forma especializada y reutilizarse en flujos distintos con solo modificar la configuración.

Si no se ha configurado una herramienta de filtrado, el flujo continúa sin alteraciones, pasando todos los nodos al siguiente paso. No obstante, si se ha aplicado un filtro y el conjunto resultante está vacío, el sistema considera que no hay información útil que procesar y se interrumpe de forma controlada mediante un evento *PostIndexingProcessEvent*. Este mecanismo garantiza que el sistema no procese innecesariamente información vacía, y proporciona una vía clara para gestionar situaciones donde las referencias extraídas no contienen contenido indexable.

En condiciones normales, una vez aplicado el filtrado, el flujo emite un evento *NodesProcessingEvent*, que encapsula la lista final de nodos válidos. Estos nodos representan ya

una base refinada de conocimiento potencial, que será posteriormente enriquecida con transformaciones, resúmenes y otros procesos avanzados.

4.2.4. Procesamiento avanzado de nodos (*nodes_processing*)

Tras el filtrado de los nodos, el flujo de trabajo de indexación continúa con una etapa clave dedicada al enriquecimiento semántico del contenido. Esta fase, denominada *procesamiento avanzado*, tiene como objetivo aumentar el valor informativo de los nodos mediante la aplicación de agentes inteligentes que analizan, resumen y relacionan su contenido. Lejos de limitarse a modificar los nodos existentes, esta etapa permite incluso generar nuevos nodos derivados de los originales, optimizados para facilitar su posterior recuperación y contextualización.

La lógica de esta etapa se activa al recibir un evento de tipo *NodesProcessingEvent*, el cual contiene la lista de nodos válidos y filtrados. En lugar de operar directamente sobre estos elementos, el workflow delega el procesamiento a una herramienta configurable definida como *nodes_processing_tool*. Esta herramienta actúa como orquestador del procesamiento semántico y tiene la capacidad de invocar uno o varios agentes inteligentes, cada uno especializado en una tarea distinta. Gracias a esta arquitectura, el sistema puede personalizar, en cada ejecución, qué tipo de procesamiento se aplica a los nodos, proporcionando un nivel de flexibilidad particularmente útil en entornos con requisitos variables.

Una de las operaciones más relevantes en esta fase es la *generación de resúmenes* automáticos (summary generation). Por cada nodo recibido, se puede invocar un agente encargado de sintetizar su contenido textual en una versión más breve y representativa. A partir de estos resúmenes, se generan nuevos nodos que contienen únicamente la versión condensada del contenido. Estos nuevos nodos (que conviven con los originales) permiten posteriormente generar respuestas más eficientes, rápidas y ajustadas, sobre todo en sistemas donde la longitud del contexto es limitada o donde se desea ofrecer una vista general antes de entrar en detalles.

Además del resumen, el sistema puede aplicar agentes que identifican *relaciones entre nodos*, tanto internas como externas. Este análisis se basa en la detección de vínculos semánticos o estructurales entre fragmentos de contenido: por ejemplo, un nodo puede estar conceptualmente relacionado con otro que desarrolla la misma idea en mayor profundidad, que aporta ejemplos adicionales o que representa una continuación lógica. Estas relaciones se integran en los metadatos del nodo, ya sea como identificadores de secciones vinculadas o como referencias cruzadas a otros nodos. Este enriquecimiento no es visible directamente para el usuario, pero permite que, en la etapa de recuperación, el sistema tenga la capacidad de ampliar el contexto de una respuesta incluyendo fragmentos relacionados, lo que mejora significativamente la coherencia y la calidad del resultado final.

Una de las fortalezas del diseño de esta etapa es su capacidad de configuración dinámica. A través de la herramienta *nodes_processing_tool*, el sistema puede especificar qué agentes aplicar: puede utilizarse únicamente el generador de resúmenes, solo el extractor de relaciones, ambos de forma combinada, o incluso ninguno si se desea realizar un flujo de indexación más simple. Esta configuración se define externamente mediante un diccionario de parámetros, lo que permite adaptar el procesamiento a distintos tipos de documentos o casos de uso, sin necesidad de modificar el código del workflow.

Finalmente, una vez completadas las operaciones configuradas, el sistema emite un evento *NodesTransformationEvent*, que encapsula todos los nodos resultantes tanto los originales como los derivados, listos para ser transformados y almacenados. Esta etapa, por tanto, no solo añade valor semántico al contenido, sino que sienta las bases para una recuperación más contextualizada, trazable y eficaz.

4.2.5. Transformación de nodos (*nodes_transformation*)

Finalizado el procesamiento semántico, el siguiente paso en el flujo de trabajo consiste en preparar los nodos para su almacenamiento definitivo. Esta etapa, denominada *transformación de nodos*, tiene como objetivo aplicar una serie de operaciones estructurales y semánticas sobre los nodos para garantizar que cumplan con los requisitos técnicos necesarios para ser almacenados en la base de datos y posteriormente utilizados en procesos de recuperación. A diferencia de las etapas anteriores, esta fase se centra en convertir el contenido procesado en una representación computacionalmente eficiente y estructuralmente uniforme.

El sistema activa esta etapa al recibir un evento de tipo *NodesTransformationEvent*, que contiene tanto los nodos originales como aquellos derivados de etapas anteriores (por ejemplo los nodos con resúmenes generados automáticamente). La herramienta encargada de ejecutar esta transformación está definida como *nodes_transformation_tool*, y su funcionamiento se basa en la construcción de una *pipeline de ingestión* (*Ingestion Pipeline*, en el marco de LlamaIndex) que aplica de forma secuencial diferentes tipos de transformaciones.

Estas transformaciones pueden dividirse en dos grupos: obligatorias y opcionales. El único paso estrictamente obligatorio en esta etapa es la generación de *embeddings* (representación vectorial del contenido textual). Si el usuario no define un modelo de embeddings explícito en la configuración, el sistema utiliza por defecto el modelo *text-embedding-3-large* de OpenAI. Este modelo ha demostrado una elevada capacidad para capturar el significado semántico de los textos en contextos empresariales, lo que lo convierte en una opción sólida tanto por su rendimiento como por su equilibrio entre precisión y coste computacional.

Los otros dos pasos (la segmentación de texto y la extracción de metadatos) son opcionales y su aplicación depende completamente de la configuración del workflow. Si se desea, pueden omitirse por completo, lo que proporciona un control total sobre la complejidad y profundidad del proceso de transformación.

Las transformaciones que pueden aplicarse son:

- **Segmentación de texto (Splitter):**

Esta transformación divide el contenido de cada nodo en fragmentos más pequeños denominados *chunks* (trozos), que se ajustan a criterios de longitud y coherencia semántica. La segmentación permite controlar el tamaño de los fragmentos indexados, lo que resulta fundamental para trabajar con modelos de lenguaje que tienen límites en la longitud de contexto. El sistema puede utilizar diferentes *splitters* (segmentadores), como “*SentenceSplitter*” (que divide por frases), o “*SemanticSplitter*” (que identifica puntos de ruptura naturales según la semántica del contenido). Si no se especifica ningún *splitter*, esta etapa se omite.

- **Extracción de metadatos (Metadata Extractors):**

Esta transformación permite enriquecer automáticamente cada nodo con metadatos adicionales, que pueden luego utilizarse como filtros o referencias en la recuperación. LlamaIndex proporciona una serie de extractores predefinidos [27], entre los que destacan:

- **SummaryExtractor:** genera un resumen corto del contenido del nodo.
- **QuestionsAnsweredExtractor:** intenta inferir qué preguntas podría responder el nodo.
- **KeywordExtractor:** identifica palabras clave representativas del contenido.
- **TitleExtractor:** detecta títulos o encabezados estructurales del nodo.
- **EntityExtractor:** extrae entidades nombradas como personas, organizaciones o ubicaciones.

Al igual que la segmentación, el uso de estos extractores es completamente opcional y depende de la configuración.

- **Generación de embeddings (Embedding Model):**

Este paso es esencial para permitir la búsqueda por similitud semántica en sistemas de recuperación vectorial. Cada nodo se vectoriza utilizando un modelo de *embeddings* (una representación numérica del significado del texto). El sistema permite utilizar modelos personalizados, como OpenAIEmbedding, HuggingFaceEmbedding, o variantes especializadas. Si no se proporciona un modelo, se emplea el modelo “text-embedding-3-large” de OpenAI de forma predeterminada, asegurando una mínima configuración sin sacrificar calidad [20].

Estas transformaciones se aplican de forma secuencial mediante una instancia de *IngestionPipeline*. Esta pipeline procesa cada grupo de nodos tanto los de sección como los de resumen y actualiza sus metadatos para incluir información estructural como *chunk_id* (id del trozo generado), *type* (tipo del nodo si es resumen o un chunk o también si es el nodo original), y atributos adicionales definidos por el usuario.

Un aspecto especialmente relevante de esta etapa es su capacidad para combinar nodos de diferente naturaleza: los *chunks* generados por el *splitter*, los nodos de sección originales, y los nodos resumen producidos en el paso de procesamiento avanzado. Esta combinación permite que el sistema cuente simultáneamente con versiones completas y resumidas del contenido, optimizando así las estrategias de recuperación y generación de respuestas.

Gracias a esta fase, el sistema transforma nodos textuales en elementos estructurados, enriquecidos y preparados para su persistencia. Al incorporar mecanismos de configuración externa, modularidad y herramientas reutilizables, esta etapa consolida el paso previo a la indexación como un componente esencial de la arquitectura general, garantizando trazabilidad, calidad semántica y rendimiento técnico.

4.2.6. Almacenamiento en base de datos (*add_nodes_to_db*)

Completadas todas las transformaciones necesarias, el flujo de trabajo de indexación culmina su secuencia operativa con el almacenamiento de los nodos generados en una base de datos persistente. Esta etapa no introduce ninguna transformación adicional sobre los datos; su único propósito es insertar los nodos, tal como han sido preparados en los pasos anteriores, en una estructura de almacenamiento que permita su posterior recuperación eficiente y trazable.

La ejecución de esta fase se desencadena mediante un evento *AddingNodesToDBEvent*, que contiene una lista de nodos ya transformados, embebidos y enriquecidos con metadatos. Para realizar la operación de escritura, el sistema recurre a una herramienta configurada bajo la clave *add_nodes_to_db_tool*. Esta herramienta encapsula la lógica de conexión, inserción y validación en la base de datos, y puede adaptarse a diferentes tecnologías de backend, como MongoDB, ChromaDB o almacenes vectoriales especializados como FAISS, Weaviate o Pinecone.

Es importante destacar que, en esta etapa, los nodos no son modificados de ninguna forma. Todo el trabajo de segmentación, extracción de metadatos y generación de *embeddings* ya ha sido completado en fases anteriores. La herramienta de almacenamiento simplemente toma la estructura final de cada nodo y la persiste en la base de datos correspondiente. Esto garantiza que no se introduzca ninguna alteración inesperada en el contenido justo antes de su indexación, manteniendo así la integridad del flujo y la trazabilidad del sistema.

Cada nodo es almacenado con su contenido textual, su representación vectorial (si aplica), y sus metadatos asociados, tales como identificadores únicos (*_id*), tipo de nodo (*chunk_text*, *summary*, etc.), fuente del documento, y atributos semánticos derivados. Esta información será utilizada posteriormente por el sistema de recuperación para filtrar, ordenar o ampliar el contexto de una consulta.

Una vez completada la operación, la herramienta devuelve una señal de finalización que se encapsula en un *PostIndexingProcessEvent*, con un mensaje de éxito que puede utilizarse para auditoría o notificación. Este evento marca el paso previo al cierre del flujo de trabajo y asegura que el contenido ha sido indexado correctamente y está listo para ser consultado por el sistema de recuperación.

Esta etapa, aunque sencilla en su lógica, es fundamental en la arquitectura del sistema: representa el punto de integración entre el flujo de procesamiento de conocimiento y el subsistema de acceso a la información. Su correcto funcionamiento es imprescindible para garantizar que los datos procesados se conviertan en conocimiento accesible, duradero y contextualizado.

4.2.7. Postprocesamiento final (*post_indexing_process*)

Una vez completado el almacenamiento de los nodos, el flujo de indexación puede, de forma opcional, ejecutar una última etapa denominada *postprocesamiento final*. Esta fase no forma parte del núcleo funcional del proceso de indexación, ya que no transforma datos ni afecta al contenido almacenado, pero cumple un papel importante desde el punto de vista operativo y de control del sistema.

El postprocesamiento se activa mediante un evento *PostIndexingProcessEvent*, que indica que todas las etapas anteriores han sido completadas satisfactoriamente o cuando en las primeras etapas no había ninguna referencia o nodo a indexar. En esta fase, el sistema puede ejecutar una o varias tareas auxiliares definidas por el usuario o por la lógica de negocio, tales como:

- Generar registros de auditoría o logs de actividad que documenten el resultado de la operación.
- Notificar a otros sistemas o usuarios que el proceso de indexación ha finalizado.
- Activar flujos externos, como validaciones automáticas o actualizaciones de dashboards.
- Retornar un resumen del resultado, por ejemplo: número de nodos indexados, tiempo total de ejecución, documentos procesados, etc.

Estas acciones se llevan a cabo mediante una herramienta opcional definida como *post_indexing_process_tool*, que puede programarse libremente para cubrir necesidades específicas del entorno donde se despliegue el sistema. Si no se proporciona ninguna herramienta en esta fase, el sistema emite simplemente un evento de tipo *StopEvent*, que marca el cierre formal del workflow sin tareas adicionales.

El diseño de esta etapa responde al principio de extensibilidad del sistema: permite integrar el flujo de indexación con otros módulos empresariales sin alterar su estructura principal, y al mismo tiempo proporciona una vía para generar métricas, monitorear la ejecución o implementar notificaciones que mejoren la trazabilidad operativa.

Aunque no es una fase esencial desde el punto de vista funcional, el postprocesamiento final aporta valor añadido al garantizar una correcta finalización del proceso, mejorando la transparencia y facilitando la integración del sistema en entornos más amplios de automatización empresarial.

4.3. Diseño extensible y desacoplado por herramientas

Una de las decisiones arquitectónicas más relevantes del workflow de indexación es el uso de herramientas desacopladas para la implementación de la lógica específica de cada etapa del flujo. En lugar de codificar internamente el comportamiento de cada paso, el sistema permite delegar dichas funciones en *tools* externas, configurables y fácilmente intercambiables. Este enfoque promueve una separación estricta entre la definición estructural del flujo, es decir, la secuencia de steps y eventos y la lógica de negocio que realmente se ejecuta en cada uno de ellos.

Cada etapa del workflow se apoya en una herramienta distinta que se puede definir dinámicamente en la configuración del sistema. Por ejemplo, la extracción de contenido puede realizarse con un parser local para archivos PDF o con una herramienta conectada a OneDrive para acceder a documentos en la nube. De igual forma, el filtrado de nodos puede usar un simple validador de vacíos o aplicar filtros semánticos más complejos para restringir los

resultados a un dominio específico. Lo mismo ocurre con herramientas como los generadores de resúmenes, extractores de preguntas, clasificadores temáticos o evaluadores de calidad.

Este diseño desacoplado permite que el núcleo del workflow permanezca completamente estable, mientras que las herramientas que definen su comportamiento puedan evolucionar o adaptarse a distintos proyectos. Así, un mismo flujo de indexación puede adaptarse a múltiples fuentes de datos, como documentos en OneDrive o entradas en una base de datos, simplemente configurando las herramientas adecuadas para cada caso. Esto permite definir qué información se desea indexar y cómo debe procesarse, sin necesidad de modificar la estructura del workflow. Esta capacidad no solo favorece la reutilización de componentes, sino que también facilita el mantenimiento, ya que cada herramienta puede desarrollarse, probarse o actualizarse de forma independiente.

Otro beneficio significativo es la testabilidad aislada: cada tool puede ser evaluada por separado antes de integrarse en el flujo global, lo que acelera el desarrollo y reduce el riesgo de errores sistémicos. Por último, el sistema admite una evolución futura sin necesidad de reescritura, basta con añadir nuevas herramientas a la biblioteca disponible y referenciarlas en la configuración, lo que convierte al workflow en una arquitectura altamente extensible y sostenible a largo plazo.

4.4. Funcionamiento del workflow de indexación: ejemplo y traza del proceso

Para comprender mejor el funcionamiento del workflow de indexación, resulta útil observar su ejecución a través de un ejemplo práctico. Supongamos que un usuario desea indexar un documento PDF que contiene información relevante para su empresa, como un informe técnico o una normativa interna.

El proceso comienza cuando el usuario lanza la ejecución del workflow y proporciona como entrada el documento a indexar. A partir de ese momento, el sistema activa una secuencia de etapas automáticas que transforman progresivamente ese contenido en una estructura de conocimiento lista para ser utilizada en tareas de recuperación posteriores.

Primero, el sistema identifica y estandariza la fuente de entrada. A continuación, accede al contenido del documento, lo analiza y lo divide en fragmentos informativos llamados nodos. Estos fragmentos son limpiados y validados, eliminando posibles duplicados o partes irrelevantes. Una vez filtrados, el sistema puede enriquecer los nodos aplicando operaciones como resúmenes, detección de relaciones o extracción de información clave.

Posteriormente, los nodos son transformados mediante técnicas como la segmentación en bloques más pequeños, la extracción de metadatos estructurados y la conversión de texto a vectores semánticos (*embeddings*), lo que permite su indexación eficiente. Finalmente, los nodos son almacenados en una base de datos especializada.

Durante toda la ejecución, el workflow mantiene una traza completa del recorrido de cada elemento. Es posible conocer qué contenido fue procesado, en qué momento, con qué herramientas, y qué resultado se generó en cada paso. Esta trazabilidad garantiza transparencia,



control y capacidad de auditoría, lo que resulta esencial en entornos donde la calidad y fiabilidad del conocimiento indexado son prioritarias.

Este enfoque permite que cualquier documento, correo o fuente de información empresarial pueda ser convertido de forma automatizada en conocimiento estructurado, enriquecido y preparado para responder preguntas de forma precisa y contextualizada en fases posteriores del sistema.

4.5. Conclusiones

El desarrollo de un workflow de indexación modular y robusto como el presentado en este trabajo ha supuesto diversos desafíos técnicos, especialmente derivados de la naturaleza asincrónica del flujo, la heterogeneidad de los datos de entrada y la necesidad de garantizar una ejecución fiable en entornos reales.

Uno de los desafíos más complejos fue la gestión de la asincronía. Cada paso del flujo opera de manera autónoma y responde únicamente a eventos específicos, lo que implica coordinar la ejecución mediante un modelo reactivo. Fue necesario implementar controles que aseguren la correcta propagación de los eventos, la integridad del contexto compartido (ctx) y una gestión eficaz de errores intermedios. Además, se establecieron límites de tiempo (timeouts) para evitar bloqueos en casos de operaciones prolongadas o fallidas.

Otro reto importante surgió con la validación de nodos y limpieza del contenido. Dado que los datos de entrada pueden provenir de documentos mal formateados, correos electrónicos ruidosos o fuentes inconsistentes, era habitual encontrar nodos vacíos, duplicados o irrelevantes. Para afrontarlo, se diseñaron herramientas específicas de filtrado semántico y validación que eliminan automáticamente cualquier nodo que no aporte valor o pueda entorpecer las etapas posteriores del flujo.

Un aspecto crítico adicional fue el de la integración con APIs externas, como los servicios de Microsoft OneDrive o Salesforce. Estas integraciones permiten acceder de forma directa a los documentos almacenados en entornos empresariales reales, pero también introducen complejidades técnicas asociadas a la autenticación, la paginación de datos, los cambios en las credenciales, la disponibilidad del servicio y la gestión de errores remotos. Fue necesario construir parsers especializados para cada tipo de conector y definir flujos de autenticación robustos, capaces de funcionar de forma automatizada sin comprometer la seguridad del sistema.

En términos de configuración, se presentaron desafíos ligados a errores de parametrización y validación dinámica de la configuración. La capacidad de personalizar completamente el flujo a través de `from_config` implica asumir que cualquier fallo en la definición de herramientas, embeddings o extractores puede comprometer la ejecución completa. Para mitigar este riesgo, se introdujeron mecanismos de validación anticipada y de logging detallado que detectan inconsistencias antes de ejecutar los pasos del flujo.

Desde el punto de vista del rendimiento, también fue necesario aplicar procesamiento por lotes para optimizar la carga computacional al trabajar con volúmenes grandes de documentos. Del mismo modo, se definieron límites sobre el tamaño de los nodos a indexar, se eliminaron

campos redundantes en los metadatos, y se optó por modelos ligeros de embeddings cuando se requería reducir el coste sin sacrificar calidad.

Todas estas decisiones de diseño, lejos de ser meros ajustes técnicos, han contribuido a consolidar una arquitectura robusta, escalable y adaptable a distintos escenarios, preparada para funcionar en entornos empresariales reales donde la fiabilidad, la eficiencia y la trazabilidad son factores críticos.

5. Diseño e implementación del workflow de Recuperación

El workflow de recuperación constituye la segunda pieza clave del sistema propuesto. Su función principal es permitir a los usuarios formular consultas en lenguaje natural y obtener respuestas fundamentadas en el contenido previamente indexado. A diferencia del flujo de indexación, que se encarga de preparar y estructurar la información, el flujo de recuperación se ocupa de recuperarla, evaluarla y sintetizar una respuesta coherente, precisa y trazable.

Este capítulo describe cómo ha sido diseñado e implementado este flujo, siguiendo una arquitectura modular basada en steps asincrónicos conectados mediante eventos. Se analizan sus componentes principales, las etapas que lo conforman y su integración con agentes inteligentes, retrievers y sintetizadores configurables. Al igual que en el capítulo anterior, también se abordan aspectos relacionados con la configuración dinámica y los retos técnicos asociados al proceso de recuperación.

5.1. Arquitectura del workflow de recuperación

5.1.1. Estructura de la clase RAGWorkflow

La clase RAGWorkflow representa la implementación específica del flujo de trabajo encargado de la recuperación de información y generación de respuestas dentro del sistema propuesto. Su diseño sigue el modelo de ejecución por pasos asincrónicos definido por LlamaIndex, donde cada etapa se implementa como un método decorado con `@step` y está enlazada mediante eventos tipados que transmiten tanto el contexto como los resultados intermedios entre los distintos componentes del flujo. Esta estructura proporciona una ejecución modular, flexible y perfectamente trazable, ideal para escenarios empresariales donde las consultas y la recuperación de conocimiento requieren procesos compuestos, adaptativos y auditables.

La clase hereda de la clase base Workflow de LlamaIndex y en su constructor define todos los componentes clave del flujo: los agentes responsables de la reformulación de consultas, los distintos recuperadores configurables (vectorial, recursivo, jerárquico), los sintetizadores de respuesta y consolidación, los agentes de evaluación y clarificación, así como herramientas adicionales para enriquecimiento y comparación de nodos, manejo de metadatos y gestión avanzada de respuestas. Esta arquitectura desacoplada facilita la sustitución, extensión o personalización de cada etapa del flujo sin impactar la lógica global, garantizando la máxima flexibilidad para ajustarse a requisitos cambiantes o dominios específicos.

El flujo está compuesto por una secuencia de etapas, entre las que destacan:

- **rewrite_query**: recibe la consulta original del usuario y la reformula en subconsultas o variantes más precisas, permitiendo así resolver ambigüedades o enriquecer la búsqueda.

- **query_router_step**: actúa como un paso intermedio que decide a qué recuperadores se envían las consultas generadas, en función de los tipos de recuperación configurados, permitiendo así adaptar la recuperación a la naturaleza de la consulta.
- **recursive_retrieval**, **vector_retrieval**, **hierarchical_retriever**: ejecutan la recuperación de nodos relevantes a través de los distintos recuperadores configurados, ya sea mediante búsqueda vectorial, jerárquica o recursiva.
- **cohere_rerank**: aplica técnicas de re-ranking de resultados (nodos recuperados) para priorizar los fragmentos más relevantes y unificar la información recuperada.
- **summary_and_full_section_nodes**, **internal_related_nodes**: enriquecen el conjunto de nodos mediante la incorporación de resúmenes, secciones completas o nodos relacionados.
- **normal_synthesizer** y **response_synthesizer_agent**: sintetizan una o varias respuestas a partir de los nodos recuperados y la pregunta del usuario, utilizando para ello tanto algoritmos clásicos como agentes avanzados de generación.
- **faithfulness_evaluation**: evalúa la fidelidad y alineación de las respuestas generadas respecto a la información recuperada, filtrando aquellas que no cumplen los criterios establecidos.
- **consolidate_synthesizing**: fusiona todas las respuestas que han superado la evaluación de fidelidad en una única respuesta final, proporcionando una contestación completa, coherente y justificada para el usuario.
- **response_critic**: realiza la evaluación final de la respuesta consolidada. Si la respuesta es adecuada y suficiente, el flujo termina; en caso contrario, permite activar ramas de clarificación o reintentar la recuperación y síntesis mediante la reformulación de la consulta.

Cada una de estas etapas está conectada mediante eventos específicos, definidos en la clase, como *ExpandedQueryEvent*, *QueryEvent*, *RetrievalEvent*, *RetrieverEvent*, *AddingSummaryAndFullSectionNodes*, *InitialResponseEvent*, *FaithfulResponseEvent*, *ConsolidatedResponseEvent*, y finalmente *StopEvent*. Esta gestión explícita de los eventos garantiza que el flujo sea totalmente trazable y que cada avance esté justificado por el resultado del paso anterior, permitiendo además gestionar ramas alternativas en función de la necesidad de clarificación o reintento.

La clase incluye también el método de clase *from_config*, que permite inicializar el workflow a partir de un diccionario externo de configuración, cargando dinámicamente los agentes y herramientas necesarios mediante funciones auxiliares. De este modo, la arquitectura admite una parametrización completa sin modificar el código fuente, facilitando la adaptación a distintos contextos de consulta o bases de conocimiento (por ejemplo, consultas legales, documentación técnica o FAQs empresariales), manteniendo el núcleo de ejecución estable y escalable.

5.1.2. Diagrama del flujo completo de ejecución

El workflow de recuperación está compuesto por una secuencia estructurada de pasos que se ejecutan de forma asincrónica, en respuesta a eventos específicos. Esta organización modular permite transformar cualquier consulta de entrada en una respuesta final, justificada y alineada con la información previamente indexada. Para visualizar el comportamiento global del sistema, a continuación se presenta un diagrama de flujo que ilustra todas las etapas del proceso y los eventos que las conectan.

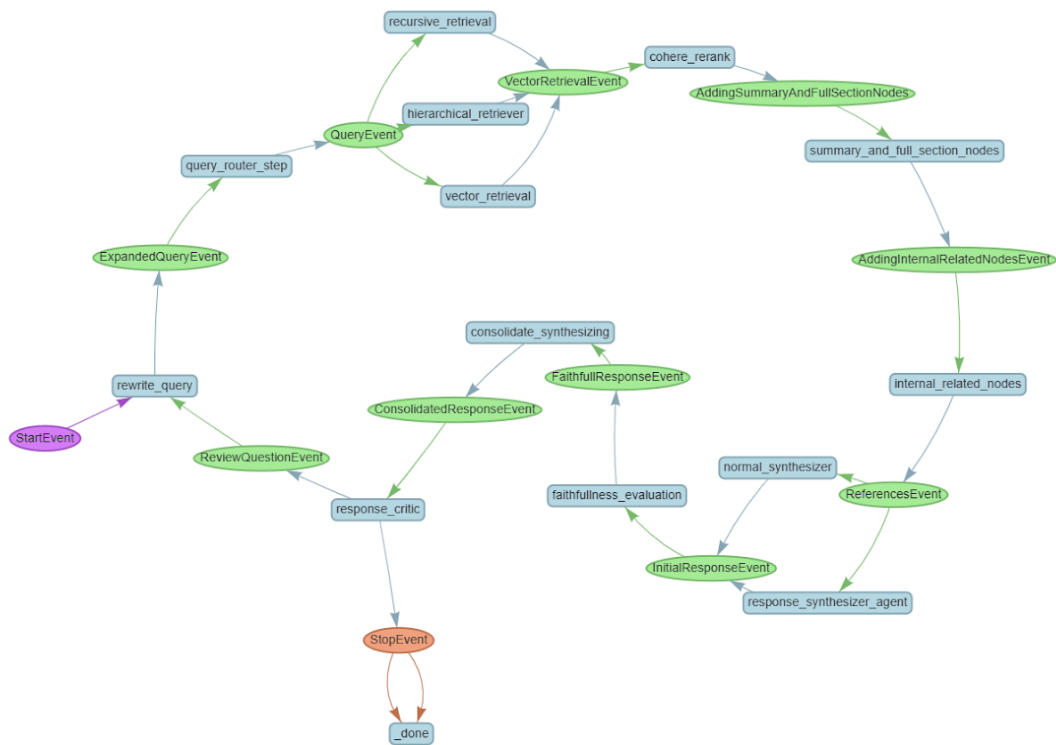


Figura 8. Arquitectura general del workflow de recuperación (RAG).

Para comprender el comportamiento global del sistema, la **Figura 8** presenta un diagrama visual que ilustra todas las etapas involucradas en el proceso de recuperación, así como los eventos que gobiernan el avance entre ellas. Cada bloque azul representa una etapa del flujo de trabajo, mientras que cada óvalo verde indica un tipo de evento que transporta los datos y el contexto desde una etapa hasta la siguiente. El flujo comienza con la recepción de una consulta, que da lugar a la generación y reformulación de posibles variantes para maximizar la relevancia de los resultados.

A continuación, el sistema distribuye estas consultas entre los distintos mecanismos de recuperación configurados, de modo que se aprovechen diferentes estrategias para localizar la información más pertinente en función del contexto. Los fragmentos de información recuperados son, posteriormente, unificados y priorizados, aplicando técnicas de combinación, ordenación y enriquecimiento para garantizar que el resultado sea lo más completo y ajustado posible a la consulta inicial.

El proceso sigue con la generación de una o varias respuestas provisionales, que son evaluadas en cuanto a su fidelidad y coherencia respecto a la información recuperada. Aquellas respuestas que cumplen con los criterios establecidos se consolidan en una contestación final,

suficientemente argumentada y alineada con la necesidad planteada. Finalmente, el flujo concluye con una evaluación crítica, que determina si la respuesta obtenida es adecuada o si es necesario reiniciar el proceso, permitiendo así ciclos de mejora y clarificación cuando la información inicial resulta ambigua o insuficiente.

Esta representación gráfica resulta especialmente útil para comprender la lógica y la secuencia general del workflow antes de analizar en profundidad cada una de las etapas que lo componen. En los apartados siguientes del capítulo, se describirá de manera individual el funcionamiento, las herramientas asociadas y las configuraciones relevantes de cada paso del proceso.

5.1.3. Encaje dentro del sistema global

El workflow de recuperación constituye el componente central a través del cual se accede y se pone en valor todo el conocimiento previamente incorporado al sistema. Su papel principal es transformar las consultas realizadas por los usuarios en respuestas precisas y fundamentadas, basándose siempre en los datos estructurados y validados que han sido generados y almacenados durante el proceso de indexación.

La interdependencia entre el workflow de recuperación y el de indexación resulta esencial para la arquitectura del sistema. Mientras que la indexación se encarga de analizar, transformar y almacenar la información en forma de nodos enriquecidos con metadatos, el workflow de recuperación actúa sobre ese repositorio estructurado para localizar, combinar y sintetizar la información relevante en cada respuesta. De este modo, el proceso de recuperación no solo garantiza que las respuestas estén alineadas con la base de conocimiento real y validada, sino que también permite mantener la trazabilidad y la coherencia del sistema en todo momento.

Este diseño modular y secuencial donde la incorporación y la recuperación del conocimiento se gestionan mediante workflows diferenciados pero coordinados permite mantener el sistema actualizado y fácilmente extensible. Además, favorece la adaptación a nuevos tipos de consultas, la integración de agentes inteligentes adicionales, y la evolución independiente de las técnicas de indexación o recuperación, asegurando la robustez y la calidad del servicio global de gestión del conocimiento.

5.2. Descripción técnica y funcional de cada etapa

5.2.1. Reformulación y distribución de la consulta (*rewrite_query*)

El primer paso del workflow de recuperación consiste en interpretar la consulta planteada por el usuario. En caso de ser necesario, esta consulta se reformula para maximizar la relevancia y la precisión de los resultados. El objetivo es transformar una pregunta original, que puede resultar ambigua, incompleta o demasiado general, en una o varias variantes más precisas. Estas nuevas formulaciones se adaptan tanto a la lógica interna del sistema como a las capacidades de los mecanismos de recuperación.

El proceso se inicia tras la recepción de un evento inicial *StartEvent* que contiene la consulta del usuario. El sistema puede emplear un agente de reformulación de consultas (*query_rewriter*) configurado previamente. Este agente analiza la consulta y genera distintas alternativas que

resuelven posibles ambigüedades, clarifican conceptos o adaptan el lenguaje utilizado por el usuario al de la base de conocimiento.

La reformulación de la consulta depende de la configuración establecida en el workflow. Si no se ha definido el agente encargado en reformular las consultas, es decir que no se ha añadido a la configuración del workflow, el sistema utiliza la consulta original como entrada para las etapas siguientes. En caso contrario, el agente actúa como una interfaz que media entre el usuario y el sistema de recuperación, permitiendo una mayor precisión y adaptabilidad ante diferentes tipos de consulta.

Tanto la pregunta original como las variantes generadas en esta etapa se emplean en los pasos posteriores de recuperación y síntesis de respuestas. Todas estas queries se encapsulan en un evento *ExpandedQueryEvent*, que será consumido por la siguiente fase del workflow. De este modo, la reformulación y distribución de la consulta contribuye a que la recuperación de información y la generación de respuestas se adapten a las características de cada pregunta, garantizando así resultados más relevantes y ajustados al contexto.

5.2.2. Selección de mecanismos de recuperación (*query_router_step*)

Una vez completada la reformulación de la consulta, el workflow activa una etapa de intermediación cuya función principal es transferir tanto la pregunta original como las variantes generadas a los recuperadores definidos en la configuración del sistema. Esta fase no introduce ninguna lógica de enrutamiento inteligente adicional, sino que se limita a distribuir las queries a los mecanismos de recuperación especificados en el parámetro *retrievers* del workflow.

El proceso se inicia tras la recepción de un evento *ExpandedQueryEvent* que contiene la lista de queries generadas en la etapa anterior. El sistema utiliza la configuración establecida para determinar qué recuperadores se emplearán, sin realizar ningún filtrado dinámico ni selección avanzada en este punto. Todas las queries se envían de forma directa a los recuperadores habilitados, asegurando así que se exploran las diferentes estrategias de búsqueda que el sistema tenga configuradas.

Un aspecto relevante de esta etapa es la gestión de dos contadores internos, *cohere_counter* y *responses_counter*, que se almacenan en el contexto del flujo para coordinar el avance de las siguientes fases. El contador *cohere_counter* indica a la etapa de reranking cuántos eventos debe esperar antes de proceder a la combinación y priorización de los resultados recuperados; así se asegura que la lógica de combinación solo se ejecuta una vez que todos los recuperadores seleccionados han finalizado su tarea. Por otro lado, el contador *responses_counter* señala cuántos eventos de respuesta deben recibirse durante la etapa de síntesis, lo cual resulta esencial cuando la configuración del workflow incluye varios sintetizadores ejecutándose en paralelo. Gracias a este mecanismo, el sistema puede gestionar de forma precisa la concurrencia tanto en la recuperación como en la síntesis de respuestas, asegurando la correcta sincronización y el procesamiento coordinado de todas las salidas generadas por los componentes configurados.

Finalmente, el sistema encapsula la información sobre las queries y los recuperadores seleccionados en un evento *QueryEvent*, que será consumido por las etapas de recuperación del workflow. Así, esta fase asegura que la consulta del usuario, junto con todas sus variantes relevantes, se distribuye correctamente a los mecanismos de recuperación establecidos,

manteniendo la coherencia y la sincronización necesarias para el funcionamiento del flujo completo.

5.2.3. Recuperación de información (*recursive_retrieval*, *vector_retrieval*, *hierarchical_retriever*)

La etapa de recuperación de información constituye un bloque fundamental del workflow, en el que se ejecutan uno o varios mecanismos de búsqueda configurados previamente en el parámetro *retrievers* de la configuración del workflow. El diseño del sistema permite que, dependiendo de la configuración seleccionada, se activen diferentes recuperadores, ya sea de manera individual o combinada. Cuando el usuario o el administrador del sistema ha definido varios recuperadores, estos se ejecutan de forma paralela. Además, si existen varias consultas generadas en las etapas anteriores, cada recuperador las procesa también en paralelo, optimizando así el tiempo de respuesta y la eficiencia global del proceso.

Esta paralelización no introduce ningún conflicto ni problema de sincronización, ya que la arquitectura del workflow está preparada para gestionar la ejecución simultánea de tareas independientes. Cada recuperador opera sobre su propio conjunto de datos y queries, y la recogida de resultados se realiza de manera controlada, garantizando la integridad y la coherencia de la información recuperada.

A continuación, se describen los principales tipos de recuperadores empleados en el sistema:

- **Recuperador recursivo (*recursive_retrieval*):**

El recuperador recursivo está especialmente diseñado para abordar escenarios donde la información relevante está dispersa en varios nodos o donde existen relaciones contextuales profundas, como sucede en documentos complejos, manuales técnicos o tablas de gran extensión. Su funcionamiento se basa en un proceso iterativo: comienza recuperando nodos candidatos mediante una primera búsqueda, y a partir de esos resultados, lanza nuevas subconsultas o refina la recuperación de acuerdo con los fragmentos encontrados. Este mecanismo permite explorar y vincular información relacionada en diferentes niveles o secciones, navegando de forma controlada y estratégica por grandes volúmenes de datos. Así, el sistema es capaz de captar conexiones transversales entre fragmentos, proporcionando respuestas más completas en casos donde la información no está contenida en un solo nodo.[22]

Cabe destacar que el recuperador recursivo admite la aplicación de filtros de metadatos, lo que permite restringir la recuperación a fragmentos que cumplan con determinados atributos o características, como el tipo de documento, la sección o cualquier otro campo indexado. Esta capacidad de filtrado resulta especialmente útil para focalizar la búsqueda y obtener respuestas más precisas y contextualizadas.

- **Recuperador vectorial (*vector_retrieval*):**

El recuperador vectorial utiliza la búsqueda por similitud semántica para localizar los fragmentos de texto más cercanos al significado de la consulta. Para ello, transforma tanto la consulta como los nodos indexados en representaciones vectoriales (*embeddings*) y calcula la distancia entre ellos en un espacio semántico. Este enfoque es

especialmente eficaz para encontrar información relevante aunque no exista coincidencia literal de palabras, ya que se basa en el sentido conceptual y la proximidad en el significado. El vectorial retrieval es la base de la mayoría de sistemas modernos de recuperación aumentada, gracias a su robustez y versatilidad ante consultas variadas.

Además, el recuperador vectorial permite la utilización de filtros de metadatos, de modo que la búsqueda puede limitarse a fragmentos que cumplan ciertos criterios definidos por el usuario o el contexto de la consulta. Esto incrementa la precisión de los resultados y posibilita adaptar dinámicamente la recuperación a diferentes necesidades o escenarios empresariales.

- **Recuperador jerárquico (*hierarchical_retriever*):**

El recuperador jerárquico está diseñado para trabajar sobre colecciones estructuradas y jerarquizadas de documentos, como manuales, conjuntos de archivos relacionados o repositorios documentales complejos. Su principal característica es la capacidad de seleccionar dinámicamente el índice o el subíndice más relevante para cada consulta, basándose en la estructura global del corpus y los metadatos disponibles. El proceso comienza evaluando a nivel global qué parte del conjunto de datos es más prometedora, y posteriormente se profundiza en el subconjunto seleccionado para realizar la recuperación específica. Esta aproximación resulta especialmente útil cuando se dispone de documentos agrupados por categorías, temas o niveles de granularidad, permitiendo optimizar la precisión y reducir el ruido en las respuestas. [23]

Independientemente del tipo de recuperador utilizado, el resultado de esta etapa consiste en una lista de nodos recuperados que representan los fragmentos del contenido más relevantes para la consulta. Todos los recuperadores, ya sean recursivos, vectoriales o jerárquicos, devuelven sus resultados utilizando un formato común, encapsulado en un evento específico *RetrievalEvent*. Este evento contiene la lista de nodos identificados como relevantes junto con la información adicional necesaria para su posterior procesamiento.

5.2.4. Reranking de resultados (*cohere_rerank*)

En esta etapa, el workflow se encarga de priorizar y ordenar los nodos recuperados por los distintos mecanismos de búsqueda, mejorando así la calidad final de los fragmentos que serán utilizados para la síntesis de la respuesta. Un aspecto clave de esta fase es la coordinación basada en el contador interno (*cohere_counter*), configurado en la etapa anterior (*query_router_step*). Gracias a este contador, la etapa de reranking espera a recibir el evento *RetrievalEvent* tantas veces como recuperadores hayan sido activados, garantizando así que se procesen conjuntamente los resultados provenientes de todos los mecanismos de recuperación seleccionados y que no se produzcan inconsistencias por la paralelización.

Previo al proceso de reranking, el sistema filtra los nodos recuperados para eliminar duplicados y asegurar que únicamente se procesan nodos únicos. Este filtrado es fundamental para evitar redundancias y mejorar la eficiencia y la calidad del ranking posterior.

Si en la configuración del workflow se ha definido el sistema de reranking, la etapa se activa para ordenar y seleccionar los nodos en función de su relevancia respecto a la consulta del usuario. Para ello, es necesario también haber configurado un agente de fusión de queries

(*query_fusioner*), especialmente en escenarios donde existen varias queries generadas a partir de la pregunta inicial. Este agente permite combinar todas las variantes de la consulta en una única query consolidada, que se utiliza posteriormente como referencia para calcular el ranking de los nodos recuperados.

El Cohere Reranker es un modelo especializado de reranking basado en técnicas avanzadas de aprendizaje profundo. Su función principal es recibir una lista de pares (query, nodo recuperado) y asignar una puntuación de relevancia a cada nodo en función de su relación semántica con la consulta. A diferencia de las métricas clásicas de similitud vectorial, el Cohere Reranker es capaz de analizar contextos complejos y matices lingüísticos, priorizando los fragmentos que realmente aportan valor informativo para responder a la consulta. El modelo no solo reordena los nodos según su relevancia, sino que también selecciona los mejores, es decir, aquellos que, según el propio modelo, presentan mayor adecuación y calidad para la consulta planteada. Este enfoque ha demostrado una alta eficacia en tareas de recuperación aumentada y es ampliamente utilizado en sistemas empresariales que requieren precisión y sensibilidad contextual en la ordenación de resultados [24].

En caso de que no se haya definido el sistema de reranking en la configuración del workflow de recuperación, esta etapa se omite y el flujo avanza directamente a la siguiente fase, procesando los nodos recuperados en el orden original.

De este modo, la etapa de reranking y fusión garantiza que los fragmentos utilizados para la síntesis de la respuesta sean no solo relevantes y únicos, sino también los más adecuados según el modelo, mejorando la calidad global del sistema de recuperación.

5.2.5. Enriquecimiento mediante resúmenes y secciones completas (*summary_and_full_section_nodes*)

En esta etapa, el workflow de recuperación se centra en enriquecer el conjunto de nodos recuperados añadiendo, de forma inteligente, fragmentos que amplían y contextualizan la información inicial. El objetivo principal es asegurar que, para cada nodo seleccionado, el sistema disponga tanto de la versión completa de la sección original como de su resumen asociado, optimizando así la cobertura y el contexto de la respuesta final. Cabe destacar que esta etapa no es obligatoria: si en la configuración del workflow no se ha habilitado la opción correspondiente (*add_summary_and_full_section_nodes*), el sistema omite este enriquecimiento y continúa el flujo con los nodos tal y como fueron producidos por la etapa anterior.

El proceso se activa tras la recepción del evento *AddingSummaryAndFullSectionNodes*, que contiene la lista inicial de nodos candidatos. Para cada nodo, el sistema examina su tipo a través de los metadatos asociados (*metadata.type*). Si se trata de un nodo correspondiente a un resumen de sección, se busca automáticamente la sección completa asociada a ese resumen dentro del vector store. Del mismo modo, si el nodo recuperado es una sección completa, se localiza el resumen correspondiente. En el caso de los nodos de tipo fragmento o *chunk*, el sistema trata de añadir tanto el resumen de la sección como la sección completa a la que pertenecen, siempre que existan en la base de datos (indexados previamente).

Esta lógica permite que, a partir de un nodo recuperado, se incorporen automáticamente nodos complementarios que aportan tanto una visión sintética como detallada de la misma unidad

documental. Los nodos adicionales identificados se agregan a la lista, garantizando que no haya duplicados y que toda la información relevante esté disponible para las etapas posteriores.

Este proceso resulta especialmente valioso en contextos donde los documentos indexados presentan una estructura jerárquica o contienen grandes volúmenes de información, como ocurre en artículos científicos, informes técnicos o archivos PDF extensos. Al combinar nodos resumen y nodos de sección completa, el sistema puede ofrecer respuestas que van desde visiones generales hasta explicaciones detalladas, adaptándose a la naturaleza de la consulta y a las necesidades del usuario.

Finalmente, el resultado de esta etapa es una lista ampliada y enriquecida de nodos, que se encapsula en un evento *AddingInternalRelatedNodesEvent* para ser procesado en las siguientes fases del workflow.

5.2.6. Añadido de nodos relacionados internos (*internal_related_nodes*)

En esta etapa, el workflow de recuperación tiene como objetivo ampliar el contexto de la información recuperada mediante la incorporación de nodos que presentan relaciones internas con los ya seleccionados. Este proceso resulta especialmente útil en documentos donde existen conexiones semánticas o estructurales entre diferentes fragmentos, como capítulos relacionados, secciones complementarias o apartados que abordan distintas perspectivas de un mismo tema.

La lógica de esta etapa se activa a partir del evento *AddingInternalRelatedNodesEvent*, que contiene la lista de nodos enriquecida por la fase anterior. Para cada nodo, el sistema examina la presencia de metadatos específicos, generalmente almacenados bajo la clave *same_document_relations*. Este campo recoge identificadores de otras secciones o fragmentos que, aunque no hayan sido recuperados en las búsquedas iniciales, guardan una relación directa con el nodo original por tratarse del mismo documento o compartir contexto temático.

Cuando se detectan estas relaciones internas, el sistema localiza los nodos correspondientes en la base de datos, asegurando que se recuperan únicamente los fragmentos relevantes y que no se introducen duplicados en la lista final. De este modo, la respuesta generada puede incorporar perspectivas adicionales, ejemplos complementarios o desarrollos que profundizan en la materia, aportando así un mayor grado de coherencia y riqueza informativa.

El resultado de esta etapa es una lista ampliada de nodos que no sólo incluye los fragmentos seleccionados en pasos anteriores, sino también aquellos que, por su relación semántica o estructural, contribuyen a mejorar la contextualización de la respuesta. Esta lista se encapsula en un evento *ReferencesEvent*, que será consumido por la siguiente fase del workflow, correspondiente a la síntesis de respuestas (*synthesizers*).

Cabe destacar que, al igual que en etapas previas, este proceso de añadido de nodos relacionados es opcional y depende de la configuración establecida en los parámetros del workflow (*add_internal_related_nodes*). Si no se ha activado esta opción en los ajustes, el sistema omite la incorporación de relaciones internas y continúa con la lista de nodos obtenida en la fase anterior.

5.2.7. Síntesis de respuestas (*normal_synthesizer*, *response_synthesizer_agent*)

Tras la fase de recuperación y enriquecimiento de nodos, el workflow aborda la etapa de síntesis, donde los fragmentos seleccionados se convierten en respuestas estructuradas y coherentes. Esta etapa se activa mediante la recepción del evento *ReferencesEvent*, que incluye la lista final de nodos relevantes y las queries que deben ser respondidas. Al igual que en la fase de recuperación, el sistema ejecuta la síntesis en paralelo para cada query disponible. Esto significa que si la lista de queries contiene tanto la consulta original como variantes generadas en etapas previas, se sintetizará una respuesta independiente para cada una de ellas. Esta ejecución paralela permite aprovechar al máximo la eficiencia del sistema y facilita la comparación entre alternativas de respuesta.

La lógica de síntesis está gestionada por uno o varios sintetizadores (*synthesizers*), seleccionados previamente en la configuración del workflow. El sistema delega el proceso de generación de respuestas a estos componentes, que pueden ser de diferentes tipos en función de la configuración elegida.

A continuación, se describen los principales tipos de sintetizadores empleados en el sistema:

- **Normal Synthesizer** (*normal_synthesizer*):
Este sintetizador es el componente estándar de LlamaIndex para la generación de respuestas a partir de nodos recuperados. Su funcionamiento se basa en aplicar técnicas de resumen extractivo, generación automática de texto y formulación coherente de respuestas, integrando toda la información relevante contenida en los nodos seleccionados. El *normal_synthesizer* puede operar bajo diferentes modos de respuesta (compacta, resumida, desarrollada, etc.) y es capaz de adaptar su estrategia de síntesis según el contexto y la configuración del sistema. Está especialmente optimizado para producir respuestas precisas y estructuradas en función del contenido recuperado.[25]
- **Response Synthesizer Agent** (*response_synthesizer_agent*):
Este agente actúa como un sintetizador inteligente, capaz de analizar la consulta hecha por el usuario y los nodos recuperados, y de generar respuestas fundamentadas que integran tanto información textual como razonamiento contextual. A diferencia del sintetizador estándar, el agente puede incorporar lógica adicional de interpretación y adaptación, recurriendo a técnicas de procesamiento de lenguaje natural avanzadas o modelos de inteligencia artificial específicos para la tarea. Esta opción es especialmente útil en contextos empresariales o en sistemas donde la calidad argumentativa y la personalización de la respuesta son prioritarias.

Independientemente del sintetizador empleado, la etapa genera una respuesta por cada query procesada, de modo que si la lista de queries contiene tanto la pregunta original como variantes creadas en fases anteriores, el resultado final será un conjunto de respuestas paralelas. Todas estas respuestas se encapsulan en un evento *InitialResponseEvent*, que será utilizado en las etapas siguientes para su evaluación y consolidación.

5.2.8. Evaluación de fidelidad de las respuestas (*faithfulness_evaluation*)

La evaluación de fidelidad es una técnica fundamental en sistemas de recuperación aumentada, cuyo propósito es garantizar que las respuestas generadas por el sistema se apoyan exclusivamente en la información contenida en los documentos recuperados. Su objetivo principal es detectar y filtrar cualquier afirmación no sustentada, es decir, cualquier información que no pueda justificarse o comprobarse a partir de los documentos recuperados, asegurando así que la respuesta final sea factual, verificable y rastreable respecto a las fuentes originales. En la práctica, la evaluación de fidelidad compara la respuesta generada con la evidencia textual extraída de los nodos, evaluando si existe una correspondencia semántica suficiente entre ambos elementos. Si la respuesta contiene información no presente o no justificada en los documentos, se descarta automáticamente.

Esta metodología ha cobrado gran relevancia en aplicaciones empresariales y científicas, donde la fiabilidad y la trazabilidad de las respuestas resultan críticas. LlamaIndex ofrece un módulo específico para la evaluación de fidelidad que automatiza este proceso, empleando modelos de lenguaje y técnicas de comparación semántica para validar la alineación factual de cada respuesta. [26]

En el contexto del workflow, la etapa de evaluación de fidelidad se activa tras la síntesis de respuestas. Su ejecución está coordinada mediante el contador interno (*responses_counter*), que determina cuántos eventos de respuesta *InitialResponseEvent* deben recibirse antes de iniciar la evaluación. Si no se ha definido un agente de evaluación de fidelidad (*faithfulness_evaluator*) en la configuración del sistema, esta etapa se omite, y el flujo continúa con todas las respuestas sintetizadas. Por el contrario, cuando el evaluador está presente, cada respuesta se analiza en relación con la query y los fragmentos de evidencia recuperados. Solo aquellas respuestas que superan el control de fidelidad, es decir, que se apoyan estrictamente en la información de los nodos son retenidas y encapsuladas en el evento *FaithfulResponseEvent* para las fases siguientes del flujo.

Esta validación final permite ofrecer respuestas confiables, filtrando cualquier posible invención o desviación, y aportando garantías adicionales en entornos donde la exactitud y la justificación documental son imprescindibles. Como resultado de esta etapa, el sistema produce una nueva lista de respuestas depuradas, que cumplen con el criterio de fidelidad y que serán utilizadas en las siguientes fases de consolidación y presentación al usuario.

5.2.9. Consolidación de respuestas (*consolidate_synthesizing*)

En esta etapa, el workflow aborda la consolidación final de las respuestas generadas y validadas. El objetivo es seleccionar, unificar o perfeccionar la respuesta que será finalmente presentada al usuario. Esta fase suele gestionarse mediante un agente especializado (*consolidate_synthesizer*), cuya función es analizar el conjunto de respuestas fieles producidas en las etapas anteriores y elaborar, a partir de ellas, una única respuesta final, más completa, coherente y estructurada.

El proceso de consolidación es opcional: su ejecución depende de que se haya definido el agente correspondiente en la configuración del workflow. Si el agente está presente, este analiza todas las respuestas generadas para las distintas variantes de la consulta, fusiona la información relevante y sintetiza una respuesta final optimizada, asegurando que se integren adecuadamente todos los matices y detalles pertinentes. Este enfoque resulta especialmente útil en escenarios

donde una consulta compleja puede haber dado lugar a respuestas parciales o complementarias, y es necesario presentarlas de forma unificada y comprensible.

En el caso de que no se haya configurado un agente de consolidación, el sistema selecciona directamente la respuesta generada para la query original del usuario, saltando la fase de fusión. De este modo, el workflow mantiene la simplicidad y eficiencia cuando no es necesaria una consolidación avanzada.

El resultado de esta etapa es una respuesta única y definitiva, encapsulada en un evento *ConsolidatedResponseEvent*, que será empleada en las fases finales de revisión o directamente presentada al usuario, dependiendo de la configuración del sistema.

5.2.10. Crítica y revisión final de la respuesta (*response_critic*)

La última etapa del workflow de recuperación está orientada a la validación crítica y la revisión final de la respuesta generada antes de presentarla al usuario. Esta fase puede ser gestionada por un agente específico (*judge_agent*), encargado de analizar si la respuesta final elaborada cumple con los criterios de calidad, adecuación y suficiencia esperados en el contexto de uso.

El proceso se inicia tras la consolidación de la respuesta, recibiendo como entrada el evento *ConsolidatedResponseEvent* que contiene la respuesta unificada junto con el resto de la información relevante sobre la consulta. En primer lugar, el sistema recopila el conjunto de nodos utilizados, las queries generadas y la consulta original del usuario, preparando toda esta información para la evaluación final.

Si en la configuración del workflow no se ha definido ningún agente de crítica, el sistema considera la respuesta final como válida y concluye el flujo, devolviendo el resultado al usuario mediante un evento de cierre *StopEvent*. De este modo, la revisión crítica se omite, y la respuesta se presenta tal cual fue consolidada en la etapa previa.

Si, por el contrario, el agente de crítica está presente, este analiza la respuesta final junto con la consulta original y puede aplicar diferentes criterios de evaluación: adecuación a la pregunta, coherencia argumental, completitud o incluso alineación con políticas internas del sistema. El agente recibe la información mediante un prompt estructurado y, tras procesarla, devuelve una evaluación que puede ser positiva ("good") o negativa.

- Si la evaluación es positiva, el flujo se da por terminado y se presenta la respuesta al usuario.
- Si la evaluación es negativa y aún no se ha superado el número máximo de intentos configurado (*max_attempts*), el agente puede proponer una nueva reformulación de la consulta, que será reenviada al flujo como un nuevo ciclo de revisión *ReviewQuestionEvent*. Así, el sistema puede intentar mejorar la respuesta solicitando información adicional o reformulando la pregunta, en un proceso iterativo controlado.
- Si se alcanza el número máximo de intentos sin lograr una evaluación positiva, el flujo concluye con la mejor respuesta obtenida hasta el momento, cerrando el proceso de recuperación.

Esta etapa culmina el proceso de recuperación aportando una última capa de revisión crítica y control de calidad sobre la respuesta generada. Gracias a la intervención del agente evaluador, el sistema puede detectar posibles insuficiencias, ambigüedades o desviaciones en la respuesta final, permitiendo iterar de manera controlada en busca de una contestación más precisa y alineada con la consulta del usuario. Además, el mecanismo de reformulación automática de la pregunta, en caso de evaluación negativa, dota al sistema de una capacidad de mejora adaptativa, incrementando la robustez y la utilidad del flujo de recuperación en entornos exigentes o ante consultas complejas.

De este modo, el workflow de recuperación no solo proporciona una respuesta fundamentada y contextualizada, sino que incorpora mecanismos activos de revisión y mejora continua, orientados a ofrecer la mayor garantía de calidad y adecuación posible. El resultado de esta etapa es la respuesta final, ya validada y depurada, que se presenta al usuario como cierre formal del proceso de recuperación.

5.3. Configuraciones evaluadas del workflow de recuperación

Con el objetivo de analizar el rendimiento y la calidad del workflow de recuperación en diferentes escenarios de uso, se han definido tres configuraciones principales del sistema. Cada una de ellas representa un nivel distinto de complejidad funcional, permitiendo evaluar el impacto de los distintos módulos y etapas sobre la precisión, eficiencia y robustez del proceso de recuperación.

- **Configuración básica:**

Pensada para situaciones donde se prioriza la eficiencia y la simplicidad, esta variante incluye solo los componentes esenciales para el funcionamiento del sistema. El flujo realiza una reescritura y expansión inicial de la consulta mediante el agente (*query_rewriter*), utiliza un recuperador semántico (*vector_retriever*) para localizar los fragmentos relevantes, sintetiza la respuesta con el agente de respuesta (*response_synthesizer_agent*), y finalmente la consolidación de las respuestas generadas en una (*consolidate_synthesizing*). Esta configuración está orientada a casos en los que la cobertura básica de las consultas y la rapidez de ejecución son prioritarias.

- **Configuración normal:**

Dirigida a un equilibrio entre rendimiento y calidad, esta variante introduce etapas adicionales para mejorar la precisión y el control de calidad de las respuestas. Junto a la reescritura de la consulta (*query_rewriter*), incorpora recuperación tanto recursiva como semántica (*recursive_retriever* y *vector_retriever*), un reranking de nodos con el modelo Cohere (*cohere_reranker*), síntesis mediante el agente de respuesta, evaluación de fidelidad y consolidación final. Además, añade una evaluación crítica de la respuesta final a través de un agente (*judge_agent*). Esta configuración busca optimizar tanto la calidad de las respuestas como la cobertura de escenarios más complejos.

- **Configuración completa:**

Pensada para contextos donde se requiere la máxima precisión y cobertura, la configuración completa activa todos los módulos avanzados del workflow. Incluye la reescritura y expansión de la consulta, recuperación recursiva y semántica y jerárquica (*hierarchical_retriever*), reranking con Cohere, incorporación de nodos relacionados

internos (*internal_related_nodes*) y resúmenes de secciones y secciones completas (*add_summary_and_full_section_nodes*), síntesis de respuestas, evaluación de fidelidad, consolidación y evaluación final de la respuesta. Además, establece un máximo de dos intentos de reformulación en caso de que la evaluación no sea satisfactoria, lo que permite una mejora adaptativa del proceso. Esta configuración está diseñada para entornos críticos donde la calidad, trazabilidad y justificación documental de la respuesta son aspectos fundamentales.

Estas tres configuraciones permiten evaluar el sistema de recuperación en distintos niveles de exigencia, proporcionando una visión clara de las ventajas y limitaciones de cada enfoque y facilitando la toma de decisiones para su despliegue en contextos empresariales reales.

5.4. Funcionamiento del workflow de recuperación: ejemplo y traza del proceso

Para comprender el funcionamiento del workflow de recuperación, se presenta a continuación un ejemplo basado en la configuración normal descrita en el apartado anterior. Esta configuración permite evaluar no solo la recuperación y síntesis de respuestas, sino también su fidelidad y calidad final mediante mecanismos adicionales.

Supongamos que un usuario desea realizar una consulta como: “¿Qué productos incluye el catálogo de marzo de 2025?”. Al iniciar la ejecución del workflow, el sistema activa una serie de etapas automatizadas que transforman esta consulta en una respuesta fundamentada y trazable.

El proceso comienza con la reformulación de la consulta. El sistema utiliza un agente especializado para generar tres variantes de la pregunta original, con el objetivo de mejorar la precisión de la recuperación. Estas variantes permiten explorar diferentes formas de formular la misma intención del usuario, resolviendo posibles ambigüedades o enriqueciendo el contexto.

Cada una de estas preguntas (incluyendo la original) es enviada a los mecanismos de recuperación configurados. En esta configuración, se utilizan tanto un recuperador vectorial como uno recursivo, que localizan fragmentos relevantes del conocimiento indexado basándose en similitud semántica y relaciones contextuales profundas.

Los fragmentos (*nodos*) recuperados por ambos mecanismos se combinan y se priorizan mediante un modelo de reranking (*Cohere*), seleccionando aquellos más adecuados para responder a las consultas generadas. Después, se pueden enriquecer añadiendo resúmenes y nodos relacionados, si estos están disponibles en la base de datos.

A continuación, el sistema genera una respuesta independiente para cada una de las preguntas reformuladas y para la pregunta original. Esto permite comparar diferentes formulaciones y obtener alternativas que puedan cubrir distintos matices de la intención del usuario.

Cada respuesta generada es sometida a una evaluación de fidelidad, que verifica si la información incluida está correctamente respaldada por los fragmentos recuperados. Solo aquellas respuestas que pasan esta evaluación continúan en el flujo.

Luego, todas las respuestas fieles se unifican mediante un proceso de consolidación, que permite generar una única respuesta final, más completa y coherente. Esta etapa está gestionada por un agente especializado que combina los elementos clave de todas las respuestas anteriores en una síntesis única.

Por último, el sistema activa un agente crítico (judge agent) que evalúa la calidad final de la respuesta consolidada. Este agente analiza si la respuesta es clara, completa y adecuada. Si lo considera necesario, puede activar una nueva reformulación de la pregunta, reiniciando el flujo en busca de una mejor respuesta. En caso contrario, el workflow se da por finalizado, y la respuesta se presenta al usuario.

Durante todo el proceso, el sistema mantiene una traza completa de los pasos realizados, las consultas generadas, los fragmentos utilizados y las decisiones tomadas. Esta trazabilidad garantiza transparencia, control y confianza en entornos donde la calidad de la información es prioritaria.

5.5. Conclusiones

El desarrollo de un workflow de recuperación modular y robusto como el presentado en este trabajo ha supuesto una serie de desafíos técnicos particulares, condicionados por la asincronía del flujo, la diversidad de mecanismos de búsqueda y síntesis, y la exigencia de mantener la trazabilidad y la calidad en todo el proceso.

Uno de los retos más significativos ha sido la gestión eficiente de la ejecución paralela y la sincronización de eventos. Al igual que en el workflow de indexación, cada etapa del flujo de recuperación opera de manera autónoma y responde únicamente a los eventos definidos para su activación. Sin embargo, en este caso, la existencia de múltiples recuperadores y sintetizadores potencialmente ejecutándose en paralelo ha requerido diseñar contadores internos y mecanismos de coordinación precisos. Estos controles aseguran que la lógica de combinación, reranking y evaluación solo se ejecuta cuando todos los componentes han finalizado, evitando así inconsistencias y problemas de concurrencia.

La validación de la calidad y la fidelidad de las respuestas ha supuesto otro desafío central. En la práctica, el sistema debe garantizar que las respuestas generadas no solo sean coherentes y pertinentes, sino también verificables y alineadas estrictamente con la evidencia recuperada. Para ello, se integraron agentes de evaluación automática y crítica que permiten filtrar respuestas poco fundamentadas y proponer iteraciones en busca de una mayor precisión. Este enfoque, aunque más complejo desde el punto de vista del diseño, ha resultado fundamental para dotar al sistema de fiabilidad en contextos empresariales exigentes.

Otro aspecto crítico ha sido la adaptación de la arquitectura a distintos tipos de consultas y dominios documentales. El workflow debía ser capaz de operar tanto con preguntas simples como con consultas compuestas, cubriendo escenarios en los que la información relevante pudiera estar distribuida entre diferentes fragmentos, secciones o incluso documentos independientes. La decisión de encapsular los resultados en eventos unificados y de utilizar herramientas externas y agentes inteligentes ha facilitado esta versatilidad, permitiendo ajustar el comportamiento del sistema mediante configuración, sin necesidad de modificar el núcleo del flujo.

Desde la perspectiva del rendimiento y la escalabilidad, fue necesario optimizar tanto la recuperación como la síntesis y evaluación, aplicando técnicas de ejecución paralela, de duplicación de nodos y procesamiento por lotes cuando el volumen de información así lo requería. Además, se establecieron límites y controles para evitar bucles infinitos en la etapa de revisión y para garantizar una experiencia de usuario ágil y predecible.

En suma, todas estas decisiones de diseño han contribuido a consolidar una arquitectura de recuperación avanzada, preparada para funcionar en entornos empresariales reales donde la calidad de la respuesta, la adaptabilidad y la trazabilidad son requisitos fundamentales. La solución resultante es robusta, extensible y capaz de evolucionar ante nuevas necesidades tecnológicas o cambios en el entorno operativo.

6. Evaluación y Resultados

El presente capítulo tiene como finalidad evaluar el comportamiento del sistema desarrollado en este proyecto, centrándose en la calidad de las respuestas generadas ante diferentes tipos de consultas. Aunque el sistema completo incluye un proceso previo de indexación, en esta fase se asume que dicha indexación ha sido correctamente realizada para cada uno de los casos de uso definidos, permitiendo así enfocar el análisis exclusivamente en la etapa de recuperación y generación de respuestas (workflow de recuperación). Esta aproximación resulta adecuada, dado que el rendimiento global del sistema está estrechamente vinculado a la capacidad del workflow para aprovechar la información previamente indexada.

Para ello, se han definido cinco casos de uso representativos de escenarios empresariales reales, que han sido evaluados utilizando tres configuraciones distintas del workflow de recuperación descritas en el capítulo anterior: configuración básica, configuración normal y configuración completa. Estas configuraciones permiten analizar cómo la incorporación progresiva de etapas y mecanismos de evaluación afecta tanto a la calidad de las respuestas como al tiempo de ejecución.

La evaluación de las respuestas generadas se ha llevado a cabo mediante un agente evaluador automático, diseñado para aplicar de forma sistemática un conjunto de criterios predefinidos. El uso de este agente garantiza coherencia en el proceso de evaluación, al aplicar siempre las mismas instrucciones y referencias para cada respuesta. Esto permite comparar de manera objetiva los resultados obtenidos con cada configuración del sistema y asegurar la reproducibilidad del análisis.

El apartado se estructura en secciones dedicadas a cada uno de los casos de uso, donde se presentan los resultados obtenidos y se explican las diferencias observadas. A continuación, se ofrece un análisis comparativo entre configuraciones y, finalmente, se expone una reflexión general sobre los hallazgos más relevantes de la evaluación.

6.1. Metodología de evaluación

6.1.1. Casos de uso definidos

Con el objetivo de evaluar el sistema propuesto en contextos realistas y alineados con entornos empresariales, se han definido cinco casos de uso representativos que simulan situaciones habituales en organizaciones. Estos casos permiten explorar cómo responde el workflow de recuperación cuando se enfrenta a diferentes necesidades de recuperación de información, todas ellas relevantes para mejorar la gestión del conocimiento dentro de una empresa.

Cada caso de uso refleja un escenario práctico en el que una organización necesita acceder de forma ágil, precisa y trazable a su conocimiento interno. Esta aproximación permite examinar tanto la eficacia como la adaptabilidad del sistema propuesto ante distintos tipos de información y requisitos de consulta.

A continuación se describen brevemente los cinco casos de uso definidos:

- **Caso de uso 1: Consulta sobre documentos legales y normativos**
Representa escenarios donde una empresa necesita acceder a legislación vigente, normativas internas o boletines oficiales para garantizar el cumplimiento normativo y apoyar procesos jurídicos o administrativos.
- **Caso de uso 2: Búsqueda de URLs de documentos**
Simula situaciones en las que los usuarios requieren recuperar documentos clave, como contratos, informes o facturas, a través de su ubicación (URL) en sistemas cloud internos de la organización.
- **Caso de uso 3: Análisis y recuperación de información en correos y conversaciones**
Reproduce necesidades frecuentes de recuperación de conocimiento implícito en comunicaciones, como respuestas a clientes, acuerdos con proveedores o información compartida internamente por correo electrónico.
- **Caso de uso 4: Consulta sobre código fuente y funciones técnicas**
Aborda el acceso al conocimiento técnico almacenado en el código fuente de la empresa, permitiendo a los equipos de desarrollo localizar funciones, entender lógicas de negocio o reutilizar componentes ya implementados.
- **Caso de uso 5: Indexación de fichas y catálogos de productos**
Este caso refleja situaciones donde se requiere acceder a información estructurada almacenada en bases de datos internas, como fichas de productos o catálogos de servicios. Las consultas típicas están relacionadas con características, funcionalidades o condiciones de uso de las soluciones ofrecidas por la empresa.

Estos casos de uso han sido seleccionados por su representatividad en distintos dominios empresariales: jurídico, administrativo, comercial, técnico y comunicativo. Esta variedad asegura una evaluación exhaustiva del workflow de recuperación, no solo desde un punto de vista funcional, sino también en cuanto a su utilidad real en contextos corporativos donde la información debe ser accesible, confiable y contextualizada.

6.1.2. Agente evaluador y criterios de evaluación

Para garantizar una evaluación objetiva, coherente y repetible de los resultados generados por el workflow de recuperación, se ha diseñado un agente inteligente evaluador basado en lenguaje natural. Este agente ha sido desarrollado con un prompt cuidadosamente definido que le permite aplicar siempre los mismos criterios de análisis, eliminando así cualquier posible sesgo humano o variabilidad interpretativa en el proceso de evaluación.

El agente actúa de forma automática sobre cada respuesta generada por el sistema, tomando como entrada tres elementos fundamentales: la pregunta original planteada por el usuario, la respuesta proporcionada por el workflow de recuperación y los nodos de información que han sido utilizados para sintetizar dicha respuesta. A partir de estos elementos, el agente evalúa la calidad de la respuesta según cinco criterios predefinidos. Para cada uno de ellos, asigna una puntuación discreta entre 0.0 y 1.0, reflejando el nivel de cumplimiento observado.

Los criterios de evaluación considerados son los siguientes:

- **Relevance (Relevancia):** Evalúa si la respuesta se ajusta de forma directa y clara a la intención de la consulta del usuario. Una respuesta relevante debe abordar con precisión el contenido esperado y evitar desviaciones innecesarias.
- **Completeness (Compleitud):** Mide si la respuesta cubre todos los aspectos relevantes de la pregunta, incluyendo detalles, pasos o elementos adicionales que puedan ser necesarios para una comprensión completa de la información solicitada.
- **Coherence (Coherencia):** Analiza si la respuesta está bien estructurada, redactada con claridad y coherencia interna, y si utiliza un lenguaje adecuado para un entorno técnico y profesional. También considera la fluidez y legibilidad del texto.
- **Source Logic (Lógica de las fuentes):** Verifica si los nodos recuperados guardan una relación lógica y semántica con la consulta planteada. Se espera que los fragmentos utilizados estén efectivamente alineados con el contenido de la pregunta.
- **Source Usage (Uso de las fuentes):** Evalúa hasta qué punto la respuesta generada se fundamenta en el contenido real de los nodos utilizados, reutilizando adecuadamente ideas, conceptos o expresiones de los textos fuente.

Cada uno de estos criterios es evaluado de manera independiente, y el agente asigna una puntuación específica en función del grado de cumplimiento observado. Además de las valoraciones individuales, el agente calcula una nota final que representa la calidad global de la respuesta, obtenida como promedio de las puntuaciones de los cinco criterios anteriores. Este enfoque permite analizar en profundidad los puntos fuertes y débiles de cada configuración del sistema, a la vez que facilita la comparación entre distintas ejecuciones.

Para cada uno de estos cinco criterios, el agente asigna una puntuación discreta de entre los siguientes valores:

- **1.0 (Excelente):** El criterio se cumple por completo. La respuesta es totalmente relevante, completa, coherente y correctamente fundamentada en fuentes alineadas con la consulta.
- **0.75 (Bueno):** Se cumple casi completamente, aunque puede haber pequeñas desviaciones, omisiones menores o aspectos ligeramente mejorables.
- **0.5 (Aceptable):** El criterio se cumple parcialmente. La respuesta cubre algunos aspectos, pero deja fuera otros relevantes, o presenta limitaciones notables en su estructura o fundamentación.
- **0.25 (Deficiente):** El criterio apenas se cumple. La conexión con la pregunta es débil, la respuesta es confusa o las fuentes utilizadas son poco útiles o irrelevantes.
- **0.0 (Inadecuado):** El criterio no se cumple en absoluto. La respuesta es irrelevante, incoherente o no utiliza en absoluto las fuentes recuperadas.

El uso de un agente evaluador automatizado permite mantener un estándar uniforme a lo largo de todo el proceso de análisis, asegurando que las decisiones se basan en parámetros objetivos y reproducibles. Esta metodología es especialmente relevante en entornos empresariales, donde la consistencia en la evaluación de sistemas de inteligencia artificial resulta fundamental para garantizar su fiabilidad y escalabilidad.

6.1.3. Metodología de evaluación

Para llevar a cabo una evaluación rigurosa y comparativa del sistema propuesto, se ha diseñado una metodología experimental que permita analizar su comportamiento en diferentes escenarios empresariales. Esta metodología se fundamenta en la aplicación de las tres configuraciones del workflow de recuperación previamente descritas en el capítulo cinco: configuración básica, configuración normal y configuración completa. Cada una de estas variantes incorpora distintos niveles de complejidad en los mecanismos de recuperación, síntesis y evaluación, lo que permite observar cómo varía el rendimiento global del sistema según se activan o desactivan determinados componentes.

El procedimiento de evaluación se ha estructurado en torno a los cinco casos de uso definidos en la sección anterior, los cuales representan situaciones empresariales reales donde el acceso eficiente al conocimiento es clave. En cada uno de estos casos de uso, se han formulado 15 preguntas diferentes, diseñadas para simular consultas típicas que podrían surgir en un entorno corporativo. Estas preguntas se han procesado con cada una de las tres configuraciones, generando así un total de 45 ejecuciones por caso de uso y 225 pruebas en total a lo largo del experimento.

Para cada ejecución se han registrado dos métricas clave: por un lado, la puntuación generada por el agente evaluador descrito en el apartado 6.1.2, que analiza la calidad de la respuesta según cinco criterios y proporciona una nota final global; y por otro lado, el tiempo de ejecución, medido en segundos, desde que se introduce la consulta hasta que se genera la respuesta final. En lo que respecta a las puntuaciones por criterio, se ha optado por una estrategia basada en la moda, es decir, se ha seleccionado la puntuación más repetida entre las 15 ejecuciones de cada configuración y caso de uso. Esta decisión responde a la necesidad de mantener una interpretación cualitativa clara y coherente de los resultados, en consonancia con la escala de valores definida en el apartado anterior. A diferencia del promedio, que puede diluir la representatividad de cada nivel de calidad, la moda permite identificar con mayor claridad el tipo de respuesta que se genera con mayor frecuencia bajo cada configuración. En cuanto al tiempo de ejecución, se ha calculado la media aritmética de las 15 ejecuciones para cada configuración y caso de uso, con el objetivo de ofrecer una estimación representativa del rendimiento temporal del sistema.

Este enfoque metodológico permite evaluar de forma objetiva y reproducible el impacto de las distintas configuraciones del sistema en términos tanto de calidad como de eficiencia. Asimismo, facilita la identificación de configuraciones óptimas en función del tipo de tarea o dominio, contribuyendo a la toma de decisiones sobre la adopción de sistemas de recuperación en entornos empresariales complejos.

6.2. Caso de uso 1: Consulta sobre documentos legales y normativos

Este primer caso de uso representa una situación habitual en el ámbito empresarial e institucional, la necesidad de consultar documentos legales o normativos ya indexados por el sistema. Este tipo de contenido suele incluir boletines oficiales, reglamentos internos, leyes sectoriales o resoluciones administrativas. La consulta puede provenir tanto de personal jurídico como de otros departamentos que requieren confirmar obligaciones, normativas o procesos regulados.

Las consultas típicas en este escenario podrían adoptar formas como:

“¿Qué ley regula el aspecto X?” o “¿Dónde se menciona la obligación de presentar el documento X?”, siendo X un elemento variable que representa conceptos jurídicos, procedimientos administrativos o tipos documentales específicos.

A continuación, se presenta la tabla con los resultados obtenidos al evaluar las tres configuraciones del workflow de recuperación para este caso de uso:

Criterio	Tiempo	Nota relevancia	Nota completitud	Nota coherencia	Nota lógica de las fuentes	Nota uso de las fuentes	Nota final
Config básica	32s	0.75	0.75	1.0	0.75	1.0	0.85
Config normal	47s	1.0	0.75	1.0	1.0	1.0	0.95
Config completa	58s	1.0	1.0	1.0	1.0	1.0	1.0

Tabla 5. Resultados de la evaluación de configuraciones del caso de uso 1

La **Tabla 5** muestra de forma clara la evolución de los resultados en función del nivel de complejidad de la configuración utilizada en el workflow de recuperación. En primer lugar, se puede observar un incremento progresivo en el tiempo de ejecución: mientras que la configuración básica tarda una media de 32 segundos en generar una respuesta completa, la configuración normal asciende a 47 segundos y la configuración completa alcanza los 58 segundos. Este aumento es coherente con el número de etapas incluidas en cada configuración, especialmente las relacionadas con recuperación jerárquica, evaluación y consolidación avanzada.

En cuanto a las puntuaciones asignadas por el agente evaluador, la configuración básica obtiene una media final de 0.85, reflejando una calidad aceptable en todos los criterios menos la coherencia y el uso de las fuentes que ha obtenido la nota máxima 1.0. En concreto, se observan puntuaciones de 0.75 en relevancia, completitud y lógica de las fuentes, lo que sugiere que, si bien las respuestas son razonablemente acertadas, pueden dejar fuera matices importantes o no fundamentarse del todo bien en los documentos utilizados.

La configuración normal mejora estos resultados de manera significativa, obteniendo una nota final de 0.95. Aquí destaca especialmente la puntuación de 1.0 en cuatro de los cinco criterios evaluados (relevancia, coherencia, lógica de las fuentes y uso de las fuentes), lo cual indica que las respuestas generadas son precisas, bien fundamentadas y estructuralmente sólidas. El único criterio que se mantiene en 0.75 es completitud, lo que puede reflejar casos en los que ciertos detalles específicos no fueron completamente abordados en la respuesta.

Finalmente, la configuración completa alcanza la puntuación máxima posible en todos los criterios, con una media final de 1.0. Este resultado sugiere que la respuesta generada es plenamente relevante, completa, coherente y respaldada por una lógica clara basada en las fuentes. Además, a pesar de ser la configuración más costosa en términos de tiempo, demuestra un rendimiento óptimo en cuanto a calidad de respuesta, lo que la convierte en la solución más robusta para contextos donde la precisión y la trazabilidad son prioritarias.

6.3. Caso de uso 2: Recuperación de URLs de documentos

Este caso de uso representa una situación frecuente en entornos empresariales donde los documentos clave como facturas, contratos o informes técnicos están almacenados en ubicaciones accesibles mediante enlaces directos (URLs), ya sea en servicios cloud internos o externos. La función del workflow de recuperación en este contexto consiste en identificar correctamente los documentos solicitados y devolver las URLs asociadas.

Ejemplos de consultas formuladas por los usuarios incluyen: “Muéstreme las facturas de electricidad del mes de X” o “¿Dónde está el contrato firmado con los salarios del equipo técnico?”. Estas preguntas requieren no tanto una síntesis textual, sino una identificación precisa del documento pertinente y su localización.

A continuación, se presenta la tabla con los resultados obtenidos al evaluar las tres configuraciones del workflow de recuperación para este caso de uso:

Criterio	Tiempo	Nota relevancia	Nota completitud	Nota coherencia	Nota lógica de las fuentes	Nota uso de las fuentes	Nota final
Config básica	18s	1.0	1.0	1.0	0.75	1.0	0.95
Config normal	30s	1.0	1.0	1.0	1.0	1.0	1.0
Config completa	40s	1.0	1.0	1.0	1.0	1.0	1.0

Tabla 6. Resultados de la evaluación de configuraciones del caso de uso 2

La **Tabla 6** muestra que todas las configuraciones alcanzan niveles sobresalientes de rendimiento en este tipo de tareas. La configuración básica obtiene puntuaciones máximas (1.0) en relevancia, completitud, coherencia y uso de las fuentes, lo que indica que el sistema identificó correctamente los documentos requeridos y devolvió las URLs correspondientes de

forma clara y estructurada. La única ligera desviación se observa en la lógica de las fuentes (0.75), lo que sugiere que, en algunos casos, los nodos utilizados para generar la respuesta podrían no estar óptimamente alineados semánticamente con la consulta, a pesar de que el resultado final fue adecuado. La nota global resultante es de 0.95, con un tiempo de ejecución muy reducido de 18 segundos, lo que subraya su eficiencia operativa.

Tanto la configuración normal como la completa alcanzan la puntuación máxima (1.0) en todos los criterios evaluados. Esto confirma que ambas variantes del sistema fueron capaces de identificar correctamente los documentos deseados, justificar la respuesta con evidencia textual relevante, y estructurar la salida con claridad. La diferencia entre ambas configuraciones reside únicamente en el tiempo de ejecución: 30 segundos para la configuración normal y 40 segundos para la completa. Esta diferencia refleja la complejidad añadida de los pasos adicionales, que si bien no aportaron mejoras adicionales en este escenario, siguen funcionando correctamente y sin degradar la calidad de la respuesta.

6.4. Caso de uso 3: Análisis y recuperación de información en correos y conversaciones

Este caso de uso se centra en la recuperación de información contenida en correos electrónicos y conversaciones mantenidas con clientes, proveedores u otros actores empresariales. Se trata de una situación muy habitual en entornos corporativos donde la comunicación escrita constituye una fuente clave de conocimiento, seguimiento de compromisos y trazabilidad operativa. La finalidad principal de este flujo es permitir que los usuarios puedan plantear preguntas como: “¿Qué cliente preguntó sobre el producto X?” o “¿Qué le respondimos al proveedor X sobre la entrega?”, siendo X un elemento variable dependiente del contexto. El sistema debe ser capaz de identificar los hilos de conversación relevantes, recuperar los fragmentos de texto adecuados y sintetizar respuestas fundamentadas que reflejen fielmente lo que se comunicó en su momento.

A continuación, se presenta la tabla con los resultados obtenidos al evaluar las tres configuraciones del workflow de recuperación para este caso de uso:

Criterio	Tiempo	Nota relevancia	Nota completitud	Nota coherencia	Nota lógica de las fuentes	Nota uso de las fuentes	Nota final
Config básica	25s	0.75	0.75	1.0	0.75	1.0	0.85
Config normal	38s	1.0	0.75	1.0	1.0	1.0	0.95
Config completa	57s	1.0	1.0	1.0	1.0	1.0	1.0

Tabla 7. Resultados de la evaluación de configuraciones del caso de uso 3

Como se observa en la **Tabla 7**, la configuración básica ha obtenido una nota final de 0.85, con puntuaciones parciales de 0.75 tanto en relevancia como en completitud, y máximas en

coherencia y uso de las fuentes. La puntuación de 0.75 en lógica de las fuentes sugiere que, en algunas de las respuestas generadas, los fragmentos recuperados no estaban suficientemente alineados con la consulta del usuario, aunque fueron utilizados adecuadamente en la síntesis final.

La configuración normal, en cambio, mejora notablemente en casi todos los aspectos, alcanzando una nota final de 0.95. Se mantiene la nota de 0.75 en completitud, lo que indica que algunas respuestas, si bien correctas, podrían haber incluido detalles adicionales. Aun así, destaca por una excelente alineación entre los nodos recuperados y la consulta, así como una correcta utilización de los mismos.

La configuración completa, como en los casos anteriores, obtiene la máxima puntuación en todos los criterios, con una nota final de 1.0. Esta configuración ha demostrado ser capaz de recuperar no solo los fragmentos adecuados, sino también aquellos que complementan el contexto de forma más amplia, incorporando tanto las secciones completas como los resúmenes necesarios para una respuesta precisa y argumentada.

En cuanto al tiempo de ejecución, se mantiene la tendencia observada en los casos anteriores: la configuración básica es la más eficiente (25 segundos), mientras que la completa, con 57 segundos, es la más costosa en tiempo. La configuración normal presenta un equilibrio razonable entre ambas (38 segundos), lo cual puede ser relevante en entornos donde se prioriza la rapidez sin sacrificar demasiado la calidad.

6.5. Caso de uso 4: Consulta sobre código fuente y funciones técnicas

Este caso de uso representa una situación habitual en empresas tecnológicas o departamentos de IT, donde es necesario consultar grandes bases de código fuente o documentación técnica interna. El objetivo principal es permitir a los desarrolladores acceder de forma eficiente al conocimiento técnico previamente indexado. Entre las consultas típicas se encuentran preguntas como: *"¿Qué función realiza el cálculo de comisiones?"* o *"¿Cómo se usa la función que genera los informes en PDF?"*. Este tipo de preguntas requieren precisión semántica, capacidad para interpretar código estructurado y la habilidad de sintetizar respuestas claras a partir de fragmentos dispersos.

La siguiente tabla resume los resultados obtenidos al evaluar las tres configuraciones del workflow de recuperación en este contexto:

Criterio	Tiempo	Nota relevancia	Nota completitud	Nota coherencia	Nota lógica de las fuentes	Nota uso de las fuentes	Nota final
Config básica	28s	0.75	0.5	1.0	0.75	1.0	0.8
Config normal	40s	1.0	0.75	1.0	1.0	1.0	0.95
Config completa	65s	1.0	1.0	1.0	1.0	1.0	1.0

Tabla 8. Resultados de la evaluación de configuraciones del caso de uso 4

El análisis de los datos de la **Tabla 8** revela que la configuración básica obtuvo una nota final de 0.8, destacando en coherencia (1.0) y uso de las fuentes (1.0), pero con puntuaciones más bajas en relevancia (0.75) y especialmente en completitud (0.5). Esto sugiere que, si bien las respuestas eran claras y bien estructuradas, en muchos casos no lograban cubrir todos los aspectos técnicos solicitados por el usuario ni identificar con exactitud el fragmento de código más relevante.

La configuración normal mejora considerablemente estos aspectos, alcanzando una nota final de 0.95. Se observa una mayor capacidad de identificar funciones relevantes y aportar información más completa sobre su ubicación, lógica o uso esperado. El uso combinado de recuperación semántica y recursiva, junto con el reranking de nodos, permitió una recuperación más precisa en entornos donde la lógica está repartida entre distintos módulos o archivos.

La configuración completa obtuvo una nota perfecta de 1.0 en todos los criterios, gracias a los mecanismos adicionales de enriquecimiento, como la incorporación de secciones completas y nodos relacionados. Este enfoque fue especialmente útil en escenarios donde la respuesta requería comprender relaciones entre diferentes fragmentos de código, dependencias internas o reutilización de funciones. Las respuestas generadas no solo eran correctas, sino también completas y bien fundamentadas.

En cuanto al rendimiento, la configuración básica fue la más rápida (28 segundos), mientras que la completa alcanzó los 65 segundos debido al mayor número de operaciones. La configuración normal se mantuvo en un punto intermedio (40 segundos), logrando un buen equilibrio entre eficiencia y calidad.

6.6. Caso de uso 5: Indexación de fichas y catálogos de productos

Este caso de uso representa una situación común en empresas que gestionan un amplio catálogo de productos, servicios o soluciones, especialmente en sectores como el comercio electrónico, el software como servicio (*SaaS*), o la atención al cliente. En estos contextos, es habitual disponer de bases de datos corporativas que contienen información estructurada sobre productos, funcionalidades, condiciones de venta, integración con otras plataformas, así como características técnicas, precios o disponibilidad.

Para este caso, se ha asumido que toda la información relevante ha sido previamente indexada a partir de dichas bases de datos empresariales. Esto incluye tanto fichas de producto como documentación técnica asociada, permitiendo que el sistema de recuperación pueda operar sobre una representación estructurada del catálogo interno.

Ejemplos representativos de consultas realizadas en este escenario incluyen preguntas como: “¿Qué producto ofrece integración con Microsoft Teams?” o “¿Cuál de nuestros servicios incluye soporte técnico 24/7?”. Este tipo de preguntas requieren localizar información precisa y actualizada dentro de grandes volúmenes de contenido descriptivo y técnico.

A continuación, se presentan los resultados obtenidos al evaluar las distintas configuraciones del workflow de recuperación para este caso:

Criterio	Tiempo	Nota relevancia	Nota completitud	Nota coherencia	Nota lógica de las fuentes	Nota uso de las fuentes	Nota final
Config básica	32s	1.0	0.75	1.0	0.75	1.0	0.9
Config normal	42s	1.0	0.75	1.0	1.0	1.0	0.95
Config completa	53s	1.0	1.0	1.0	1.0	1.0	1.0

Tabla 9. Resultados de la evaluación de configuraciones del caso de uso 5

Como se observa en la **Tabla 9**, en cuanto a la configuración básica, el sistema obtuvo una nota final de 0.9. Esta puntuación refleja un buen desempeño general, especialmente en términos de relevancia, coherencia y uso adecuado de las fuentes (1.0 en todos estos criterios). Sin embargo, se observa una ligera penalización en los criterios de completitud (0.75) y lógica de las fuentes (0.75), lo que indica que en algunas ocasiones la respuesta no cubría todos los aspectos esperados o no justificaba completamente la información proporcionada a partir del contenido indexado.

La configuración normal mejora la calidad de la recuperación y síntesis, alcanzando una nota final de 0.95. Gracias a la combinación de recuperación semántica y recursiva, así como al uso del reranking y evaluación de fidelidad, se logró ofrecer respuestas más completas, estructuradas y alineadas con los nodos relevantes del catálogo de productos. En particular, se observa una mejora en la lógica de las fuentes, donde se obtuvo la puntuación máxima.

La configuración completa volvió a alcanzar la puntuación perfecta (1.0) en todos los criterios. Esto confirma que el uso de nodos relacionados, secciones completas, y resúmenes adicionales resulta especialmente útil en dominios donde los datos están muy interconectados o distribuidos en diferentes fuentes. Las respuestas generadas fueron no solo correctas y claras, sino también exhaustivas y bien fundamentadas, lo cual es fundamental en contextos empresariales donde las decisiones comerciales pueden depender directamente de la información recuperada.

En términos de eficiencia, la configuración básica fue la más rápida (32 segundos), seguida por la configuración normal (42 segundos), mientras que la completa requirió algo más de tiempo (53 segundos), lo cual resulta esperable dadas las etapas adicionales implicadas en su ejecución.

6.7. Análisis comparativo y discusión

Evaluar el comportamiento de un sistema de recuperación basado en inteligencia artificial requiere no solo medir su rendimiento global, sino también analizar su eficacia en distintos escenarios concretos que reflejen necesidades reales del entorno empresarial. En este contexto, se ha llevado a cabo un análisis comparativo exhaustivo a partir de los cinco casos de uso definidos previamente, cada uno representando una categoría distinta de tareas relevantes para organizaciones que gestionan grandes volúmenes de información documental.

Este análisis permite identificar en qué medida cada configuración del sistema “básica, normal y completa” se adapta a los requerimientos específicos de cada situación, considerando tanto la calidad de las respuestas generadas como la eficiencia temporal de ejecución. A través de este enfoque, es posible detectar fortalezas y debilidades relativas de cada variante del workflow de recuperación, y establecer recomendaciones precisas sobre cuál resulta más adecuada en función del tipo de consulta y del contexto operativo.

En el caso de uso 1, centrado en la consulta de documentos legales y normativos, la configuración completa ha ofrecido los mejores resultados en términos de calidad global. Esto se debe principalmente a que este tipo de consultas requiere una comprensión precisa del lenguaje jurídico y una recuperación exhaustiva de referencias normativas. Sin embargo, la configuración normal también mostró un rendimiento muy elevado, con una diferencia mínima respecto a la completa, por lo que podría ser preferible si se desea un menor coste computacional.

En el caso de uso 2, relativo a la búsqueda de documentos empresariales almacenados por URL, todas las configuraciones alcanzaron puntuaciones perfectas, lo que indica que este tipo de tareas puede resolverse adecuadamente incluso con una configuración básica. Dado que el objetivo principal es identificar documentos relevantes, y la respuesta consiste principalmente en devolver un enlace preciso, la complejidad adicional de las configuraciones más avanzadas no aporta beneficios sustanciales en este escenario. Por tanto, la configuración básica resulta la opción más eficiente y suficiente.

En el caso de uso 3, enfocado en la recuperación de información contenida en correos electrónicos y conversaciones, la configuración normal ofreció un equilibrio óptimo entre calidad y tiempo de respuesta. Aunque la configuración completa también alcanzó puntuaciones máximas, la diferencia en el coste temporal no se justifica plenamente en comparación con la mejora marginal obtenida. La configuración básica, por su parte, mostró limitaciones en completitud y lógica de las fuentes, lo que sugiere que no es adecuada para este tipo de tareas que requieren inferencia contextual.

Para el caso de uso 4, orientado a consultas sobre código fuente y funciones técnicas, la configuración completa resultó claramente superior. Estas tareas requieren analizar contenido estructurado y técnico, donde los pasos adicionales como la jerarquía de recuperación y la inclusión de secciones completas ofrecen una ventaja significativa. La configuración básica, en

cambio, mostró problemas de completitud y lógica de las fuentes, lo que compromete la utilidad de las respuestas.

Finalmente, en el caso de uso 5, que trata la indexación de fichas y catálogos de productos extraídos de bases de datos empresariales, la configuración normal demostró ser la más adecuada. Ofrece una calidad elevada con un tiempo razonable, lo que la convierte en una opción eficiente para consultas frecuentes relacionadas con productos y servicios. Aunque la configuración completa ofreció la mejor puntuación, la diferencia con la normal es mínima, y el aumento de tiempo de ejecución puede no ser justificable para este tipo de tareas.

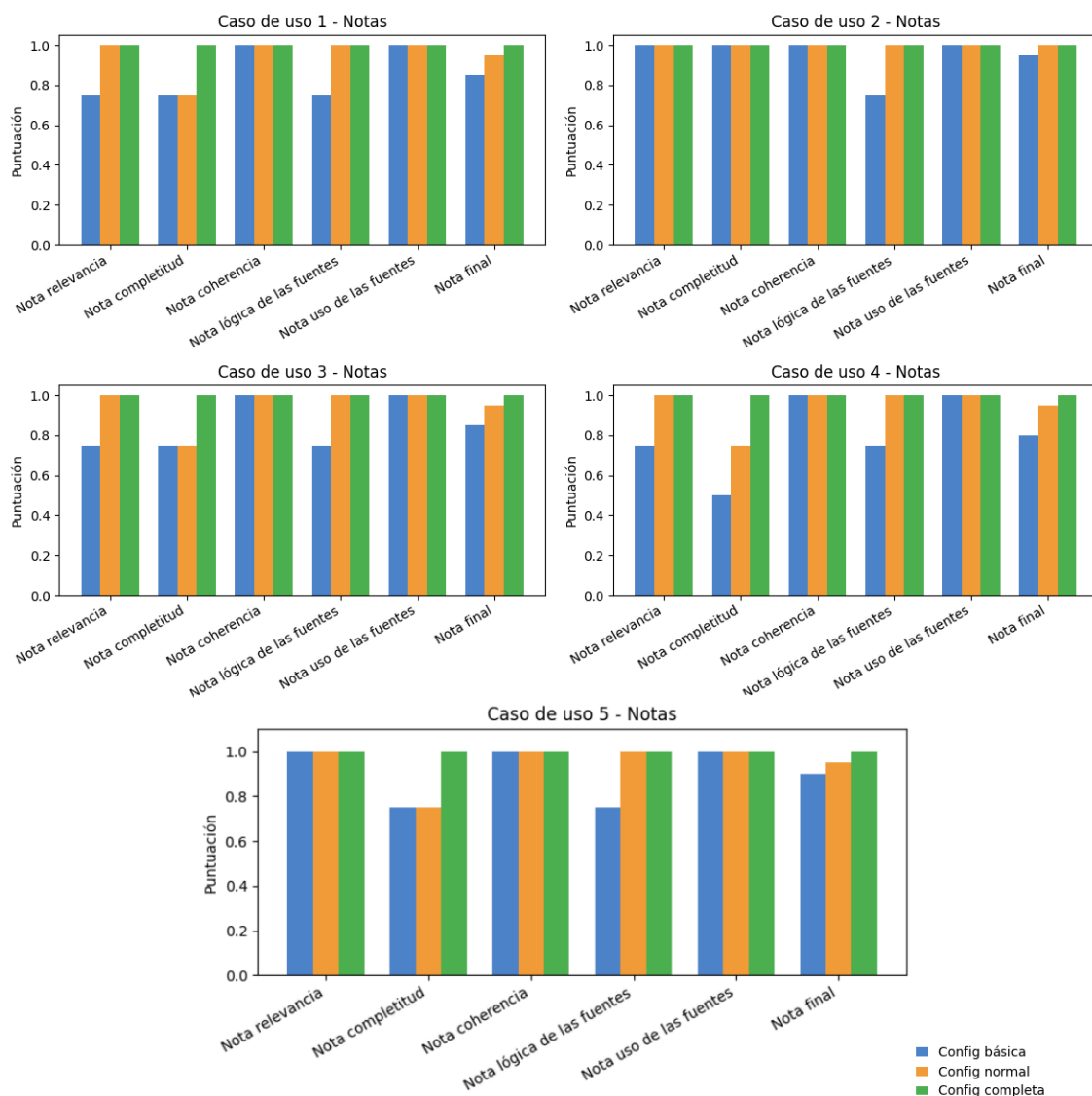


Figura 9. Comparativa de puntuaciones obtenidas por configuración en los cinco casos de uso

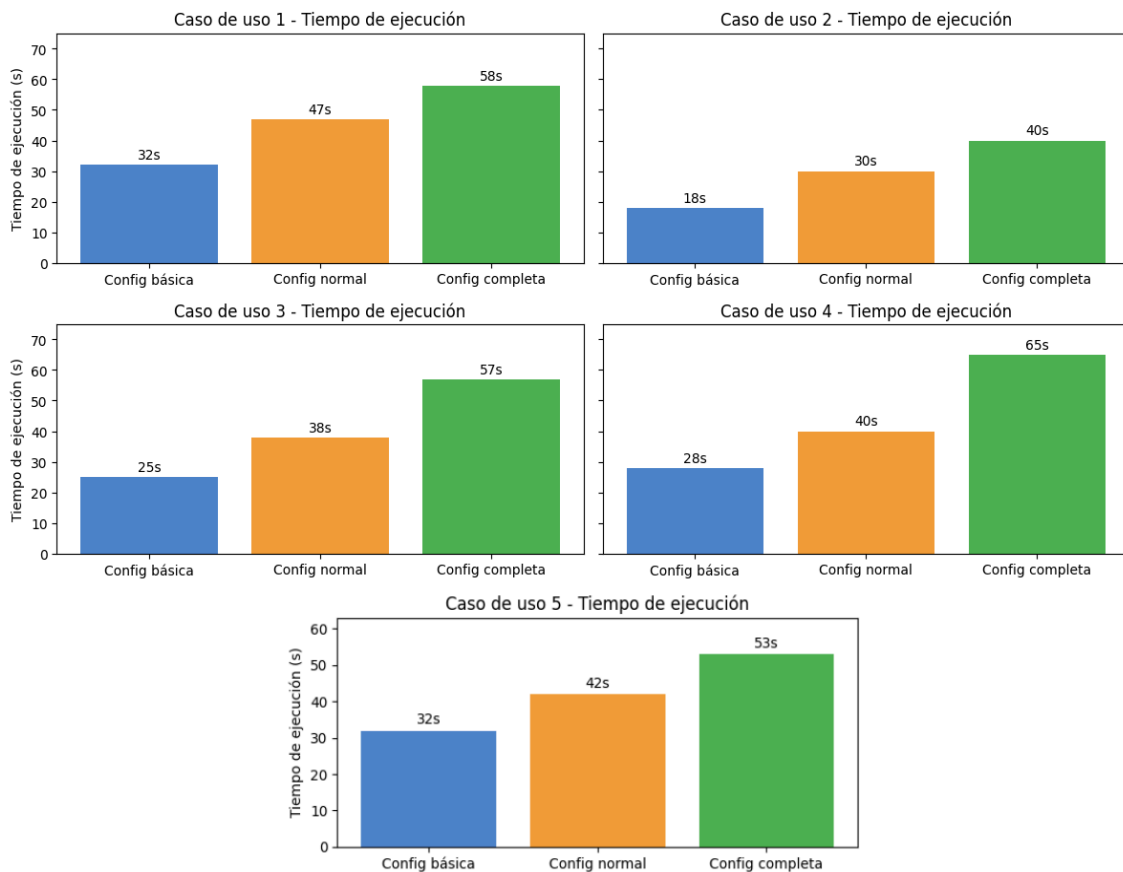


Figura 10. Comparativa del tiempo medio de ejecución por configuración en los cinco casos de uso

La **Figura 9** y la **Figura 10** recogen de forma visual los resultados obtenidos en los cinco casos de uso evaluados. La primera muestra la evolución de las puntuaciones por cada criterio de evaluación, permitiendo observar cómo varía la calidad de las respuestas en función de la configuración utilizada. La segunda presenta los tiempos medios de ejecución por configuración, destacando las diferencias de eficiencia computacional entre las variantes del sistema. Estas visualizaciones permiten contrastar de manera global los compromisos entre precisión y eficiencia, reforzando las conclusiones obtenidas en el análisis comparativo anterior.

6.8. Conclusiones

El presente capítulo ha permitido evaluar de manera rigurosa el comportamiento del workflow de recuperación desarrollada en este proyecto, contrastando su rendimiento bajo tres configuraciones distintas y aplicándolas a cinco casos de uso representativos de necesidades reales en entornos empresariales. La metodología empleada, basada en la evaluación automática mediante un agente estructurado, ha garantizado objetividad, repetibilidad y trazabilidad en todo el proceso.

Los resultados obtenidos han evidenciado que no existe una configuración universalmente superior, sino que la idoneidad de cada variante depende en gran medida del contexto y de la naturaleza de las consultas realizadas. Así, la configuración básica se ha mostrado adecuada

para tareas directas como la recuperación de documentos por URL, donde la precisión puntual es suficiente y se valora una alta eficiencia temporal. La configuración normal ha ofrecido un equilibrio sólido en la mayoría de los escenarios, siendo especialmente recomendable en situaciones que combinan precisión y agilidad operativa. Por su parte, la configuración completa ha destacado en casos que requieren un análisis profundo, como consultas jurídicas complejas o preguntas sobre código fuente, donde la exhaustividad y la solidez argumentativa son prioritarias.

Este análisis comparativo no solo ha permitido identificar las fortalezas y limitaciones de cada variante del sistema, sino que también ha demostrado la importancia de diseñar soluciones adaptativas, capaces de ajustarse dinámicamente a los requerimientos del entorno. En el contexto empresarial, donde las necesidades de información pueden ser muy heterogéneas, esta capacidad de adaptación resulta clave para garantizar tanto la utilidad como la escalabilidad de sistemas basados en inteligencia artificial.

Con esta evaluación se cierra el ciclo de diseño, implementación y validación de la propuesta técnica, y se sientan las bases para futuras líneas de mejora y optimización, que se abordarán en las conclusiones generales del trabajo.

7. Conclusiones y Futuras Líneas de Trabajo

El presente capítulo tiene como finalidad sintetizar las principales conclusiones obtenidas tras el desarrollo del Trabajo de Fin de Grado, así como analizar el alcance real del sistema propuesto y las oportunidades de mejora identificadas a lo largo del proceso. Además, se analiza cómo este proyecto se relaciona con las competencias adquiridas durante la titulación, destacando el uso integrado de tecnologías y metodologías avanzadas para resolver un problema real del ámbito empresarial. Finalmente, se presentan diversas líneas de trabajo futuras que permitirían ampliar, optimizar o automatizar el sistema desarrollado, facilitando su aplicación práctica en entornos corporativos.

7.1. Conclusiones generales

A lo largo de este Trabajo de Fin de Grado se ha llevado a cabo la implementación de una solución orientada a la automatización de la gestión del conocimiento empresarial mediante el diseño y desarrollo de workflows inteligentes. El proyecto ha cumplido con los objetivos planteados inicialmente, permitiendo abordar de forma estructurada un problema real que afecta a muchas organizaciones: la dificultad para acceder y utilizar eficazmente la información dispersa en múltiples fuentes.

En primer lugar, se ha realizado un análisis exhaustivo del estado del arte en relación con frameworks, arquitecturas y herramientas disponibles para la construcción de sistemas basados en workflows. Esta revisión ha servido de base para justificar el uso de LlamaIndex como tecnología principal, destacando su adecuación para entornos donde la trazabilidad, la personalización y el control del flujo de datos son esenciales.

A partir de este análisis, se ha diseñado e implementado un workflow de indexación flexible y modular, capaz de integrar documentos desde fuentes heterogéneas como carpetas locales, plataformas en la nube como OneDrive o servicios corporativos como Salesforce. El flujo permite preprocesar los datos, extraer metadatos relevantes y almacenarlos de manera estructurada en una base de datos optimizada para recuperación semántica.

De forma complementaria, se ha desarrollado un workflow de recuperación basado en técnicas de Retrieval-Augmented Generation, diseñado para responder de forma precisa y contextualizada a consultas en lenguaje natural. Este flujo integra diversos componentes inteligentes, como agentes de reescritura de preguntas, sintetizadores de respuestas y evaluadores de fidelidad, que trabajan de manera coordinada para generar respuestas trazables, comprensibles y adaptadas al contexto empresarial.

El sistema propuesto ha sido evaluado con datos reales, a través de cinco casos de uso representativos de escenarios empresariales. Las pruebas han incluido no sólo el análisis de calidad de las respuestas generadas mediante un agente evaluador, sino también la medición de tiempos de ejecución para valorar la eficiencia de cada configuración del sistema. Los

resultados han demostrado que el enfoque diseñado permite alcanzar un equilibrio adecuado entre precisión, adaptabilidad y coste temporal.

Durante el desarrollo del proyecto, ha sido necesario adquirir conocimientos avanzados no contemplados de forma directa en el plan de estudios, incluyendo el uso de arquitecturas orientadas a eventos, agentes inteligentes, evaluación automatizada de resultados y procesamiento avanzado de lenguaje natural. Esta adquisición de competencias técnicas y metodológicas ha supuesto un aprendizaje significativo y ha consolidado la capacidad para diseñar soluciones completas y robustas ante problemas complejos del entorno profesional.

Con todo ello, el trabajo desarrollado no solo ha demostrado la viabilidad técnica de los workflows inteligentes aplicados a la gestión del conocimiento empresarial, sino que también ha evidenciado la capacidad de combinar tecnologías avanzadas con un diseño modular y controlado. La solución obtenida constituye una base sólida sobre la que seguir construyendo sistemas más sofisticados y adaptables a nuevas necesidades, consolidando así una línea de trabajo con proyección real en entornos corporativos.

7.2. Relación del trabajo desarrollado con los estudios cursados

A lo largo de mis estudios del Grado en Ingeniería Informática, me he especializado en la rama de Computación, lo que me ha permitido adquirir una base conceptual en áreas relacionadas con el desarrollo de este proyecto. Aunque muchas de las tecnologías utilizadas en el Trabajo Fin de Grado van más allá del contenido abordado en las asignaturas, algunas materias clave me han proporcionado las ideas y fundamentos necesarios para entender y aplicar conceptos como los agentes inteligentes, la recuperación de información o la ejecución concurrente de tareas. Estas aportaciones académicas han sido útiles para enfrentar el diseño de workflows y la implementación de componentes basados en inteligencia artificial de forma más estructurada y razonada.

En primer lugar, la asignatura de Agentes Inteligentes (AIN) proporcionó el marco conceptual necesario para comprender el papel de los agentes autónomos dentro de un sistema distribuido. Aunque la forma de implementar agentes en este TFG —mediante arquitecturas basadas en LLM y flujos conversacionales estructurados— va más allá de lo visto en la asignatura, la familiaridad con conceptos como autonomía, comportamiento reactivo y coordinación entre entidades ha sido esencial para diseñar la lógica de interacción entre agentes dentro de los workflows desarrollados.

Por otro lado, la asignatura de Sistemas de Almacenamiento y Recuperación de Información (SAR) ha sido fundamental para abordar la parte teórica del sistema de recuperación. El conocimiento adquirido sobre estructuras de datos para indexación, modelos de representación como el vectorial, y técnicas de búsqueda semántica ha permitido entender y aplicar de forma rigurosa los mecanismos de recuperación utilizados en el sistema, como la búsqueda basada en embeddings y la selección de fragmentos relevantes para sintetizar respuestas.

Finalmente, la asignatura de Concurrencia y Sistemas Distribuidos (CSD) ha contribuido notablemente en la comprensión y aplicación de conceptos de ejecución paralela y asincronía. El workflow de recuperación implementado hace uso intensivo de programación asíncrona para ejecutar múltiples tareas en paralelo (como la recuperación desde varios retrievers o la síntesis

de múltiples respuestas), y la gestión correcta de estas operaciones requiere una base sólida en concurrencia y sincronización, tal como se trabajó en dicha asignatura.

En definitiva, aunque la mayoría de conocimientos específicos necesarios para llevar a cabo este proyecto han sido adquiridos de forma autodidacta o durante la realización del propio TFG, las asignaturas cursadas durante el grado han proporcionado una base sólida sobre la que construir. Algunas materias han sido especialmente útiles para comprender ciertos conceptos clave o para tomar decisiones técnicas con mayor criterio. Este trabajo ha sido una oportunidad para integrar y aplicar de manera práctica todo ese aprendizaje, y también para ampliar de forma significativa mis competencias en áreas como la inteligencia artificial aplicada, el diseño de sistemas distribuidos y la automatización de procesos.

7.3. Trabajos futuros

El desarrollo de este proyecto ha abierto múltiples líneas de evolución que permitirían mejorar y ampliar las capacidades del sistema propuesto, tanto en la fase de indexación como en la de recuperación. A continuación, se describen algunas propuestas de futuro que podrían consolidar el sistema como una herramienta aún más robusta, automatizada e integrada en entornos empresariales reales.

Una de las principales mejoras planteadas se centra en la automatización del proceso de indexación, especialmente para aquellos casos en los que los documentos provienen de fuentes dinámicas. Por ejemplo, en el caso de los archivos alojados en OneDrive, sería viable implementar una integración con *Microsoft Power Automate*, de manera que, al detectar automáticamente la incorporación de un nuevo documento en una carpeta designada (por ejemplo, “A indexar”), se desencadene la ejecución completa del flujo de indexación. Una vez finalizado el proceso, el documento podría trasladarse automáticamente a una carpeta final (“Indexados”), garantizando trazabilidad y organización sin intervención manual.

Además, se contempla la incorporación de *webhooks* como mecanismo de activación automática del flujo de indexación. Esta estrategia permitiría recibir notificaciones directas desde servicios externos como OneDrive, APIs empresariales o sistemas internos, y ejecutar de forma inmediata el workflow correspondiente sin necesidad de supervisión manual. Con ello se conseguiría una automatización más reactiva y conectada con los eventos reales del entorno de la organización.

Asimismo, para fuentes estructuradas como bases de datos, se plantea el uso de tareas programadas (*cron jobs*) que ejecuten de forma periódica el proceso de indexación. Esta estrategia permitiría mantener actualizado el conocimiento almacenado en el sistema sin necesidad de intervención directa, especialmente útil cuando se trata de bases de datos que registran interacciones recurrentes, como pueden ser registros de correos electrónicos, nuevos productos, entradas de clientes o cambios en servicios empresariales.

En cuanto a la fase de recuperación, una línea de trabajo especialmente interesante es la integración del sistema en plataformas de mensajería como Microsoft Teams, Slack o incluso WhatsApp. A través de la creación de un agente conversacional (*bot*), los usuarios podrían plantear sus consultas directamente en el canal de comunicación que ya utilizan habitualmente en su entorno laboral. Este bot actuaría como interfaz del sistema de recuperación, ejecutando el flujo correspondiente en segundo plano y devolviendo la respuesta sintetizada directamente en

el mismo canal de conversación, mejorando así la accesibilidad, usabilidad y adopción del sistema.

Por último, una mejora destacada sería la incorporación de mecanismos de *Human-In-The-Loop* (HITL), una técnica que introduce de forma controlada la intervención del usuario en flujos automatizados cuando se detecta ambigüedad o falta de contexto. En este enfoque, el sistema analiza la consulta inicial y, si identifica que no contiene suficiente información para generar una respuesta fiable, activa un agente especializado que detiene temporalmente la ejecución del flujo. Este agente formula una pregunta de aclaración al usuario utilizando el mismo canal de comunicación en el que se realizó la consulta (por ejemplo, Microsoft Teams o Slack). Una vez el usuario responde, el flujo se reanuda automáticamente utilizando la nueva información aportada para generar una respuesta más precisa y contextualizada.

En definitiva, las propuestas aquí expuestas no solo buscan perfeccionar el sistema actual, sino también avanzar hacia una solución más autónoma, integrada y adaptada a los flujos reales de las organizaciones. Al incorporar estas líneas de trabajo, el sistema podría convertirse en una herramienta aún más estratégica dentro del proceso de gestión del conocimiento empresarial, consolidando su utilidad práctica y su potencial de escalabilidad en contextos productivos reales.

Referencias:

1. *LlamaIndex - Build Knowledge Assistants over your Enterprise Data*. (n.d.-d). <https://www.llamaindex.ai/>
2. *LangChain*. (n.d.-c). <https://www.langchain.com/>
3. *Haystack | Haystack*. (n.d.-c). Haystack. <https://haystack.deepset.ai/>
4. *Flowise - Build AI agents, visually*. (n.d.-c). <https://flowiseai.com/>
5. n8n. (n.d.-b). *n8n.io - a powerful workflow automation tool*. N8n. <https://n8n.io/>
6. Pérez-Montoro, M. (2016). Gestión del conocimiento: orígenes y evolución. *Profesional de la información*, 25(4), 526-534.
7. Torres, K., & Lamenta, P. (2015). La gestión del conocimiento y los sistemas de información en las organizaciones. *Negotium*, 11(32), 3-20.
8. Dante, G. P., & Ponjuán, D. T. (2021). Gestionar la ignorancia para gestionar el conocimiento: una necesidad organizacional. *Revista Cubana de Información en Ciencias de la Salud (ACIMED)*, 32(1), 1-17.
9. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33, 9459-9474.
10. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2, 1.
11. Winland, V., & Russi, E. (2025, January 30). *¿Qué es LlamaIndex?*. IBM.
12. Krampah, P.(2024, July 22). Agentic RAG con Llama-index | Motor de consulta del enrutador #01 - Planeta Chatbot. Planeta Chatbot - Comunidad de expertos en IA Conversacional. <https://planetachatbot.com/agentic-rag-con-llama-index-motor-de-consulta-del-enrutador-01/>
13. LangChain vs LlamaIndex: Elija el mejor marco para sus aplicaciones de IA. (2024b, August 6). <https://myscale.com/blog/es/llamaindex-vs-langchain-detailed-comparison/>
14. Bruckhaus, T. (2024). Rag does not work for enterprises. *arXiv preprint arXiv:2406.04369*.
15. DataCamp. (2023). *Técnicas avanzadas de GAR*. <https://www.datacamp.com/es/blog/rag-advanced>
16. Nonaka, I., & Takeuchi, H. (1995). *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press.

17. Davenport, T. H., & Prusak, L. (1998). Working knowledge: How organizations manage what they know. Harvard Business Press.
18. Izacard, G., & Grave, E. (2020). Leveraging passage retrieval with generative models for open domain question answering. arXiv preprint arXiv:2007.01282.
19. Karpukhin, V., Oguz, B., Min, S., Lewis, P. S., Wu, L., Edunov, S., ... & Yih, W. T. (2020, November). Dense Passage Retrieval for Open-Domain Question Answering. In EMNLP (1) (pp. 6769-6781).
20. *Embeddings* - *LlamaIndex*. (n.d.).
https://docs.llamaindex.ai/en/stable/module_guides/models/embeddings/
21. Gil, Y., Ratnakar, V., Kim, J., Gonzalez-Calero, P., Groth, P., Moody, J., & Deelman, E. (2010). Wings: Intelligent workflow-based design of computational experiments. IEEE Intelligent Systems, 26(1), 62-72.
22. *Recursive Retriever + Query Engine Demo* - *LlamaIndex*. (n.d.).
https://docs.llamaindex.ai/en/stable/examples/query_engine/pdf_tables/recursive_retriever/
23. *Structured Hierarchical Retrieval* - *LlamaIndex*. (n.d.).
https://docs.llamaindex.ai/en/stable/examples/query_engine/multi_doc_auto_retrieval/multi_doc_auto_retrieval/
24. *Cohere's Rerank Model (Details and Application)* | *Cohere*. (n.d.). Cohere.
<https://docs.cohere.com/docs/rerank>
25. *Response Synthesizer* - *LlamaIndex*. (n.d.).
https://docs.llamaindex.ai/en/stable/module_guides/querying/response_synthesizers/
26. *Faithfulness Evaluator* - *LlamaIndex*. (n.d.).
https://docs.llamaindex.ai/en/stable/examples/evaluation/faithfulness_eval/
27. *Metadata Extraction* - *LlamaIndex*. (n.d.).
https://docs.llamaindex.ai/en/stable/module_guides/indexing/metadata_extraction/



Anexo A: Extractos representativos del código:

Este apartado recoge una selección de extractos reales del código desarrollado durante el proyecto, con el propósito de ilustrar de forma clara y estructurada cómo se han abordado algunas de las tareas clave en el sistema. No se trata de un análisis exhaustivo de la implementación, sino de mostrar ejemplos prácticos que ayuden a comprender la arquitectura, el flujo de ejecución y las herramientas empleadas.

A lo largo del apartado se presentan extractos que muestran:

- cómo se define la configuración de un workflow de recuperación, incluyendo la selección de recuperadores, sintetizadores, almacenes vectoriales y modelos de embeddings, y como elegir cual usar
- cómo se inicializan dinámicamente los componentes mediante estructuras del tipo `from_config`, almacenando cada elemento como atributo de la instancia
- cómo se estructura una etapa del workflow a través de eventos tipados, accediendo a entradas y generando salidas mediante agentes LLM
- cómo se ejecutan, en paralelo y de forma asíncrona, los procesos centrales de recuperación, síntesis de respuestas y evaluación de fidelidad.

Adicionalmente, se incluye un extracto dedicado a ejemplos de prompts utilizados por algunos de los agentes definidos en el sistema. Estos ejemplos permiten observar las estrategias de “prompt engineering” aplicadas en la interacción con modelos de lenguaje, la estructura de los mensajes y los objetivos comunicativos específicos buscados.

La inclusión de estos fragmentos permite visualizar el enfoque modular adoptado, así como el uso práctico de conceptos como asincronía, paralelismo, arquitectura basada en eventos y agentes LLM, todos integrados dentro del framework LlamaIndex. Además, refuerza la comprensión de la solución técnica presentada sin necesidad de reproducir el código completo, facilitando su revisión y evaluación por parte del lector.

Extracto 1: Configuración del workflow de recuperación

El bloque de configuración se estructura a través de la clave "settings", la cual agrupa las distintas herramientas utilizadas en el flujo. A continuación se presenta un ejemplo simplificado:


```

workflow_config = {
  > "agents": { ...
    "settings": {
      "retriever": [
        {
          "cls": "VectorRetriever",
          "params": {
            "similarity_top_k": 5,
          }
        },
      ],
      "synthesizer": [
        {"cls": "response_synthesizer_agent", "params": {}},
      ],
      'vector_store': {
        'cls': 'MongoDBAtlasVectorSearch',
        'params': {
          'db_name': 'test-dev',
          'collection_name': 'knowledge_index',
          'vector_index_name': 'vector_index',
          'id_key': '_id'
        },
      },
      "embedding_model": {"cls": "OpenAIEmbedding", "params": {"model": "text-embedding-3-large"}},
    },
  }
}

```

Figura 11. Ejemplo simplificado de configuración de un workflow de recuperación

La **Figura 11** muestra un ejemplo simplificado de cómo se define la configuración de los principales componentes utilizados en un workflow de recuperación. En ella se observa cómo se especifican, dentro del bloque "settings", los distintos módulos que componen el sistema: los recuperadores (*retriever*), los sintetizadores (*synthesizer*), el vector store (*vector_store*) y el modelo de embeddings ("embedding_model"). Cada uno de estos elementos se declara a través de una estructura basada en clase y parámetros, lo que permite una inicialización flexible y adaptable a distintos contextos. Esta configuración modular facilita la incorporación de diferentes estrategias de recuperación, síntesis y almacenamiento, según las necesidades del caso de uso.

► Selección de retrievers

El campo "retriever" es una lista de objetos, cada uno representando un recuperador de información que será utilizado por el sistema. La clave "cls" especifica el nombre del recuperador que se desea utilizar (*VectorRetriever*, *RecursiveRetriever*, *HierarchicalRetriever*), y "params" incluye los parámetros necesarios para su funcionamiento. Esta estructura permite añadir múltiples recuperadores, lo que facilita estrategias de recuperación combinada (por ejemplo, recuperación semántica y jerárquica).

► Selección de synthesizers

En "synthesizer" se define la lista de sintetizadores encargados de generar las respuestas finales a partir de los fragmentos recuperados. Al igual que en el caso de los recuperadores, cada sintetizador se configura indicando la clase a utilizar y los parámetros correspondientes. Esta sección admite sintetizadores simples, basados en reglas, o agentes inteligentes capaces de analizar y consolidar múltiples respuestas.

► Configuración del `vector_store`

El campo `vector_store` permite especificar el sistema utilizado para almacenar y consultar los vectores que representan los fragmentos indexados. Esta configuración es esencial para conectar con la base de datos subyacente, ya sea una solución local o un servicio cloud, y asegurar la correcta recuperación de la información durante el proceso. La estructura de configuración sigue el mismo esquema basado en clase y parámetros, donde en la clase especificamos cual usaremos (MongoDB, ChromaDB, etc..) y en los parámetros especificaremos el nombre de la database a usar y el nombre de la colección dentro de esta database

► Selección del modelo de `embeddings`

Finalmente, el campo `embedding_model` define el modelo encargado de convertir los textos en representaciones vectoriales. Este modelo se utilizará tanto durante la indexación como durante la recuperación para calcular la similitud semántica entre las consultas y los documentos. La estructura de configuración sigue el mismo esquema basado en clase y parámetros, donde en la clase especificamos cual usaremos (OpenAIEmbedding, HuggingFaceEmbedding, etc..).

Extracto 2: Inicialización dinámica del workflow mediante `from_config`

Este fragmento muestra una versión simplificada del método `from_config`, utilizado para inicializar automáticamente los distintos componentes del workflow a partir de una configuración externa. A través de esta función, se cargan los modelos, agentes y herramientas definidos en la configuración, y se asignan como atributos de la instancia del workflow. De esta forma, todos los elementos necesarios quedan preparados para ser utilizados posteriormente en las distintas etapas del flujo de recuperación.

```

# Fragmento parcial del método from_config (simplificado para su presentación)
@classmethod
def from_config(cls, workflow_config: dict, *, db: Database = None) -> "RAGWorkflow":
    # Cargar configuración
    settings = workflow_config["settings"]
    agents = workflow_config["agents"]

    # Inicializar modelos y componentes según configuración
    vector_store = init_vector_store(settings["vector_store"])
    retrievers = [r for r in settings["retriever"]]
    embedding_model = init_embed_model(settings["embedding_model"])
    llm = init_llm(settings["llm"])
    synthesizers = [r for r in settings["synthesizer"]]
    # ... (se omiten otros elementos por brevedad)

    # Inicializar los agentes inteligentes
    query_rewriter = init_agent(agents.get("query_rewriter"))
    response_synthesizer = init_agent(agents.get("response_synthesizer"))
    # ... (otros agentes omitidos por brevedad)

    # Devolver la instancia del workflow con los componentes inicializados
    return cls(
        query_rewriter=query_rewriter,
        retrievers=retrievers,
        vector_store=vector_store,
        response_synthesizer=response_synthesizer,
        synthesizers=synthesizers,
        embedding_model=embedding_model,
        llm=llm,
        # ... otros componentes omitidos por brevedad
    )

```

Figura 12. Inicialización del workflow a partir de una configuración usando el metodo `from_config`

Como se puede ver en la **Figura 12**, este método permite extraer directamente los elementos definidos en el bloque de configuración, inicializarlos de forma dinámica y devolver una instancia del workflow completamente preparada para ser utilizada. Esto facilita que todo el sistema sea reutilizable, modular y fácilmente adaptable a diferentes contextos sin modificar el código base, simplemente cambiando los parámetros de configuración.

Extracto 3: Definición de etapas en el workflow mediante eventos y agentes

```
@step(pass_context=True)
async def rewrite_query(self, ctx: Context, ev: StartEvent | ReviewQuestionEvent) -> ExpandedQueryEvent:
    query = ev.get("query")
    await ctx.set("users_query", query)

    if self.query_rewriter is None:
        await ctx.set("generated_queries", [query])
        return ExpandedQueryEvent(generated_queries=[query])

    query_rewriter = self.query_rewriter
    query_rewriter_prompt = query_rewriter._prefix_messages[0].content
    formatted_prompt = query_rewriter_prompt.format(query=query)
    evaluation = query_rewriter.chat(formatted_prompt)
    evaluation_response = json.loads(evaluation.response)
    generated_queries = evaluation_response.get("generated_queries", [])
    await ctx.set("generated_queries", generated_queries)
    return ExpandedQueryEvent(generated_queries=generated_queries)
```

Figura 13. Ejemplo de definición de etapa en un workflow

Este fragmento de código en la **Figura 13** corresponde a una etapa real del workflow desarrollada para el proyecto. Aunque no se presenta con fines de análisis detallado, sirve como ejemplo ilustrativo para mostrar cómo se define y estructura una etapa típica dentro del flujo de trabajo basado en eventos.

En él se puede observar cómo se:

- Declara una función decorada como etapa mediante `@step`, indicando los tipos de eventos que puede recibir (`StartEvent` o `ReviewQuestionEvent`) y el tipo de evento que devolverá `ExpandedQueryEvent`.
- Accede a datos del evento recibido utilizando `ev.get(...)`, lo cual permite extraer información dinámica pasada desde etapas anteriores.
- Accede a componentes previamente inicializados en `from_config` a través de `self.<nombre>`, como por ejemplo `"self.query_rewriter"`.
- Interactúa con un agente LLM utilizando el método `.chat(...)`, al que se le pasa un prompt previamente formateado.
- Procesa la respuesta estructurada del agente, interpretando el JSON devuelto para extraer los datos necesarios.
- Guarda variables intermedias en el contexto del workflow mediante `ctx.set(...)`, lo que permite su reutilización por etapas posteriores.
- Devuelve un nuevo evento tipado al final de la ejecución, que contiene la información generada y permite continuar el flujo de forma trazable.

Este ejemplo pone de manifiesto el modelo de ejecución modular por pasos (steps), característico de la arquitectura basada en LlamaIndex, y cómo cada etapa se apoya en los agentes y configuraciones definidos inicialmente para llevar a cabo su tarea específica.

Extracto 4: Recuperación, síntesis y evaluación de respuestas

El siguiente fragmento muestra, a través de tres bloques diferenciados, cómo se realizan las tres operaciones centrales en el flujo de recuperación: la búsqueda de información, la generación de respuestas y la evaluación de su fidelidad. Cada proceso se ejecuta en paralelo para mejorar la eficiencia y se organiza en tareas asincrónicas que permiten escalar el sistema sin comprometer el tiempo de respuesta.

► Recuperación de nodos

En el primer bloque representado en la **Figura 14** se inicializa una lista de tareas de recuperación, donde se llama al método “*aretrieve*” del recuperador para cada consulta generada. Estas tareas se ejecutan de forma concurrente utilizando `asyncio.gather`, y los nodos resultantes se consolidan en una lista eliminando duplicados por ID. Esta estrategia permite obtener, de forma eficiente, los fragmentos de texto relevantes que luego serán utilizados para sintetizar la respuesta.

```
retrieval_tasks = [
    retriever.aretrieve(query) for query in generated_queries
]
results = await asyncio.gather(*retrieval_tasks)
total_nodes = list(
    {
        node.node.id_: node for sublist in results for node in sublist
    }.values()
)
```

Figura 14. Ejecución paralela de tareas de recuperación de nodos

► Síntesis de respuestas

El segundo bloque representado en la **Figura 15** se encarga de generar las respuestas correspondientes a cada consulta. Para ello, se crean tareas asincrónicas con el método “*asynthesize*”, que toma como entrada tanto la consulta del usuario junto con las generadas por el agente como los nodos recuperados, así nos permite generar una respuesta por cada pregunta. El uso de “*await asyncio.gather*” permite que todas las respuestas se procesen en paralelo.

```
response_tasks = [
    normal_synthesizer.asynthesize(query=query, nodes=ev.all_nodes)
    for query in queries
]
response_list = await asyncio.gather(*response_tasks)
```

Figura 15. Ejecución paralela de tareas de generación de respuestas

► Evaluación de fidelidad

Por último, el tercer bloque representado en la **Figura 16** muestra cómo se filtran las respuestas mediante un agente evaluador. A cada combinación de consulta y respuesta se le aplica un proceso de evaluación utilizando el método “*evaluate_response*” del evaluador. Este método recibe como parámetros la pregunta del usuario junto con las generadas por el agente y una respuesta generada para cada una de estas preguntas, y devuelve un objeto que incluye, entre otros atributos, el campo *passing*, que indica si la respuesta supera o no los criterios definidos. Solo aquellas respuestas cuya evaluación devuelve *passing = True* son conservadas como candidatas válidas para los pasos posteriores del workflow.

```
evaluator = self.faithfulness_evaluator
final_responses = []
for query in queries:
    for response in responses_list:
        eval_result = evaluator.evaluate_response(
            query=query, response=response
        )
        if eval_result.passing:
            if response not in final_responses:
                final_responses.append(response.response)
```

Figura 16. Evaluación de fidelidad de respuestas generadas

Extracto 5: Diseño y uso de prompts en el sistema

A lo largo del desarrollo del sistema, se han utilizado múltiples agentes LLM con funciones diversas, desde la reformulación de consultas hasta la síntesis de respuestas o la evaluación de fidelidad. A pesar de las diferencias en sus objetivos, todos estos agentes comparten un enfoque común en cuanto al diseño de sus instrucciones (*prompts*). Esta sección muestra cómo se han definido estas instrucciones para tres de los agentes más representativos del sistema, con el objetivo de ilustrar la arquitectura general seguida.

El diseño de los prompts parte de una estructura clara, compuesta por:

- Un papel asignado explícitamente al agente (por ejemplo, evaluador, sintetizador o reformulador).
- Una serie de reglas detalladas que guían su comportamiento.
- Un formato de salida estructurado, cuando corresponde, que permite interpretar los resultados del agente en los pasos posteriores del flujo de trabajo.

A continuación, se presentan tres ejemplos reales utilizados en el sistema.

► Agente sintetizador de respuestas

```
response_synthesizer_prompt= """

You are a QA assistant that provides recommendations, examples, and instructions for
the users query.

Always answer the query using the provided context information, and not prior
knowledge.

Some rules to follow:

1. Always include reference to a source of the information.

Include a numbered reference in square brackets in the answer text,
and a list of used references (for example doc_name or file_name, etc..) at the end.

2. If asked for a code example, always include it in a code block using the format:

```python
"""
```

**Figura 17. Prompt del agente sintetizador de respuestas (response\_synthesizer)**

La **Figura 17** muestra el prompt utilizado por el agente encargado de sintetizar respuestas a partir de los nodos recuperados. El contenido del prompt define el papel del agente como asistente técnico en sistemas de preguntas y respuestas, con una orientación clara hacia la generación de respuestas completas, basadas únicamente en el contexto proporcionado. Se exige que toda respuesta contenga referencias explícitas a las fuentes utilizadas, tanto dentro del texto como al final de la respuesta, y que se incluyan ejemplos de código cuando el contenido lo requiera. Asimismo, si la pregunta está relacionada con la creación o uso de herramientas o equipos, el agente debe proporcionar fragmentos de configuración o ejemplos ejecutables, lo que refuerza la utilidad práctica de la respuesta. El objetivo es asegurar que el resultado generado sea accionable, coherente y directamente aplicable por parte del usuario, priorizando siempre la precisión y la trazabilidad.

## ► Agente reformulador de consultas

```
query_rewriter_prompt="""

As an AI query analyzer and improver, your task is to perform a detailed analysis and
enhancement

of the user's original query within a Retrieval-Augmented Generation (RAG) system.

Your responsibilities include:

1. Analyzing the query (when it is clear):

 - Provide a brief and concise analysis of the intent and possible scope.

 - Suggest what a reasonable response could look like.

2. Generating new related queries:

 - Create up to four total queries, starting with the original one.
```

```
- These must be meaningful variations or expansions.

3. Improving the original query:

 - Rewrite the query to make it more specific and unambiguous.

Here is the original query: {query}

Please provide the following:

1. A brief analysis of the query.
2. An improvised response to the query.
3. A list of up to four new related queries with the original query included.

These queries should be in the same language as the original query.

If the input query is nonsensical, incomplete, or clearly not a real question, do not
generate new related queries,

as they would not be meaningful and the returned list should be a list with only the
original query [{query}]

4. An improved version of the original query.

Your responses should be clear, concise, and oriented toward improving the relevance
of the retrieval process.

IMPORTANT:

 - All queries should be in the same language as the original query.

"""
```

**Figura 18. Prompt del agente reformulador de consultas (query\_rewriter)**

En la **Figura 18** se representa el prompt empleado por el agente cuya función consiste en analizar y mejorar las consultas introducidas por el usuario. El prompt describe un proceso en el que el agente evalúa primero la claridad de la consulta y, si esta es comprensible, realiza un análisis de su intención, propone una posible respuesta, genera hasta cuatro versiones o expansiones relacionadas de la misma y finalmente ofrece una reformulación más específica y desambigua. En caso de que la consulta no tenga sentido o esté incompleta, el agente debe abstenerse de producir variantes irrelevantes y devolver únicamente la original. Esta estrategia permite mejorar la efectividad del sistema de recuperación sin introducir ruido innecesario, facilitando respuestas más precisas y alineadas con la intención del usuario.

#### ► Agente evaluador de respuestas

```
evaluator_agent_prompt = """\

Host Assistant - RAG Response Evaluation Prompt

You are a technical evaluator assessing the quality of an AI-generated response
designed to assist developers with tasks involving teams and tools within a codebase.
```



```

Given the user query, the system's response, and the retrieved documents (sources),
evaluate the answer across the following dimensions.

Assign a score between 0.0 and 1.0 for each, and provide a clear justification.

1. Relevance (0-1)

Definition:

Does the response directly and effectively address the user's query? Is it focused
and aligned with the user's intent?

Important Rules:

- Do not score based on the correctness of the response. The focus is on
relevance to the query.

- Do not score based on the correctness of the sources. Only take into account
the response and the query.

Scoring Guidelines:

- 1.0 (Fully Relevant): The response clearly and accurately answers the query.
- 0.75: Mostly relevant, with minor digressions or unnecessary content.
- 0.5: Addresses the query partially, missing key elements or including unrelated
info.
- 0.25: Only loosely connected to the query.
- 0.0 (Not Relevant): Unrelated to the user's intent or question.

2. Completeness (0-1)

Definition:

Does the response cover all essential aspects of the question, including config/code
snippets if required?

Important Rules:

- Do not score based on the correctness of the response. The focus is on
relevance to the query.

- Do not score based on the correctness of the sources. Only take into account
the response and the query.

Scoring Guidelines:

```



```
- **1.0 (Fully Complete):** Covers all required steps, details, and provides code/config if relevant.

- **0.75:** Mostly complete with minor omissions.

- **0.5:** Some required components or examples are missing.

- **0.25:** Very partial; provides a limited or vague answer.

- **0.0 (Not Complete):** Misses the main point or fails to provide any actionable help.
```

### ### 3. Coherence (0-1)

#### \*\*Definition:\*\*

Is the response well-structured, clearly written, and appropriate in tone for a technical developer audience?

#### \*\*Important Rules:\*\*

- Do **not** score based on the correctness of the response. The focus is on relevance to the query.
- Do **not** score based on the correctness of the sources. Only take into account the response and the query.

#### \*\*Scoring Guidelines:\*\*

- \*\*1.0 (Excellent):\*\* Logically organized, precise, with a clear and professional tone.
- \*\*0.75:\*\* Generally clear, with minor stylistic or structural issues.
- \*\*0.5:\*\* Understandable but includes disorganized or awkward phrasing.
- \*\*0.25:\*\* Poorly structured, making comprehension difficult.
- \*\*0.0 (Incoherent):\*\* Disorganized, unclear, or inappropriate in tone/style.

### ### 4. Source Logic (0-1)

#### \*\*Definition:\*\*

Are the provided source texts logically relevant to the user's query?

You must **analyze the content** of each source provided under the `sources` dictionary.

These are key-value pairs where the key is an identifier (e.g., "source1") and the value is the actual content (text) of that source.

```

Important Rules:

- Do **not** assume the content based on the source ID or name.

- Do **not** assume that the source is relevant just because the response mentions similar concepts.

- Only score sources as relevant if their **actual text** includes concepts, instructions, structures, or examples that relate to the query.

Scoring Guidelines:

- **1.0 (Fully Logical):** All source texts are clearly relevant to the query.

- **0.75:** Most sources align with the query, with minor misfits.

- **0.5:** Half the sources are relevant; others are unrelated or generic.

- **0.25:** Very few sources relate to the query's needs.

- **0.0 (Not Logical):** None of the source texts are meaningfully related to the query.

-

5. Source Usage (0-1)

Definition:

Does the response clearly reflect or use information from the provided source texts?

Evaluate whether the response **draws content, terminology, structure, or logic directly from the source texts** provided in the `sources` dictionary. Do not score based on whether the response references a source ID – only the actual usage of the source text matters.

Important Rules:

- Do **not** assume the response is based on sources just because it includes known technical terms.

- Do **not** accept placeholder references (e.g., [1], [sourceX]) as proof of usage.

- Only score positively if the response reuses or clearly reflects the **ideas or content** in the actual source texts.

Scoring Guidelines:

- **1.0 (Fully Based on Sources):** The response is fully derived from and grounded in the sources.

- **0.75:** Mostly reflects the sources, with minor additions.

- **0.5:** Some grounding, but significant unsupported elements.

- **0.25:** Minimal use of source content.

```

```
- **0.0 (Not Based on Sources):** No meaningful connection to source texts; response appears fabricated.

'''
```

**Figura 19. Prompt del agente evaluador de respuestas (evaluator\_agent)**

Por último, la **Figura 19** ilustra el prompt utilizado por el agente evaluador, que tiene como finalidad valorar la calidad de las respuestas generadas por el sistema RAG. Este prompt define un marco de evaluación estructurado en torno a cinco dimensiones fundamentales: la relevancia de la respuesta con respecto a la consulta, su nivel de completitud, la coherencia del discurso técnico, la lógica de los documentos fuente recuperados y la medida en que la respuesta hace uso efectivo del contenido de esas fuentes. Para cada dimensión, el agente debe asignar una puntuación entre 0.0 y 1.0 y acompañarla de una justificación clara. Esta estructura garantiza evaluaciones objetivas, consistentes y repetibles, permitiendo no solo validar la calidad de las respuestas generadas, sino también identificar oportunidades de mejora en los diferentes componentes del sistema.

# Anexo B: Objetivos de desarrollo sostenible (ODS)

Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible (ODS).

Objetivos de Desarrollo Sostenibles	Alto	Medio	Bajo	No Procede
ODS 1. <b>Fin de la pobreza.</b>				X
ODS 2. <b>Hambre cero.</b>				X
ODS 3. <b>Salud y bienestar.</b>				X
ODS 4. <b>Educación de calidad.</b>		X		
ODS 5. <b>Igualdad de género.</b>				X
ODS 6. <b>Agua limpia y saneamiento.</b>				X
ODS 7. <b>Energía asequible y no contaminante.</b>				X
ODS 8. <b>Trabajo decente y crecimiento económico.</b>	X			
ODS 9. <b>Industria, innovación e infraestructuras.</b>	X			
ODS 10. <b>Reducción de las desigualdades.</b>				X
ODS 11. <b>Ciudades y comunidades sostenibles.</b>				X
ODS 12. <b>Producción y consumo responsables.</b>		X		
ODS 13. <b>Acción por el clima.</b>				X
ODS 14. <b>Vida submarina.</b>				X
ODS 15. <b>Vida de ecosistemas terrestres.</b>				X
ODS 16. <b>Paz, justicia e instituciones sólidas.</b>				X
ODS 17. <b>Alianzas para lograr objetivos.</b>		X		

**Tabla 10: Tabla relación con ODS**

El Trabajo Fin de Grado desarrollado se alinea con diversos Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030, especialmente aquellos vinculados con la innovación tecnológica, la eficiencia organizativa, la sostenibilidad digital y la mejora del acceso al conocimiento en entornos empresariales.

En este sentido, el ODS 4 (Educación de calidad) se ve reflejado en la capacidad del sistema para organizar y recuperar grandes volúmenes de información de forma estructurada y trazable. Esta funcionalidad no solo facilita la toma de decisiones informadas, sino que también puede ser aplicada en contextos de formación continua o capacitación profesional, promoviendo entornos de aprendizaje más accesibles y eficaces.

Asimismo, el ODS 8 (Trabajo decente y crecimiento económico) cobra relevancia a través de la automatización de tareas rutinarias vinculadas a la búsqueda, clasificación y gestión del conocimiento. Este enfoque permite optimizar el uso del tiempo y los recursos humanos dentro de las organizaciones, impulsando entornos laborales más productivos, sostenibles y centrados en actividades de mayor valor añadido.

En cuanto al ODS 9 (Industria, innovación e infraestructura), el proyecto destaca por su apuesta por la modernización digital mediante la implementación de workflows inteligentes basados en tecnologías avanzadas como los sistemas RAG y los agentes inteligentes. Estas soluciones no solo mejoran los procesos internos de las empresas, sino que también promueven el desarrollo de infraestructuras digitales más ágiles, trazables y resilientes.

Por otro lado, el ODS 12 (Producción y consumo responsables) se materializa en la digitalización de procesos documentales que tradicionalmente requerían un uso intensivo de papel u otros recursos físicos. Al automatizar y centralizar la gestión de información, se evita la duplicación innecesaria de datos y se fomenta un uso más consciente y sostenible de los recursos tecnológicos disponibles.

Finalmente, el ODS 17 (Alianzas para lograr los objetivos) está presente en la integración de múltiples tecnologías y servicios externos como APIs empresariales, bases de datos estructuradas o plataformas en la nube que permiten extender la funcionalidad del sistema y adaptarlo a diferentes contextos de uso. Esta capacidad de interoperar y colaborar con otras herramientas refuerza la escalabilidad del sistema y su aplicabilidad en entornos reales y heterogéneos.

En conjunto, este proyecto pone de manifiesto cómo la aplicación estratégica de la inteligencia artificial, los workflows automatizados y la integración de sistemas heterogéneos puede contribuir simultáneamente a mejorar la eficiencia operativa de las empresas y a avanzar en varios de los grandes retos globales definidos por la Agenda 2030.

# Glosario:

---

- **LLM (Large Language Model):** Modelo de lenguaje de gran tamaño entrenado con grandes volúmenes de texto para generar o comprender lenguaje natural.
- **RAG (Retrieval-Augmented Generation):** Arquitectura que combina recuperación de información y generación de texto con modelos de lenguaje. Permite que un LLM acceda a información externa antes de responder.
- **Query (Consulta):** Pregunta o instrucción formulada por el usuario para obtener información del sistema.
- **Nodo:** Unidad de información dentro del sistema que contiene un fragmento de texto y metadatos asociados. Puede ser el resultado de dividir un documento.
- **Embedding:** Representación vectorial de un texto que captura su significado semántico, utilizada para comparar y recuperar información de forma eficiente.
- **Chunking (Segmentación):** Proceso de dividir un documento en fragmentos más pequeños ("chunks") para facilitar su indexación y posterior recuperación.
- **Parser (Analizador):** Componente encargado de leer y transformar documentos en estructuras que puedan ser procesadas por el sistema, como nodos o árboles de información.
- **Prompt (Instrucción de entrada):** Texto cuidadosamente diseñado y enviado a un modelo de lenguaje para obtener una respuesta específica. Su redacción influye directamente en la calidad de la respuesta generada.
- **Workflow:** Flujo estructurado de pasos o tareas automatizadas, conectado mediante eventos, que define cómo se procesa o responde una consulta en el sistema.
- **Human-In-The-Loop:** Mecanismo dentro de un sistema automatizado que permite la intervención de una persona cuando no hay suficiente información, el contexto es ambiguo, o se requiere validación humana para continuar el flujo de trabajo.
- **Cron job:** Tarea programada que se ejecuta automáticamente en intervalos regulares definidos por el usuario, comúnmente utilizada en sistemas Unix/Linux para automatizar procesos como indexación periódica o backups.
- **Webhook:** Punto de conexión HTTP que permite recibir datos o activar acciones automáticamente cuando ocurre un evento externo, como la llegada de un nuevo documento a una carpeta o una respuesta en Slack.
- **Tool (Herramienta):** Componente modular dentro de un workflow encargado de realizar una tarea específica (como extracción de metadatos o síntesis de respuestas).



- **Agente inteligente:** Componente que toma decisiones o ejecuta tareas complejas de forma autónoma en base al contexto y a la interacción con modelos de lenguaje.
- **Retriever (Recuperador):** Elemento encargado de buscar y seleccionar la información más relevante ante una consulta del usuario.
- **Metadatos:** Información adicional sobre un fragmento de texto, como su origen, fecha, título, categoría, etc., que permite filtrarlo o clasificarlo.
- **Prompt engineering:** Técnica que consiste en diseñar cuidadosamente los mensajes (prompts) enviados a un modelo de lenguaje para obtener respuestas útiles y coherentes.
- **Cohere (Reranker):** Herramienta basada en modelos de lenguaje que reordena los resultados recuperados según su relevancia semántica respecto a una consulta.
- **API (Application Programming Interface):** Conjunto de reglas que permite que una aplicación se comuniquen con otra, por ejemplo, para recuperar datos o automatizar tareas.
- **Chunk ID / Document ID / Index ID:** Identificadores únicos asignados a fragmentos de texto (chunks), documentos completos o índices, usados para organizar la recuperación.